

Desarrollo de Aplicaciones JSE

Desarrollo de Aplicaciones JSE

Profesor: Ana Isabel Vegas

INDICE

INDICE 3
1. INTRODUCCIÓN A JAVA 7
JRE y JDK 9
EDICIONES JAVA 10
GARBAGE COLLECTOR 10
CARACTERÍSTICAS DE JAVA 11
2.- CLASES Y OBJETOS 12
DECLARACION DE CLASES 12
DECLARACION DE PROPIEDADES 13
DECLARACION DE METODOS 14
DECLARACION DE CONSTRUCTORES 15
CREACIÓN DE OBJETOS Y ADMINISTRACIÓN DE MEMORIA 16
ACCESO A MIEMBROS 18
ENCAPSULACION 18
3.- IDENTIFICADORES, PALABRAS CLAVE Y TIPOS 22
REGLAS PARA NOMBRAR UN IDENTIFICADOR 22
PALABRAS CLAVE 22
TIPOS PRIMITIVOS 23
4.- OPERADORES Y CONTROL DE FLUJO 25
OPERADORES MATEMÁTICOS 25
OPERADORES INCREMENTO Y DECREMENTO 25
OPERADORES BIT A BIT 25
OPERADORES DE DESPLAZAMIENTO 26
OPERADORES LÓGICOS 27
OPERADORES RELACIONALES 27
CONDICIONAL IF -ELSE 27
SWITCH-CASE 28
BUCLE FOR 30
BUCLE FOR - EACH 31
BUCLE WHILE 31
BUCLE DO-WHILE 32

SENTENCIAS BREAK Y CONTINUE....32
5.- ARRAYS....34
UNA DIMENSIÓN....34
VARIAS DIMENSIONES....39
6.- CLASES AVANZADAS....44
HERENCIA....44
SOBREESCRITURA DE MÉTODOS....46
SOBRECARGA DE MÉTODOS....49
SOBRECARGA DE CONSTRUCTORES....50
LA CLASE OBJECT....52
CLASES ABSTRACTAS....58
INTERFACES....62
POLIMORFISMO....67
CLASES ENVOLVENTES....73
AUTOBOXING....74
RECURSOS ESTÁTICOS....75
PALABRA CLAVE FINAL....79
TIPOS ENUMERADOS....81
CONSULTA API....84
7.- EXCEPCIONES Y ASERCIONES....87
CAPTURAR EXCEPCIONES....87
MANEJO DE EXCEPCIONES MÚLTIPLES....90
API EXCEPTION....90
PROPAGAR EXCEPCIONES....91
EXCEPCIONES PERSONALIZADAS....92
GESTIÓN DE ASERCIONES....93
BUENAS PRÁCTICAS CON ASERCIONES....94
HABILITAR Y DESHABILITAR ASERCIONES....94
8.- COLECCIONES Y GENERICOS....96
API COLLECTIONS....96
PRINCIPALES COLECCIONES....97
SET....98
LIST....98
MAP....99

ORDENAR COLECCIONES....100
INTERFACE COMPARABLE....100
INTERFACE COMPARATOR....103
GENÉRICOS....105
9.- MECANISMOS DE ENTRADA / SALIDA....110
ENTRADA DE ARGUMENTOS....110
PROPIEDADES....110
FUNDAMENTOS DE ENTRADA Y SALIDA....112
FLUJOS DE BYTES....112
FLUJOS DE CARACTERES....113
CLASES IMPORTANTES....114
PROCESAR STREAMS....116
JERARQUIA DE CLASES INPUTSTREAM....118
JERARQUIA DE CLASES OUTPUTSTREAM....118
JERARQUIA DE CLASES READER....118
JERARQUIA DE CLASES WRITER....119
SERIALIZACIÓN....119
ENTRADA Y SALIDA DE CONSOLA....121
ENTRADA Y SALIDA DE ARCHIVOS....124
CIERRE DE RECURSOS....127
10.- INTERFACE GRÁFICA CON SWING....130
QUE ES SWING....130
CONTENEDORES SWING....131
ADMINISTRADORES DE DISEÑO....136
COMPONENTES SWING....146
CREAR INTERFACES GRAFICAS DE USUARIO....148
EVENTOS....151
CREACIÓN DE MENÚS....158
11.- HILOS....162
FORMAS DE CREAR UN HILO....162
ARRANCAR UN HILO....163
CICLO DE VIDA DE UN HILO....164
PLANIFICAR HILOS....165
SINCRONIZACIÓN....166

ITERACIÓN ENTRE HILOS 168
12.- CONEXIONES EN RED CON SOCKETS 173
CREAR SOCKETS 173
SERVIDOR Y CLIENTE 173
13.- CONEXIÓN CON BASES DE DATOS 177
API JDBC 177
TIPOS DE CONTROLADORES 177
ABRIR Y CERRAR CONEXIONES 180
CREAR CONSULTAS CON SQL 181
PROCESAR DATOS 184
MANEJO DE TRANSACCIONES 185
METADATOS 185
14.- NOVEDADES JAVA 8 187
EXPRESIONES LAMBDA 187
STREAMS 188
REFERENCIAS DE MÉTODOS 188
INTERFACES FUNCIONALES 189
MÉTODOS POR DEFECTO 190
MÉTODOS ESTÁTICOS EN INTERFACES 190
ÍNDICE DE GRÁFICOS 192

1. INTRODUCCIÓN A JAVA

El lenguaje de programación Java es un **lenguaje orientado a objetos** lo que quiere decir que todos los recursos que creamos y administramos los vamos a considerar como objetos. Una factura será un objeto, un cliente será un objeto, una conexión a una base de datos también será considerada como un objeto.

Otra característica importante de Java es que es un **lenguaje independiente de la plataforma**. En otros lenguajes de programación, por ejemplo C, el hecho de crear una aplicación consta de una serie de pasos:

1. Se crea el código fuente de la aplicación
2. Se compila para un determinado Sistema Operativo (Plataforma) para conseguir el archivo binario (0,1)
3. Se linka el código para ese S.O. para obtener el archivo ejecutable (.exe)

El resultado es que si necesitamos que nuestra aplicación esté disponible para varias plataformas hay que generar un archivo ejecutable para cada una de ellas.

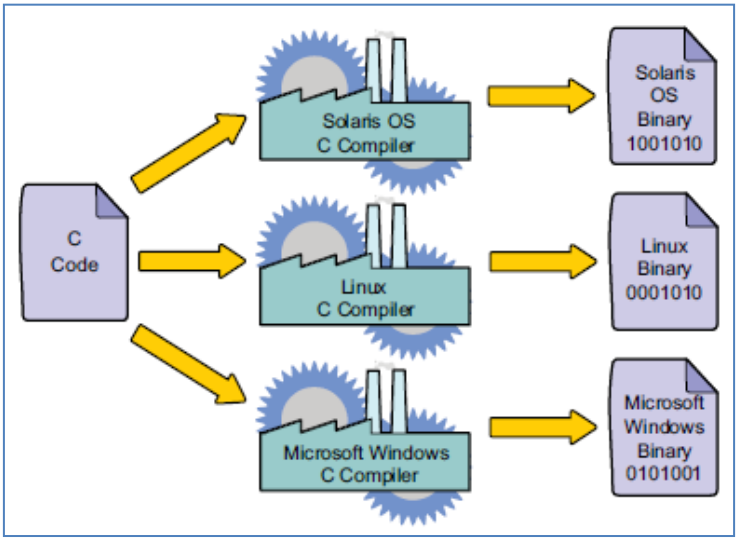


Gráfico 1. El código fuente se compila para cada plataforma

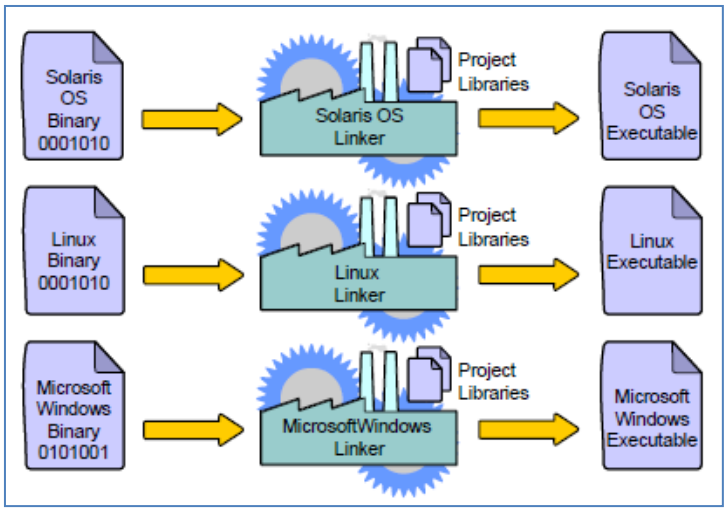


Gráfico 2. Se efectúa el linkado para cada plataforma

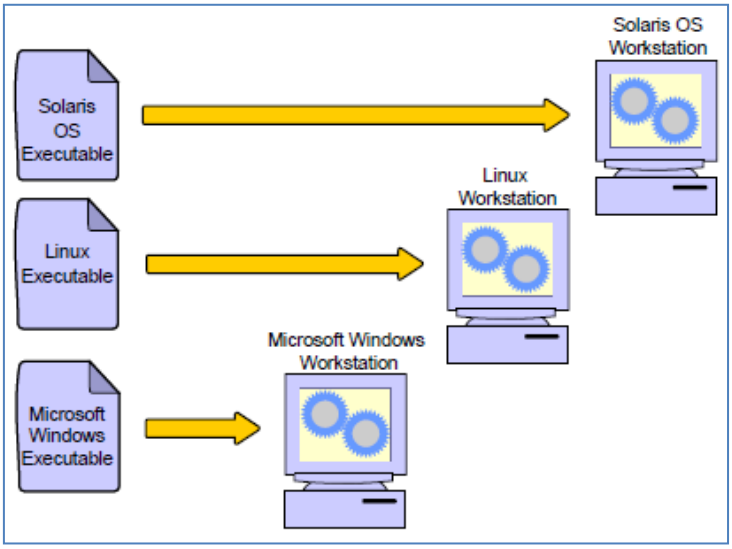


Gráfico 3. El ejecutable se distribuye a cada plataforma

Como decíamos, Java es un lenguaje independiente a la plataforma esto implica una reducción considerable de trabajo a la hora de crear una aplicación.

1. Se crea el código fuente de la aplicación (archivos .java)
2. Se compila para obtener archivos bytecode (.class)
3. No se necesita obtener un ejecutable, ya que los archivos bytecode los interpretara la maquina virtual de java (JVM Java Virtual Machine)

El archivo bytecode obtenido se puede ejecutar en cualquier plataforma, para ello tan solo es necesario tener instalada la maquina virtual de Java adecuada al S.O. instalado.

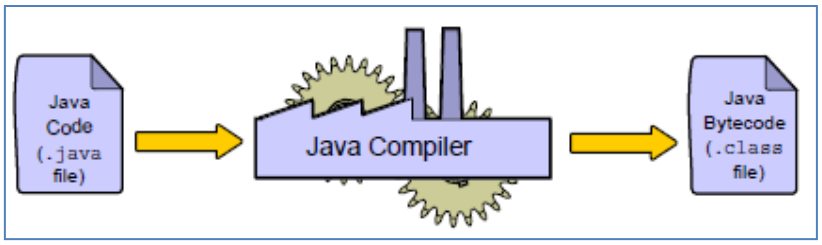


Gráfico 4. El código fuente se compila independiente de la plataforma

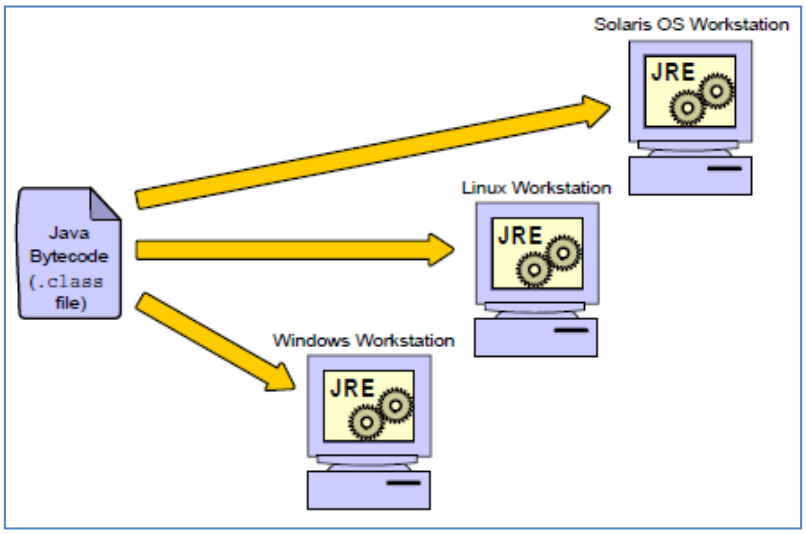


Gráfico 5. El archivo Bytecode puede ser interpretado por distintas plataformas

JRE Y JDK

Necesitamos tener una maquina virtual de Java (JVM) para poder ejecutar nuestras aplicaciones, esta se puede descargar dependiendo del perfil del usuario:

- **JRE (Java Runtime Environment)**; Sería necesario únicamente para ejecutar una aplicación. Esta sería la versión que se deben descargar los clientes, usuarios finales de nuestra aplicación. Incluye únicamente la JVM y un conjunto de librerías para poder ejecutar.
- **JDK (Java Development Kit)**; Estos son los recursos que necesitamos los desarrolladores ya que incluye lo siguiente:
  - JRE
  - Compilador de java
  - Documentación del API (todas las librerías de Java)
  - Otras utilidades por ejemplo para generar archivos .jar, crear documentación, efectuar un debug.
  - También incluye ejemplos de programas.

EDICIONES JAVA

El lenguaje Java se distribuye en tres ediciones:

- **JSE (Java Standard Edition)**; Es la edición más básica de Java pero no menos importante. Recoge los fundamentos básicos del lenguaje pero solo nos permite desarrollar aplicaciones locales, aplicaciones escritorio y applets (aplicaciones que se ejecutan en el navegador del cliente).
- **JEE (Java Enterprise Edition)**; Esta edición es la más completa de todas. Gracias a ella podremos desarrollar aplicaciones empresariales tales como aplicaciones web, aplicaciones eCommerce, aplicaciones eBusiness, ...etc.
- **JME (Java Micro Edition)**; Con esta edición podremos desarrollar aplicaciones para micro dispositivos tales como teléfonos móviles, PDAs, sistemas de navegación, ...etc.

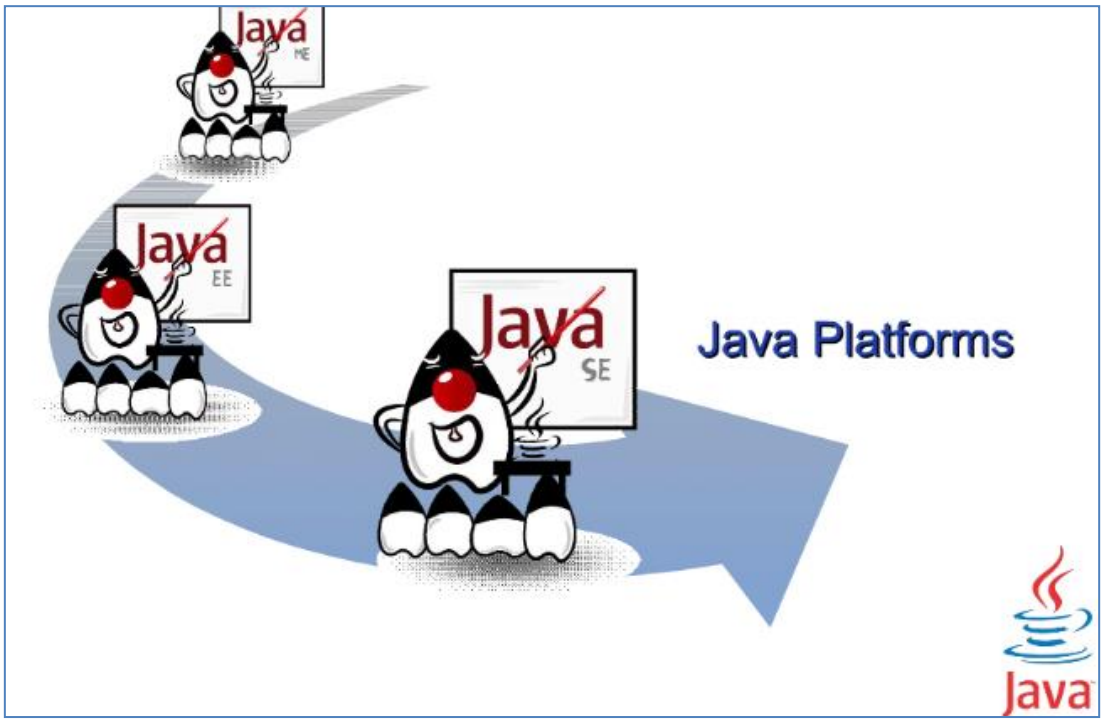


Gráfico 6. Ediciones Java

GARBAGE COLLECTOR

Cada vez que creamos un objeto este reside en la memoria de nuestro equipo. En otros lenguajes es necesario destruir dichos objetos cuando ya no son necesarios.

Java incorpora una ventaja a través de un hilo conocido como Garbage Collector, lo podríamos traducir como el recolector de basura.

Dicho proceso se lanza de vez en cuando de una forma similar a la función de salvar de un editor de texto. Este proceso recorre la memoria localizando aquellos objetos que ya no son necesarios porque no se van a utilizar más y los elimina.

El modo como selecciona dichos objetos lo estudiaremos en capítulos siguientes.

CARACTERÍSTICAS DE JAVA

Resumimos las características principales del lenguaje de programación Java:

- **Orientado a objetos**; como se mencionaba al inicio del capítulo, en Java todo se considera como un objeto. La OO facilita la reutilización de código como se verá más adelante.
- **Distribuido**; Java permite desarrollar aplicaciones distribuidas, esto significa que parte de la aplicación puede alojarse en un servidor de Madrid y otra parte puede residir en otro servidor de Barcelona por ejemplo.
- **Simple**; Aunque os parezca mentira en este momento, Java es un lenguaje muy simple de programar. Además, contamos con muchas librerías ya desarrolladas, listas para utilizarse y lo más importante una buena documentación de uso.
- **Multihilo**; El lenguaje Java funciona mediante hilos no a través de procesos como otros lenguajes. Esto hace que sea más rápida y más segura su ejecución.
- **Seguro**; En el curso iremos viendo las diversas formas de implementar seguridad en aplicaciones Java.
- **Independiente de la plataforma**; como ya comentamos anteriormente.



RECUERDA QUE...

- Java es un lenguaje multiplataforma, por lo cual solo debes instalar el jdk adecuado a tu sistema operativo.
- El lenguaje Java se distribuye en tres ediciones: JSE, JEE, JME.
- No debes preocuparte por eliminar los objetos creados, el Garbage Collector se encarga de esta tarea.

2.- CLASES Y OBJETOS

Un **objeto** es un recurso dinámico que se crea y se almacena en memoria durante un tiempo determinado. A los objetos también se les conoce con el nombre de **instancias** de clase porque realmente esto es lo que son, una copia de la clase con unos valores determinados.

Una **clase** la podríamos definir como la plantilla a partir de la cual se va a generar el objeto.

Como ejemplo, podríamos definir una clase Cliente donde declararíamos todas las propiedades y métodos comunes a todos los clientes. Pues bien, un objeto o instancia de tipo cliente sería un cliente personalizado con su propio Nif, su nombre, su dirección, ...etc.

La clase es el archivo .java para definir un cliente y los objetos son los recursos dinámicos que se generan en la memoria.

DECLARACION DE CLASES

Para declarar una clase nos debemos ajustar a la sintaxis de Java. Dicha sintaxis nos indica que el archivo en el cual declaramos la clase se debe seguir este orden.

1. **Declaración del paquete**; Un paquete cumple una función similar a una carpeta en Windows. El paquete nos sirve para organizar nuestros recursos y además poder implementar niveles de acceso como veremos más adelante.
2. **Importaciones**; En una clase podremos utilizar otras clases ya creadas. Estas pueden ser del propio API o también clases desarrolladas por terceras personas. Para poder acceder a otras clases es necesario importarlas previamente.
3. **Declaración de la clase**; Aquí se definirán los recursos de la clase.

```java
// Declaracion del paquete
package ejemplo1clasesyobjetos;

// importaciones si fuesen necesarias

// Declaracion de la clase
public class Cliente {

    // propiedades
    public String nif;
    public String nombre;
    public String direccion;

    // constructores
    public Cliente() {
    }

    // metodos
    public String mostrarDatos() {
        return "nif=" + nif + " nombre=" + nombre + " direccion=" + direccion;
    }
}
```

Gráfico 7. Ejemplo declaración de clase

Como vemos en el ejemplo, una clase puede tener los siguientes recursos:

Propiedades; Las propiedades o también llamadas atributos o campos recogen las características de la clase. Según nuestro ejemplo un cliente se representa por su nif, nombre y dirección.

Constructores; Los constructores son un tipo de método especifico que se invocará cada vez que vayamos a crear un objeto o instancia de tipo Cliente.

Métodos; Son las acciones que podremos llevar a cabo sobre un cliente. En este caso, mostrar sus datos.

A lo largo del capítulo aprenderemos a crear y gestionar estos recursos.

DECLARACION DE PROPIEDADES

Una propiedad se entiende como una variable global a la cual se puede acceder desde cualquier otro recurso dentro de la misma clase y dependiendo del modo de acceso, también se podrá acceder a ella desde otras clases.

La sintaxis para declarar una propiedad es la siguiente:

acceso tipo nombreVariable;

Ejemplos:

```
public int numero;

private String nombre;

protected double medida;

long capital;
```

DECLARACION DE METODOS

Utilizamos los métodos para introducir el código de nuestra aplicación. Podremos invocar a un método tantas veces como se quiera por lo cual nos permite una reutilización de código para aquellas tareas repetitivas.

Igual que ocurre con las propiedades, un método se puede invocar desde otro recurso declarado en la misma clase y dependiendo de su acceso, se podrá llamar desde otras clases.

La sintaxis para declarar un método es la siguiente:

1.- si el método no devuelve ningún dato:

```
acceso void nombreMetodo(){

}
```

Ejemplos:

```
public void mostrarDatos(){
        // logica de negocio
}

protected void abrirConexion(){
        // logica de negocio
}
```

2.- si el método devuelve un dato:

```
acceso tipoDevuelto nombreMetodo(){
}
```

Ejemplos:

```
public int sumar(int num1, int num2){
    return n1 + n2;
}

public String getNombre(){
    return nombre;
}
```

Como observamos en los ejemplos, cuando un método devuelve un dato incluimos la clausula **return**. Esta instrucción lo que hace es devolver un dato a la sentencia que ha invocado al método.

En el ejemplo del método sumar, se han incluido dos argumentos en el método. Los argumentos son los valores que recibe el método para procesarlos. Los argumentos no son obligatorios, es decir, hay métodos que declaran argumentos y métodos que no. En este último caso los paréntesis siguen siendo obligatorios.

También destacamos que todo el cuerpo del método (lógica de negocio) va encerrado entre llaves.

DECLARACION DE CONSTRUCTORES

Un constructor lo hemos definido como una especie de método que se invoca a la hora de crear un objeto.

La sintaxis para definir un constructor es diferente a la del método ya que nunca puede devolver un dato y tampoco se declara como tipo void.

```
acceso nombreClase(){
}
```

El constructor debe tener obligatoriamente el mismo nombre que la clase. También puede ser que reciba argumentos o no. Estos no son obligatorios al igual que los métodos.

Ejemplos:

```
public Cliente(){

}


public Cliente(String nif, String nombre, String direccion){
        // Asignamos los valores recibidos a las propiedades del objeto
        this.nif = nif;
        this.nombre = nombre;
        this.direccion = dirección;
}
```

CREACIÓN DE OBJETOS Y ADMINISTRACIÓN DE MEMORIA

A la hora de crear un objeto utilizamos la palabra new seguida del constructor.

Ejemplo: new Cliente();

Por definición, en Java, los objetos son anónimos, que significa realmente esto?

Cuando creamos una variable siempre le asignamos un nombre de esta forma podemos acceder a dicha variable a través de dicho nombre. Pues bien, cuando creamos un objeto como hemos hecho en el ejemplo anterior, no asignamos un nombre al objeto por lo cual lo único que se sabe es la dirección de memoria donde ha sido generado.

De esta forma no podremos acceder a dicho objeto. Para solucionar este problema vamos a crear una variable del tipo de la clase que vamos a instanciar y almacenaremos en ella la dirección de memoria del objeto para que nos sirva de referencia.

Ejemplo: Cliente c = new Cliente();

De esta forma ahora podremos acceder al objeto a través de la variable c.

La memoria se divide en 2 partes:

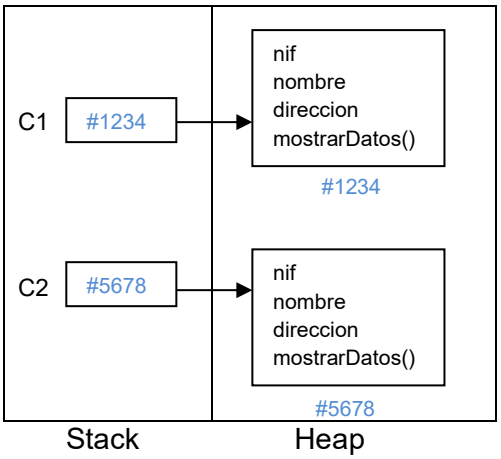
- **Stack** (Pila); donde se almacenan las variables

- **Heap** (Montón); donde se almacenan los objetos

Si creamos dos objetos de tipo Cliente:

```
Cliente c1 = new Cliente();
Cliente c2 = new Cliente();
```

Este es el aspecto que presentaría la memoria:



Como vemos cada variable apunta a su propio objeto, esto permite tener varios objetos del mismo tipo en memoria.

Para acceder al objeto simplemente pondremos el nombre de la variable c1 o c2. Si queremos acceder a un recurso determinado del objeto utilizaremos el operador miembro (.).

Ejemplo:

```
c1.nombre = "Juan";
c2.nombre = "Maria";

System.out.println(c1.mostrarDatos());
System.out.println(c2.mostrarDatos());
```

El ámbito del objeto queda comprometido al ámbito de la variable. Cuando la variable pierde su ámbito por ejemplo se había declarado dentro de un método y la ejecución del método ha finalizado. La siguiente vez que se ejecute el Garbage Collector eliminará dicha variable de memoria y de esta forma el objeto perderá su referencia por lo cual no se podrá volver a referenciar. El Garbage Collector también elimina el objeto por esta razón.

Todo el código de este ejemplo lo encontrareis en **Ejemplo1_ClasesObjetos.zip**

ACCESO A MIEMBROS

Para declarar el acceso a un miembro de la clase podemos utilizar los siguientes modos:

Modo de Acceso	Desde la misma clase	Desde el mismo paquete	Subclase	Resto de paquetes
public	SI	SI	SI	SI
protected	SI	SI	SI	NO
default	SI	SI	NO	NO
private	SI	NO	NO	NO

Como podemos observar están organizados de mayor a menor acceso. Expliquemos cada uno de ellos:

- **public**; un recurso definido como public se considera público por lo cual permite el acceso desde cualquier otro recurso ya sea declarado en la misma clase, en otra clase del mismo paquete, desde una subclase (herencia que se verá posteriormente) o incluso desde otras clases declaradas en otros paquetes.
- **protected**; un recurso definido como protected se considera protegido. Este modo permite el acceso desde cualquier otro recurso ya sea declarado en la misma clase, en otra clase del mismo paquete, desde una subclase. En este último caso es lo mismo si la subclase se encuentra en el mismo paquete o en otro.
- **default**; significa no establecer un modo de acceso. De esta forma solo se podrá invocar el recurso desde la misma clase o desde otra clase declarada en el mismo paquete.
- **private**; un recurso privado solo permite el acceso desde la misma clase donde ha sido declarado.

ENCAPSULACION

Una clase encapsulada es una clase que declara todas sus propiedades como privadas y solo se puede acceder a ella a través de los métodos get() y set().

Toda propiedad privada va a tener asociados dos métodos públicos (get y set) a través de los cuales vamos a poder modificar el valor de la propiedad (set) o recuperar su valor (get).

Veamos un ejemplo de una clase no encapsulada:

```java
package ejemplo2encapsulacion;

public class FechaNoEncapsulada {

    public int dia;
    public int mes;
    public int anyo;

    public FechaNoEncapsulada() {
    }

    public void mostrarFecha(){
        System.out.println(dia + "/" + mes + "/" + anyo);
    }

}
```

Gráfico 8. Clase no encapsulada

```java
public class Main {

    public static void main(String[] args) {

        FechaNoEncapsulada fechaErronea = new FechaNoEncapsulada();

        fechaErronea.dia = 78;
        fechaErronea.mes = 0;
        fechaErronea.anyo = -12;

        fechaErronea.mostrarFecha();
    }

}
```

Gráfico 9. Crear un objeto con datos erróneos

Al generar una instancia de una clase no encapsulada, como las propiedades son publicas puedo acceder directamente a ellas y establecer cualquier valor mientras que sea un numero entero.

El resultado al mostrar la fecha será: 78/0/-12

Para evitar la introducción de datos erróneos optamos por encapsular la clase, de esta forma tengo que acceder a las propiedades a través del método setXXX() en el cual puedo incorporar la lógica de negocio necesaria para poder controlar si el valor introducido es correcto.

```
public class FechaEncapsulada {

    private int dia, mes, anyo;

    public void setDia(int dia) {
        // si dia es un valor correcto se asigna a la propiedad
        this.dia = dia;
    }

    public void setMes(int mes) {
        // si mes es un valor correcto se asigna a la propiedad
        this.mes = mes;
    }

    public void setAnyo(int anyo) {
        // si anyo es un valor correcto lo asigno
        this.anyo = anyo;
    }
```

Gráfico 10. Fragmento de una clase encapsulada

Si ahora intentamos introducir los valores anteriores el código no lo permitirá.

```
FechaEncapsulada fechaCorrecta = new FechaEncapsulada();

fechaCorrecta.setDia(78);
fechaCorrecta.setMes(0);
fechaCorrecta.setAnyo(-12);

fechaCorrecta.mostrarFecha();
```

Gráfico 11. Fragmento de acceso a la clase encapsulada

El resultado tras la ejecución de este ejemplo será: 0/0/0

Todo el código de este ejemplo lo encontrareis en **Ejemplo2_Encapsulacion.zip**



RECUERDA QUE...

- Una clase es el código que nosotros escribimos, mientras que un objeto es una copia de esa clase en memoria.
- Una clase puede contener: propiedades, métodos y constructores.
- La encapsulación consiste en definir todas las propiedades de la clase como privadas y acceder a ellas mediante los métodos de acceso: getxxx() y setxxx().

3.- IDENTIFICADORES, PALABRAS CLAVE Y TIPOS

REGLAS PARA NOMBRAR UN IDENTIFICADOR

Consideramos un identificador al nombre de una clase, propiedad o método.

Las reglas a seguir para nombrar un identificador son las siguientes:

- Debe comenzar por una letra (mayúscula o minúscula), guión bajo (_) o símbolo de dólar ($).
- Son case sensitive por lo que se diferencias las letras mayúsculas y minúsculas. Se consideran 2 variables diferentes: nombre y Nombre.
- No existe una longitud máxima.

Ejemplos:

identifier
userName
user_name
_sys_var1
$change

PALABRAS CLAVE

Como todos los lenguajes de programación, Java define una serie de palabras clave que son reservadas para uso interno. No podremos utilizar ninguna palabra de esta lista como identificador.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Gráfico 12. Palabras clave

TIPOS PRIMITIVOS

Java define ocho tipos de datos primitivos. Los cuales se dividen a su vez en 4 categorías:

- Enteros: byte, short, int y long
- Decimales: float y double
- Lógico: boolean
- Textual: char

Enteros: byte, short, int y long

Los enteros primitivos pueden utilizar 3 formatos: decimal, octal y hexadecimal.

- 2 La representación decimal del numero 2.
- 077 Si el numero comienza por 0 indica un formato octal
- 0xBAAC Si el numero comienza por 0x indica un formato hexadecimal

El entero por defecto es el tipo int. Lo que quiere decir que cualquier numero entero se representará en 32 bits por defecto.

Los valores enteros de tipo long deben ir con el sufijo L o l.

Los diferentes tipos enteros representan los siguientes rangos:

Longitud	Nombre	Rango
8 bits	byte	2^7 a 2^7 -1
16 bits	short	2^{15} a 2^{15} -1
32 bits	int	2^{31} a 2^{31} -1
64 bits	long	2^{63} a 2^{63} -1

Decimales: float y double

Los decimales primitivos pueden utilizar 2 formatos: simple y exponencial.

- 3.14 Un valor simple decimal (un double)
- 6.02E23 Un numero decimal expresado con formato exponencial.

Los valores de tipo float deben ir con el sufijo F o f.

El decimal por defecto es el tipo double.

Los diferentes tipos decimales representan los siguientes rangos:

Longitud	Nombre	Rango
32 bits	float	2^{31} a 2^{31} -1
64 bits	double	2^{63} a 2^{63} -1

Otra de las novedades introducidas en Java 7 es el uso de la barra baja para separar números literales y que sea más fácil leerlos. Por ejemplo, si hasta ahora un millón se escribía así:

```
1000000
```

ahora lo escribiremos de esta forma:

```
1_000_000
```

También se introducen los literales binarios, con lo que ya no hay que convertirlos a hexadecimales y nos ahorramos un trabajo.

Lógico: boolean

Admite solo 2 literales true o false.

Textual: char

Se representa mediante 16 bits. El literal debe ir entre comillas simples ('). También admite las siguientes notaciones:

- 'a' La letra a
- '\t' Un tabulador
- '\u????' Un carácter Unicode, ????, se reemplaza por 4 dígitos hexadecimales
- Por ejemplo, '\u03A6' es la letra griega phi [Φ].



RECUERDA QUE...

- Al igual que todos los lenguajes de programación, Java reserva una serie de palabras para su uso interno. Estas no se pueden utilizar como identificadores.
- Java define ocho tipos primitivos: byte, short, int, long, float, double, char y boolean.

4.- OPERADORES Y CONTROL DE FLUJO

OPERADORES MATEMÁTICOS

Operador	Operación	Ejemplo	Comentario
+	Suma	sum = num1 + num2;	
-	Resta	diff = num1 – num2;	
*	Multiplicación	prod = num1 * num2;	
/	División	quot = num1 / num2;	La división devuelve un valor entero
%	Módulo	mod = num1 % num2;	El módulo devuelve el resto de la división.

OPERADORES INCREMENTO Y DECREMENTO

Operador	Operación	Ejemplo	Comentario

++	Pre-incremento (++variable)	int i = 6;
		int j = ++i;
		i vale 7, j vale 7
			La variable i se incrementa antes de asignar su valor a j.

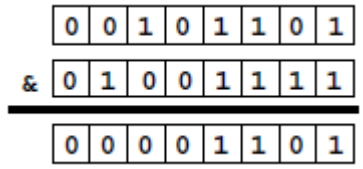
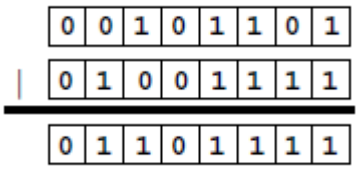
	Post-incremento (variable++)	int i = 6;
		int j = i++;
		i vale 7, j vale 6
			Primero se asigna el valor de i a j y después se incrementa i.

--	Pre-decremento (--variable)	int i = 6;
		int j = --i;
		i vale 5, j vale 5
			La variable i se decrementa antes de asignar su valor a j.

	Post-decremento (variable--)	int i = 6; int j = i--; i vale 5, j vale 6	Primero se asigna el valor de i a j y después se decrementa i.

OPERADORES BIT A BIT

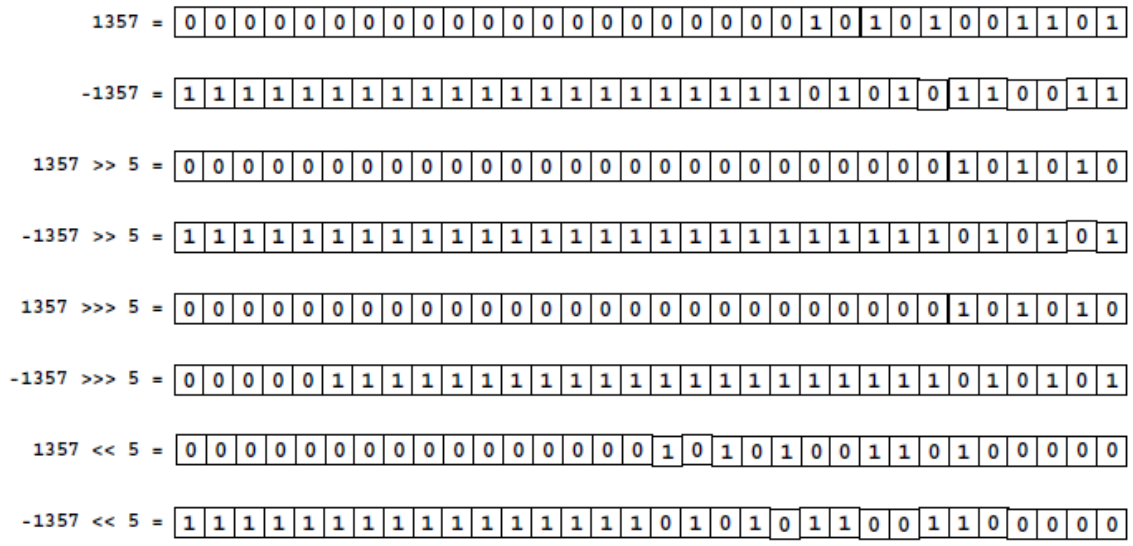
Operador	Operación	Ejemplo	Comentario
~	Complemento	~ 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 0	Invierte valores, los 1 los convierte a 0 y viceversa.
^	Xor	0 0 1 0 1 1 0 1 ^ 0 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0	Devuelve 0 si encuentra la combinación 0 y 0 ó 1 y 1. El resto devuelve 1.

&	And		Devuelve 1 si encuentra la combinación 1 y 1. En caso contrario devuelve 0.
|	OR		Devuelve 1 si encuentra algún 1 en la combinación. Si no lo encuentra devuelve 0.

OPERADORES DE DESPLAZAMIENTO

Operador	Operación	Comentario
>>	Desplazamiento derecha con signo	Desplaza a la derecha tantos bits se especifiquen. Si el numero es positivo se rellenaran los bits a 0. Si es negativo se rellenarán a 1.
>>>	Desplazamiento derecha sin signo	Desplaza a la derecha tantos bits se especifiquen sin tener en cuenta el signo. Los bits se rellenarán a 0.
<<	Desplazamiento izquierda	Desplaza a la izquierda tantos bits se especifiquen. Los bits se rellenarán a 0.

Ejemplos:



OPERADORES LÓGICOS

Operador	Operación	Ejemplo	Comentario

&&	AND	int i = 2;
		int j = 8;
		((i < 1) && (j > 6))
			Devuelve true si ambas condiciones se evalúan como ciertas.
			El resultado del ejemplo sería false.

||	OR	int i = 2;
		int j = 8;
		((i < 1) || (j > 10))
			Devuelve true si alguna de las condiciones o ambas se evalúan como ciertas.
			El resultado del ejemplo sería false.

!	NOT	int i = 2; (!(i < 3))	Devuelve lo contrario de como se evalúe la condición. El resultado del ejemplo sería false.

OPERADORES RELACIONALES

Operador	Operación	Ejemplo	Comentario
==	Igual a	int i=1; (i == 1)	El resultado es true.
!=	Diferente de	int i=2; (i != 1)	El resultado es true.
<	Menor que	int i=0; (i < 1)	El resultado es true.
<=	Menor o igual que	int i=1; (i <= 1)	El resultado es true.
>	Mayor que	int i=2; (i > 1)	El resultado es true.
>=	Mayor o igual que	int i=1; (i >= 1)	El resultado es true.

CONDICIONAL IF -ELSE

Una estructura condicional nos permite ejecutar un bloque de código u otro en función del resultado de una condición.

Sintaxis:

```
if(condición){
        // bloque de código
}
```

En este caso el bloque de código se ejecutará únicamente si se cumple la condición, dicho de otro modo, si la condición se evalúa como true.

Podemos completar este condición ofreciendo una alternativa.

Sintaxis:
```
if(condición){
        // bloque de código 1
}else{
        //bloque de código 2
}
```

En este otro caso, si la condición se evalúa como true se ejecutará el bloque de código 1, si la condición se evalúa como false entonces se ejecutará el bloque de código 2.

También podemos anidar condiciones.

Sintaxis:
```
if(condición 1){
        // bloque de código 1
}else if (condición 2){
        //bloque de código 2
}else{
        // bloque de código 3
}
```

El bloque de código 1 se ejecutará si se cumple la condición 1. El bloque de código 2 se ejecutará si no se cumple la condición 1 y si se cumple la condición 2. Por último el bloque de código 3 se ejecutará si no se cumplen ninguna de las condiciones anteriores.

SWITCH-CASE

Esta estructura permite ejecutar un bloque de código dependiendo del valor devuelto por una expresión.

Sintaxis:

```
switch (expresión){
        case valor1:
            // bloque de código 1
            [break;]
        case valor2:
            // bloque de código 2
            [break;]
        default:
            // bloque de código 3
}
```

Si la expresión devuelve el valor1 se ejecutará el bloque de código 1. Si devuelve el valor2, se ejecutará el bloque de código 2 y si devuelve otro valor que no sea ni valor1, ni valor 2 entonces se ejecutará el bloque de código 3.

Las sentencias break son optativas. Si se pone esta sentencia se produce una ruptura lo que quiere decir que se ejecutará únicamente el bloque de código asociado a cada caso. Si no utilizamos break, entonces se ejecutarán todos los bloques de código desde el caso que se cumpla hasta el final. Por ejemplo, si la expresión devuelve el valor2 y no hemos puesto break, se ejecutará el bloque de código 2 y el bloque de código 3.

La expresión debe ser de uno de los siguientes tipos: byte, short, int, char o enumerado.

STRINGS EN SENTENCIAS SWITCH

Hasta ahora, las sentencias Switch usaban o bien tipos primitivos o bien tipos enumerados. Con Java 7 se pueden usar también strings, que antes tendrían que ir en una serie de if-else como estos:

```
private void processTrade(Trade t){
        String status = t.getStatus( );
        if(status.equalsIgnoreCase(NEW)){
            newTrade(t);
        }else if(status.equalsIgnoreCase(EXECUTE)){
```

```
                executeTrade(t);
        }else if(status.equalsIgnoreCase(PENDING)){
                pendingTrade(t);
        }
}
```

Comparadlo ahora como queda al poder usar una sentencia Switch:

```
public void processTrade(Trade t){
        String status = t.getStatus( );
        switch (status){
                case NEW : newTrade(t);
                                break;
                case EXECUTE : executeTrade(t);
                                break;
                case PENDING : pendingTrade(t);
                                break;
         }
  }
```

Todo el código de este ejemplo lo encontrareis en **Ejemplo3_Condicionales.zip**

BUCLE FOR

Un bucle es un conjunto de instrucciones que se ejecutan varias veces dependiendo de una condición.

El bucle for se considera un bucle determinado porque sabemos de antemano el número de veces que se va a ejecutar.

Sintaxis:

```
for(inicio variable contador; condición; modificación){
        // cuerpo del bucle
}
```

Los términos utilizados los detallamos a continuación:

- **inicio variable contador**; para poder controlar el número de veces que se va a ejecutar el bucle utilizamos una variable. Esta variable puede estar declarada anteriormente o podemos declararla en el bucle. Con iniciar nos referimos a dar un valor inicial a la variable.
- **condición**; el bucle se ejecutará mientras se cumpla la condición.
- **modificación**; cada vez que se ejecute el bucle procedemos a modificar la variable contador. Lo más común suele ser incrementar o decrementar su valor.
- **cuerpo del bucle**; se denomina de esta forma al conjunto de instrucción que se ejecutan de forma repetitiva.

BUCLE FOR - EACH

A partir de Java 5 surge una mejora en el bucle for denominada for-each. Este bucle nos permite recorrer una colección de datos sin necesidad de tener una variable contador y condición.

Sintaxis:
```
for(tipo variable : coleccion){
        // cuerpo de bucle
}
```

Se define una variable del mismo tipo que los elementos de la colección. Si esta contiene números enteros se declarará una variable de tipo int, por ejemplo.

El funcionamiento, sería el siguiente, se recorre la colección comenzando por el primer elemento y terminando por el último. Cada elemento se asigna a la variable declarada y se ejecuta el cuerpo del bucle. Se pasa al siguiente elemento, el cual se almacena de nuevo en la variable y se procesa de nuevo el cuerpo del bucle. Estas operaciones se repiten hasta el último elemento de la colección.

BUCLE WHILE

El bucle while se considera un bucle indeterminado puesto que depende de una condición el número de veces que se va a ejecutar.

Sintaxis:
```
while(condición){
        // cuerpo del bucle
}
```

Si la condición se evalúa por primera vez como false no se ejecutará ninguna vez el cuerpo del bucle.

BUCLE DO-WHILE

Es una variante del anterior. También se considera un bucle indeterminado.

Sintaxis:
```
do {
        // cuerpo del bucle
} while (condición);
```

La diferencia con el bucle anterior es que esta vez la condición se evalúa al final, por lo cual si esta devuelve un resultado false, se habrá ejecutado el cuerpo del bucle una vez.

Todo el código de este ejemplo lo encontrareis en **Ejemplo4_Bucles.zip**

SENTENCIAS BREAK Y CONTINUE

Son instrucciones que permiten una alteración en la ejecución de un bucle.

BREAK .-

Permite finalizar, dar por terminado, totalmente la ejecución del bucle.

Sintaxis:
```
do {
        // sentencias 1
        if (condicion 1)
                break;
        // sentencias 2
} while (condición 2);
```

Si se cumple la condición 1 el bucle do-while habrá finalizado y continuaremos con la siguiente línea de código.

CONTINUE .-

Permite comenzar una nueva iteración sin terminar la actual.

Sintaxis:

```
do {
        // sentencias 1
        if (condicion 1)
                continue;
        // sentencias 2
} while (condición 2);
```

Si se cumple la condición 1 el bucle comenzará una nueva iteración sin terminar la actual, sin ejecutar las sentencias 2.

Todo el código de este ejemplo lo encontrareis en **Ejemplo5_Break_Continue.zip**



RECUERDA QUE...

- Java presenta las estructuras if-else y switch-case como condiciones
- Para generar bucles tenemos las estructuras: for, for-each, while y do-while
- Podemos romper o avanzar la ejecución de un bucle con las instrucciones: break y continue.

5.- ARRAYS

Un array es una estructura de datos que permite almacenar varios elementos. Cada uno de los elementos se almacena en una posición del array y para acceder a dicho elemento lo hacemos a través de un índice.

Los arrays tienen una serie de características:

- Se debe definir el tamaño del array al inicio. El tamaño se refiere al número de elementos máximo que puede almacenar.
- Todos los elementos han de ser del mismo tipo de datos.
- Son estructuras estáticas, lo que quiere decir que una vez que hemos fijado el tamaño del array este no se puede redimensionar.

Podemos definir arrays de una o varias dimensiones.

UNA DIMENSIÓN

DECLARAR UN ARRAY

En Java los arrays se tratan como objetos por lo cual necesitan de una variable de referencia para almacenar la dirección de memoria donde son creados.

Declarar un array significa crear una variable de tipo array. Lo que diferencia de la declaración de una variable normal a una variable de tipo array es el uso de corchetes ([ ]). Veremos más adelante que los corchetes también definen la dimensión del array, un grupo de corchetes será un array de una dimensión, dos grupos de corchetes definen un array de dos dimensiones y así sucesivamente.

Un array puede almacenar cualquier tipo de datos desde tipos primitivos como tipos de clase o sea objetos.

Veamos como declarar arrays de ambos tipos:

```
// Declarar un array de tipo primitivo
int numeros[];

// Declarar un array de objetos
Alumno alumnos[];
```

Gráfico 13. Ejemplos de declaración de arrays

Los corchetes se pueden posicionar antes o después de la variable y aunque en principio parece que es lo mismo no lo es. Vemos la diferencia con un ejemplo:

```
// Ejemplos de declaración
int num1[];
int [] num2;

int n1[], n2;
int [] n3, n4;
```

Gráfico 14 . Ejemplo de posición de los corchetes

En el primer ejemplo declaramos como variables de tipo array num1 y num2. En este caso tiene exactamente el mismo significado poner los corchetes antes o después de la variable.

En el segundo ejemplo declaramos diferente tipo de variables en función de la posición de los corchetes.

- Si ponemos los corchetes después de la variable n1 indicamos que es un array, sin embargo n2 es una variable de tipo int (no es un array).
- Si ponemos los corchetes delante de las variables n3 y n4 estamos declarando las dos variables como arrays.

En este punto, si intentamos acceder a la variable del array nos devolverá un **NullPointerException** ya que como toda variable de tipo objeto almacena un valor **null** si no apunta a ningún objeto creado.

CREAR EL ARRAY

Una vez declarado el array el siguiente paso es crear el array. En Java es obligatorio dar un tamaño al array, esto es, indicar el número de elementos máximo que puede contener.

```
// Creación de arrays
numeros = new int[5];
alumnos = new Alumno[3];
```

Gráfico 15. Ejemplo de creación de arrays

El array numeros puede contener hasta 5 números mientras que el array alumnos puede contener un máximo de 3 objetos de tipo Alumno.

ALMACENAR ELEMENTOS EN UN ARRAY

```
// Almacenar elementos en un array
numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;
numeros[4] = 50;

alumnos[0] = new Alumno(1, "Juan", "López", 5.6F);
alumnos[1] = new Alumno(2, "Marcos", "Saenz", 8.3F);
alumnos[2] = new Alumno(3, "Maria", "Arias", 6.3F);
```

Gráfico 16. Almacenar elementos en un array

Para poder almacenar elementos en un array es necesario hacer referencia a su índice. El índice de un array siempre comienza en 0 y termina en longitud-1. En el array de números enteros con longitud 5, el índice comienza en 0 y termina en 4. En el array de alumnos con longitud 3, el índice comienza en 0 y termina en 2.

GESTION DE MEMORIA

Vamos a representar en memoria los dos arrays creados con sus elementos.

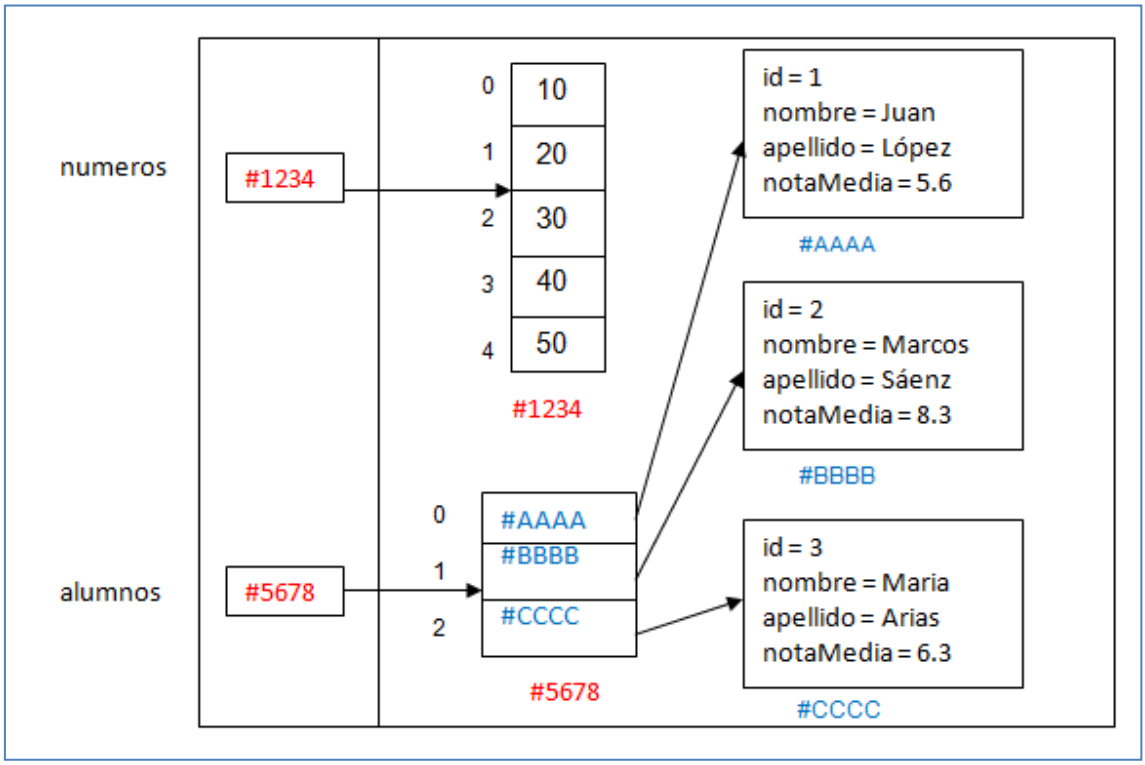


Gráfico 17. Representación en memoria de los arrays números y alumnos

Como vemos en los arrays de tipo primitivo se almacena directamente el valor, sin embargo en los arrays de tipo clase se almacenan las direcciones de memoria de los objetos almacenados.

DECLARAR, CREAR Y ALMACENAR ELEMENTOS

Hay una forma más rápida de declarar, crear y almacenar elementos en el array en una sola línea de código.

En el siguiente ejemplo se declara la variable de tipo array nums y se crea el array con los 8 números entre llaves. En este caso el array tiene una longitud de 8 elementos y sabemos que no se podrá redimensionar.

```
// Declarar, crear y almacenar elementos en una sola sentencia
int nums[] = {1,2,3,4,5,6,7,8};
```

Gráfico 18. Forma rápida de manejar arrays

ACCEDER A UN ELEMENTO

Para acceder a un elemento del array es necesario indicar su índice.

```
// Acceder a un elemento
System.out.println(numeros[3]);
System.out.println(alumnos[1].getNombre());
```

Gráfico 19. Acceso a los elementos del array

En el primer ejemplo accedemos al elemento con índice 3 en el array de numeros, el valor recuperado será el numero 40.

En el segundo ejemplo accedemos al objeto referenciado por el elemento con índice 1 del array de alumnos. Una vez que obtenemos la referencia del objeto podremos recuperar su nombre, el cual será Marcos.

RECORRER UN ARRAY

Muchas veces no nos interesa tan solo acceder a un elemento concreto del array sino más bien queremos recorrer todo el array accediendo a todos sus elementos. Pues bien, la forma más optima de llevar este proceso a cabo es con cualquiera de los dos formatos del bucle for.

- **bucle for**

La propiedad **length** de los arrays nos devuelve la longitud del mismo, o sea el número de elementos que tiene. Aprovechamos esta propiedad para establecer la condición del bucle.

```
// Recorrer el array con bucle for
for(int i=0; i<numeros.length; i++){
    System.out.println(numeros[i]);
}
```

Gráfico 20. Recorremos el array de números con el bucle for

La variable contador i nos sirve de índice, por lo cual la inicializamos a 0 que es el primer índice del array. La condición del bucle indica mientras que i sea menor que la longitud del array, hay que tener en cuenta que i nunca puede tomar el valor de la longitud. Recordamos que si un array tiene 5 elementos, su longitud es 5 y el índice empieza en 0 y termina en 4.

Si por error dejamos que i tomé el valor 5 esto producirá una excepción de tipo **ArrayIndexOutOfBoundsException** para indicar que estamos accediendo fuera del índice del array.

- **bucle for-each**

El bucle for-each es más cómodo de utilizar ya que no es preciso indicar el índice de los elementos.

```
// Recorrer el array con bucle for-each
for(Alumno a: alumnos){
    System.out.println(a);
}
```

Gráfico 21. Recorremos el array de alumnos con el bucle for-each

COPIAR ARRAYS

La clase **System** aporta un método **arraycopy** que permite copiar un array en otro. Veamos la sintaxis:

```
arraycopy(Object src, int srcPos, Object dest, int destPos, int length)
```

Gráfico 22. Sintaxis del método arraycopy

A continuación detallamos cada uno de sus parámetros:

- src; El array origen, del cual vamos a copiar los datos.
- srcPos; posición desde la cual empezamos a copiar en el array de origen
- dest; El array destino, en el cual vamos a pegar los datos.
- destPos; posición desde la cual empezamos a pegar en el array de destino
- length; el número de elementos que serán copiados.

Vamos a ver un ejemplo en el cual copiamos 3 elementos del array nums comenzando en el índice 2 y los pegamos en el array números a partir de la posición con índice 1.

```
// copiar elementos de un array en otro
System.arraycopy(nums, 2, numeros, 1, 3);
```

Gráfico 23. Ejemplo del método arraycopy

El array numeros obtenido será el siguiente:

```
10 3 4 5 50
```

Gráfico 24. Resultado del array numeros

Todo el código de este ejemplo lo encontrareis en **Ejemplo6_Arrays.zip**

VARIAS DIMENSIONES

DECLARAR UNA MATRIZ

Los arrays de varias dimensiones solemos conocerlos con el nombre de matrices. Para declarar matrices utilizamos tantos grupos de corchetes como dimensiones queremos en el array.

En el siguiente ejemplo declaramos dos matrices de dos dimensiones.

```
// Declarar un array de tipo primitivo
int numeros[][];

// Declarar un array de objetos
Alumno alumnos[][];
```

Gráfico 25. Declaración de matrices

CREAR UNA MATRIZ

- **Matrices cuadradas**

Una matriz cuadrada es aquella que contiene el mismo número de columnas para todas las filas.

En el siguiente ejemplo creamos la matriz números con 3 filas y 2 columnas y también creamos la matriz alumnos con 2 filas y 2 columnas.

```
// Creación de matrices
numeros = new int[3][2];
alumnos = new Alumno[2][2];
```

Gráfico 26. Creación de matrices cuadradas

- **Matrices no cuadradas**

Una matriz no cuadrada es aquella que cada fila tiene un determinado número de columnas.

En el siguiente ejemplo creamos una matriz no cuadrada con 2 filas, la primera de ellas contendrá 3 columnas y la segunda contendrá 2 columnas.

```
// Creación de matrices no cuadradas
int [][] nums = new int[2][];
nums[0] = new int[3];
nums[1] = new int[2];
```

Gráfico 27. Creación de matriz no cuadrada

Siempre debemos especificar el número de filas para luego definir el numero de columnas de cada fila aparte.

Lo que no podemos hacer es definir el numero de columnas, o sea, definirlo al revés.

ALMACENAR ELEMENTOS EN UNA MATRIZ

Para almacenar elementos necesitamos acceder a sus índices fila y columna.

```
// Almacenar elementos en una matriz
numeros[0][0] = 1;
numeros[0][1] = 2;
numeros[1][0] = 3;
numeros[1][1] = 4;
numeros[2][0] = 5;
numeros[2][1] = 6;

nums[0][0] = 10;
nums[0][1] = 20;
nums[0][2] = 30;
nums[1][0] = 40;
nums[1][1] = 50;
```

Gráfico 28. Almacenar elementos en una matriz

GESTION DE MEMORIA

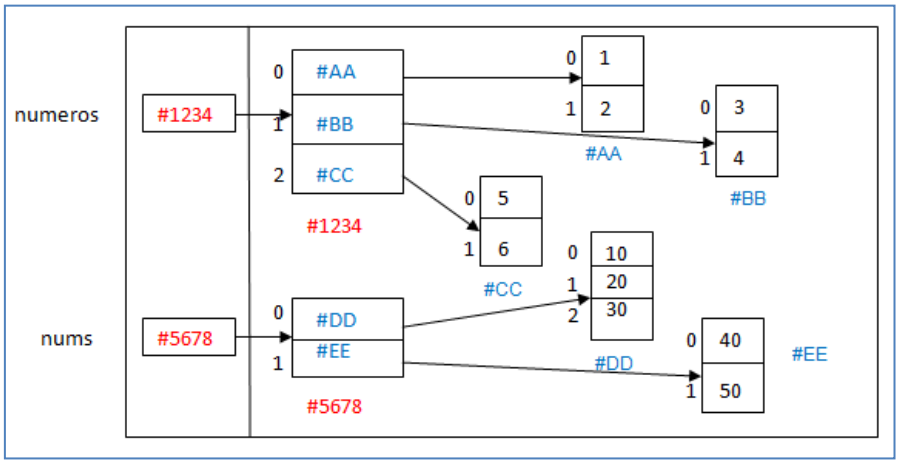


Gráfico 29. Representación en memoria de las matrices números y nums

DECLARAR, CREAR Y ALMACENAR ELEMENTOS

De la misma forma que en los arrays de una dimensión podíamos unificar todas estas tareas en una sola instrucción también lo podemos hacer con las matrices. Veamos como:

```
// Declarar, crear y almacenar elementos en una sola sentencia
int matriz1[][] = {{1,2},
                   {3,4}};

int matriz2[][] = {{1,2,3},
                   {4,5,6,7,8},
                   {9,0}};
```

Gráfico 30. Forma rápida de manejar matrices

En el primer ejemplo creamos una matriz cuadrada de 2 filas y 2 columnas. En el segundo ejemplo hemos creado un matriz no cuadrada de 3 filas: la primer fila tiene 3 elementos, la segunda fila tiene 5 elementos y la tercera fila tiene 2 elementos.

ACCEDER A UN ELEMENTO

Para acceder a un elemento de una matriz es necesario especificar la fila y columna del elemento.

```
// Acceder a un elemento
System.out.println(numeros[1][0]);
```

Gráfico 31. Acceder al elemento situado en la segunda fila y primera columna

Según nuestro ejemplo devolverá el valor 3.

RECORRER UNA MATRIZ

- **bucle for**

Necesitamos de dos bucles anidados. El primero de ellos es el encargado de recorrer el array de las filas con el índice i mientras que el segundo es quien recorre el array de las columnas. Este código es válido para recorrer matrices cuadradas y no cuadradas.

```
// Recorrer el array con bucle for
for(int i=0; i<numeros.length; i++){
    for(int j=0; j<numeros[i].length; j++){
        System.out.print(numeros[i][j] + " ");
    }
    System.out.println("");
}
```

Gráfico 32. Recorrer una matriz con el bucle for

```
1 2
3 4
5 6
```

Gráfico 33. Resultado obtenido

- **bucle for-each**

```
// Recorrer el array con bucle for-each
for(int[] columna : nums){
    for(int dato : columna){
        System.out.print(dato + " ");
    }
    System.out.println("");
}
```

Gráfico 34. Recorrer una matriz con el bucle for-each

Con el bucle for-each nuevamente necesitamos de dos bucles anidados. El primero nos devuelve los arrays de las columnas que recorremos en el bucle interno. El resultado que obtenemos es el que vemos a continuación.

```
10 20 30
40 50
```

Gráfico 35. Resultado obtenido

Todo el código de este ejemplo lo encontrareis en **Ejemplo7_Matrices.zip**



RECUERDA QUE...

- Un array se considera un objeto.
- Java solo reconoce los arrays de una dimensión, para manejar arrays de varias dimensiones se trabaja como un array de arrays.
- Podemos crear matrices cuadradas o no.

6.- CLASES AVANZADAS

HERENCIA

Una de las características de la programación orientada a objetos es la herencia. El termino herencia es conocido por todos, quien no ha oído hablar de una herencia en la familia. Nosotros no vamos a heredar dinero ni propiedades pero si vamos a aprender a heredar recursos de otras clases.

Observemos el diseño de las siguientes clases:

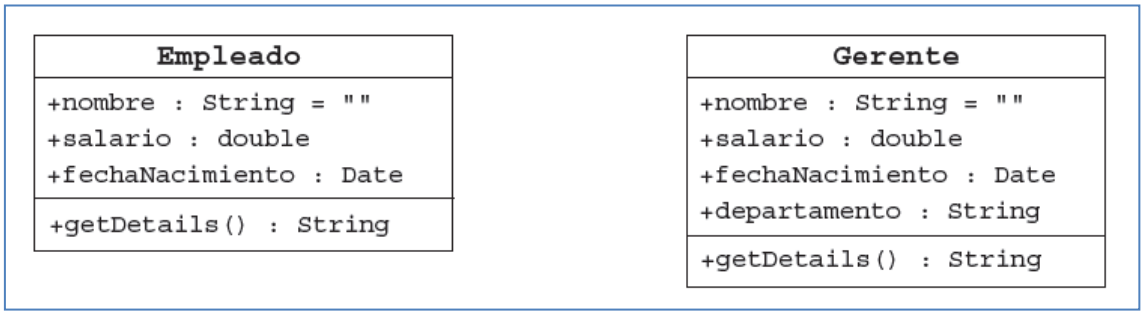


Gráfico 36. Diagramas de clase Empleado y Gerente

La implementación en código sería la siguiente:

```
public class Empleado {
  public String nombre = "";
  public double salario;
  public Date fechaNacimiento;

  public String getDetails() {...}
}
```

Gráfico 37. Implementación de la clase Empleado

```
public class Gerente {
  public String nombre = "";
  public double salario;
  public Date fechaNacimiento;
  public String departamento;

  public String getDetails() {...}
}
```

Gráfico 38. Implementación de la clase Gerente

Este ejemplo ilustra la duplicación de datos entre las clases Gerente y Empleado. Asimismo, podría haber una serie de métodos igualmente aplicables a estas dos clases.

Por tanto, necesitamos una forma de crear una clase nueva a partir de una clase existente, lo que recibe el nombre de creación de subclases.

En los lenguajes orientados a objetos, se proporcionan mecanismos especiales que permiten definir una clase sobre la base de una clase definida con anterioridad. En el siguiente gráfico puede verse un diagrama UML donde la clase Gerente es una subclase de Empleado.

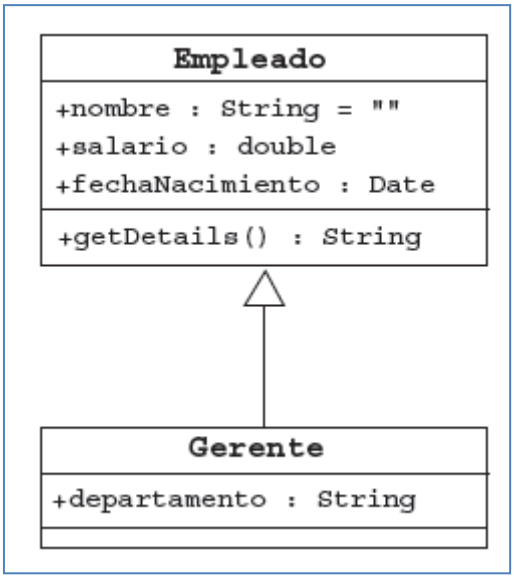


Gráfico 39. Diagramas de clase Empleado y Gerente a través de Herencia

Como se puede apreciar en el gráfico, en UML utilizamos una flecha continua para representar la herencia.

La clase Empleado se conoce como superclase, clase base o también como clase madre, mientras que la clase Gerente recibe el nombre de subclase, clase derivada o clase hija.

La implementación de la clase Gerente utilizando herencia sería la siguiente:

```
public class Gerente extends Empleado {
  public String departamento;
}
```

Gráfico 40. Implementación de la clase Gerente a través de herencia

Aunque sólo vemos la propiedad departamento declarado en la clase Gerente, de forma implícita tenemos todos los miembros heredados de la clase Empleado, nos referimos a las propiedades nombre, salario y fecha de nacimiento, así como al método getDetails().

En el lenguaje Java una clase solo puede heredar de otra, nunca de varias a la vez. Esta restricción se denomina herencia sencilla o herencia simple.

Veamos un ejemplo más ampliado de herencia.

La clase Empleado contiene tres atributos (nombre, salario y fechaNacimiento), así como un método (getDetails). La clase Gerente hereda todos estos miembros y añade un atributo más, departamento, además de usar el método getDetails. La clase Director hereda todos los miembros de Empleado y Gerente, pero además agrega el atributo vehiculoEmpresa y un nuevo método, aumentarComision.

Asimismo, las clases Ingeniero y Secretario heredan los miembros de la clase Empleado y podrían tener otros miembros específicos (no se muestran).

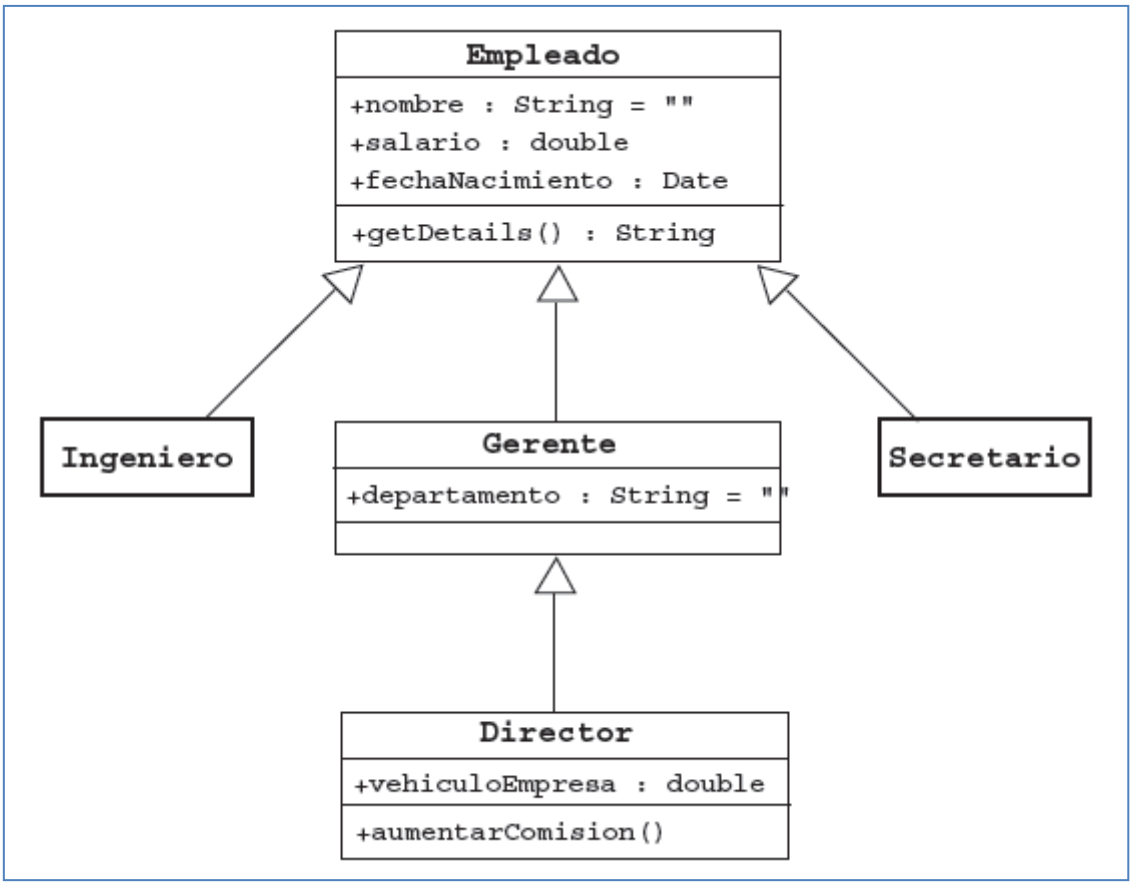


Gráfico 41. Ejemplo árbol de herencia

SOBREESCRITURA DE MÉTODOS

Además de poder agregar atributos para generar clases nuevas a partir de una clase existente, es posible modificar el comportamiento de la clase original.

Si se define un método en una subclase de tal forma que su nombre, el tipo de retorno y la lista de argumentos son idénticos a los de la superclase (la clase de nivel superior), se dice que el nuevo método sobrescribe al antiguo.

Observe estos ejemplos de métodos en las clases Empleado y Gerente:

```
public class Empleado {
  protected String nombre;
  protected double salario;
  protected Date fechaNacimiento;

  public String getDetails() {
    return "Nombre: " + nombre + "\n"
           + "Salario: " + salario;
  }
}

public class Gerente extends Empleado {
  protected String departamento;

  public String getDetails() {
    return "Nombre: " + nombre + "\n"
           + "Salario: " + salario + "\n"
           + "Gerente de: " + departamento;
  }
}
```

Gráfico 42. Método getDetails en la clase Empleado y Gerente

La clase Gerente contiene un método getDetails por definición, porque lo hereda de la clase Empleado. Pero el método original ha sido sustituido, sobrescrito, por la versión de la clase subordinada.

Recuerde que, para que un método pueda sustituir al de una clase de nivel superior, su nombre y el orden de sus argumentos deben ser idénticos a los del método de la superclase. Asimismo, el método sustitutivo no puede ser menos accesible que el método al que sustituye.

Analice este fragmento de código incorrecto:

```
public class Padre {
  public void hacerAlgo() {}
}

public class Hijo extends Padre {
  private void hacerAlgo() {} // no válido
}
```

Gráfico 43. Sobre escritura incorrecta de un método

REGLAS SOBREESCRITURA DE MÉTODOS

Si no cumplimos las reglas expuestas a continuación nos encontraremos con un error de compilación:

- El tipo de dato devuelto en el método sobreescrito debe ser el mismo o una subclase del tipo de dato devuelto en el método de la superclase.
- La lista de argumentos recibidos en el método sobreescrito debe ser la misma, nos referimos al mismo número, orden aunque pueden tener nombres diferentes.
- No se puede hacer el método sobreescrito menos accesible.
- Si el método original no tiene clausula throws no se la podemos poder. Lo que indica que si el método original no lanza excepciones, el sobreescrito tampoco.
- Sin embargo, si el método original tiene clausula throws, el método sobreescrito debe lanzar las mismas excepciones o una subclase de ella. También podemos quitar la clausula throws en el método sobreescrito.

LLAMADA A METODOS SOBREESCRITOS

Un método de una subclase puede llamar a un método de una superclase utilizando la palabra clave super.

La palabra super se refiere a la superclase de la clase en la que se ha utilizado esta clave. Se usa para hacer referencia a las variables miembro o a los métodos de la superclase. Se consigue utilizando la palabra clave super de la forma siguiente:

```
public class Empleado {
  private String nombre;
  private double salario;
  private Date fechaNacimiento;

  public String getDetails() {
    return "Nombre: " + nombre + "\nSalario: " + salario;
  }
}

public class Gerente extends Empleado {
  private String departamento;

  public String getDetails() {
    // llama al método de la superclase
    return super.getDetails()
           + "\nDepartamento: " + departamento;
  }
}
```

Gráfico 44. Invocar a un método sobreescrito

Una llamada con el formato super.metodo() llama al comportamiento completo, incluido cualquier efecto secundario del método al que se llamaría si el objeto hubiera sido una clase de nivel superior. El método no tiene por qué estar definido en la clase de nivel inmediatamente superior, puede heredarse de alguna clase situada más arriba en la jerarquía.

SOBRECARGA DE MÉTODOS

En algunos casos, puede que quiera escribir en la misma clase varios métodos que realizan la misma tarea básica pero con diferentes argumentos.

Pensemos en un método sencillo destinado a enviar una representación textual de su argumento. Este método se podría llamar println().

Java y otros lenguajes de programación permiten reutilizar un mismo nombre de método para varios métodos. Esto sólo funciona si las circunstancias bajo las que se hace la llamada permiten distinguir cuál es el método necesario. En el caso de los tres métodos de impresión, esta distinción se basa en el número de argumentos y su tipo.

Si se reutiliza el nombre de método, se obtienen los métodos siguientes:

```
public void println(int i)
public void println(float f)
public void println(String s)
```

Gráfico 45. Método println sobrecargado

Cuando se escribe el código para llamar a uno de estos métodos, se elige el método adecuado en función del tipo de argumento o argumentos que se suministran.

REGLAS SOBRECARGA DE MÉTODOS

Al igual que ocurre con la sobreescritura de métodos si no cumplimos unas determinadas reglas en la sobrecarga de métodos nos encontraremos con errores de compilación. Estas reglas son las siguientes:

- Los métodos sobrecargados han de tener el mismo nombre.
- Pueden tener diferentes modificadores de acceso, lo que indica que uno puede ser público, otro privado, ...etc.
- Pueden tener diferente numero de argumentos y en diferente orden.
- El tipo devuelto puede ser diferente, pero cuidado. Si solo cambiamos esto genera un error de compilación porque el compilador piensa que estamos efectuando una sobreescritura.
- No hay restricción en cuanto al lanzamiento de excepciones.

- Los constructores no se heredan por lo que no se pueden sobreescribir, pero sí que puede haber sobrecarga.

SOBRECARGA DE CONSTRUCTORES

Cuando se instancia un objeto, el programa puede ser capaz de suministrar varios constructores basándose en los datos del objeto que se está creando.

Por ejemplo, un sistema de nóminas podría ser capaz de crear un objeto Empleado si conoce todos los datos básicos sobre la persona: nombre, salario base y fecha de nacimiento. Puede que, en alguna ocasión, el sistema no conozca el salario base o la fecha de nacimiento.

El siguiente ejemplo contiene cuatro constructores sobrecargados para la clase Empleado.

El primer constructor inicializa todas las variables de instancia.

En el segundo, no se incluye la fecha de nacimiento. La referencia this se utiliza como forma de envío de llamada a otro constructor (siempre dentro de la misma clase), en este caso, el primer constructor.

Por su parte, el tercer constructor llama al primer constructor pasando la constante de clase SALARIO_BASE.

El cuarto constructor llama al segundo constructor pasando SALARIO_BASE, que, a su vez, llama al primer constructor pasando null en lugar de la fecha de nacimiento.

```
public class Empleado {
  private static final double SALARIO_BASE = 15000.00;
  private String nombre;
  private double salario;
  private Date fechaNacimiento;

  public Empleado(String nombre, double salario, Date FdeNac) {
    this.nombre = nombre;
    this.salario = salario;
    this.fechaNacimiento = FdeNac;
  }
  public Empleado(String nombre, double salario) {
    this(nombre, salario, null);
  }
  public Empleado(String nombre, Date FdeNac) {
    this(nombre, SALARIO_BASE, FdeNac);
  }
  public Empleado(String nombre) {
    this(nombre, SALARIO_BASE);
  }
  // más código de Empleado...
}
```

Gráfico 46. Ejemplo sobrecarga de constructores

La palabra clave this en un constructor debe ser la primera línea de código del constructor. Puede haber más código de inicialización después de la llamada this, pero no antes.

Aunque las subclases heredan todos los métodos y variables de sus superclases, no heredan sus constructores.

Las clases sólo pueden obtener un constructor de dos maneras: Debe escribirlo el programador o, si éste no lo escribe, debe usar el constructor predeterminado.

LLAMADA A UN CONSTRUCTOR DE LA SUPERCLASE

Al igual que los métodos, los constructores pueden llamar a los constructores no privados de la superclase inmediatamente anterior.

A menudo se define un constructor con determinados argumentos y se quiere usar esos mismos argumentos para controlar la construcción de la parte superior de un objeto. Puede llamar a un determinado constructor de una superclase como parte de la inicialización de una subclase utilizando la palabra clave super en la primera línea del constructor de la subclase. Para controlar la llamada de ese constructor concreto, es preciso suministrar los argumentos adecuados a super(). Cuando no hay ninguna llamada a super con argumentos, se llama implícitamente al constructor de la superclase

que tenga cero argumentos. En ese caso, si no hay ningún constructor de nivel superior que tenga cero argumentos, se produce un error de compilación.

La llamada a super() puede adoptar cualquier cantidad de argumentos adecuados para los distintos constructores disponibles en la clase superior, pero debe ser la primera sentencia del constructor.

Si recordamos que la clase Empleado tiene el conjunto de constructores definidos en el gráfico 46, entonces se definirían los siguientes constructores en Gerente.

```
public class Gerente extends Empleado {
  private String departamento;

  public Gerente(String nombre, double salario, String depto) {
    super(nombre, salario);
    departamento = depto;
  }
  public Gerente(String nombre, String depto) {
    super(nombre);
    departamento = depto;
  }
  public Gerente(String depto) { // Este código da error: no hay super()
    departamento = depto;
  }
}
```

Gráfico 47. Invocar constructores de la superclase

El último constructor no es válido porque el compilador introduce una llamada implícita a la clase super() y en la clase Empleado no se ha suministrado ningún constructor sin argumentos.

Si se utilizan, es preciso colocar super o this en la primera línea del constructor. Si escribe un constructor que no contiene ninguna llamada a super(...) ni a this(...), el compilador introduce una llamada al constructor de la superclase de forma automática y sin argumentos.

Otros constructores pueden llamar también a super(...) o this(...), con lo que se hace una llamada a una cadena de constructores. Lo que ocurre al final es que el constructor de la clase de nivel superior (o quizás varios de ellos) se ejecuta antes que ningún otro constructor subordinado de la cadena.

Todo el código de este ejemplo lo encontrareis en **Ejemplo8_Herencia.zip**

LA CLASE OBJECT

La clase Object es la raíz de todas las clases en el lenguaje Java. Si se declara una clase sin cláusula extends, el compilador agrega automáticamente el código extends Object a la declaración, por ejemplo:

```
public class Empleado {
  // más código aquí
}

Equivale a:

public class Empleado extends Object {
  // más código aquí
}
```

Gráfico 48. Declaración de clase heredando de Object

Esto permite sobrescribir varios métodos heredados de la clase Object. A continuación se describen dos métodos importantes de Object.

EL METODO EQUALS

El operador == realiza una comparación para determinar la equivalencia de dos términos. Es decir, para dos valores de referencia x e y dados, x==y devuelve true si, y sólo si, x e y se refieren al mismo objeto.

La clase Object del paquete java.lang contiene el método public boolean equals(Object obj), que compara dos objetos para comprobar su igualdad. Si no se sobrescribe, el método equals() de un objeto devuelve true únicamente si las dos referencias comparadas se refieren al mismo objeto. No obstante, la intención de equals() es comparar el contenido de dos objetos siempre que es posible. Ésta es la razón por la que se sobrescribe con frecuencia. Por ejemplo, el método equals() en la clase String devuelve true únicamente si el argumento no es null y es un objeto String que representa la misma secuencia de caracteres que el objeto String con el que se ha llamado al método.

En conclusión, el operador == compara si dos variables contienen la misma referencia mientras que con el método equals podemos comparar las propiedades del objeto para verificar si son iguales o no.

Se debería sobrescribir el método hashCode cada vez que sobrescriba equals. Una implementación sencilla podría usar un XOR de bits en los códigos hash de los elementos cuya equivalencia se quiera comprobar.

En el siguiente ejemplo, la clase MyDate sobrescribe el método equals que compara los atributos de año, mes y día.

También sobrescribimos el método hashCode implementa un XOR de bits de los atributos de fecha. Esto hace que el código hash de los objetos iguales de MyDate tenga

el mismo valor y ofrece la posibilidad de que fechas diferentes devuelvan valores diferentes.

```
public class MyDate {

    private int day;
    private int month;
    private int year;

    public MyDate(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
    }

    @Override
    public boolean equals(Object o) {
        boolean result = false;
        if ((o != null) && (o instanceof MyDate)) {
            MyDate d = (MyDate) o;
            if ((day == d.day) && (month == d.month)
                    && (year == d.year)) {
                result = true;
            }
        }
        return result;
    }

    @Override
    public int hashCode() {
        return (day ^ month ^ year);
    }
}
```

Gráfico 49. Ejemplo sobreescritura del método equals y hashCode

El programa siguiente compara dos objetos MyDate que no son idénticos pero son iguales en cuanto a la comparación de año-mes-día.

```
public class Main {

    public static void main(String[] args) {
        MyDate date1 = new MyDate(14, 3, 1976);
        MyDate date2 = new MyDate(14, 3, 1976);

        if (date1 == date2) {
            System.out.println("date1 es idéntica a date2");
        } else {
            System.out.println("date1 no es idéntica a date2");
        }

        if (date1.equals(date2)) {
            System.out.println("date1 es igual que date2");
        } else {
            System.out.println("date1 no es igual que date2");
        }

        System.out.println("set date2 = date1;");
        date2 = date1;

        if (date1 == date2) {
            System.out.println("date1 es idéntica a date2");
        } else {
            System.out.println("date1 no es idéntica a date2");
        }
    }
}
```

Gráfico 50. Ejemplo comparación de fechas

En la primera comparación con el operador == se compara si los dos objetos tienen la misma referencia.

En la segunda comparación con el método equals, se compara si los dos objetos son iguales, si tienen el mismo día, mes y año.

Después asignamos el objeto date1 a date2 y procedemos a una nueva comparación de las referencias de los objetos.

La ejecución de este programa de comparación genera la salida siguiente:

```
date1 no es idéntica a date2
date1 es igual que date2
set date2 = date1;
date1 es idéntica a date2
```

Gráfico 51. Resultados de la comparación de fechas

Todo el código de este ejemplo lo encontrareis en **Ejemplo9_Equals.zip**

EL METODO TOSTRING

El método toString convierte un objeto en una representación de cadena o String. El compilador hace referencia a él cuando se produce una conversión automática de cadenas. Por ejemplo, la llamada System.out.println():

```
Date now = new Date();
System.out.println(now);

Equivale a:

System.out.println(now.toString());
```

Gráfico 52. Impresión de objetos

La clase Object define un método toString que devuelve el nombre de la clase y su dirección de referencia (normalmente de escasa utilidad).

Muchas clases sobrescriben toString para proporcionar información de mayor utilidad.

En el siguiente ejemplo creamos la clase Fecha sin sobreescribir el método toString.

```
public class Fecha {

    private int dia;
    private int mes;
    private int anyo;

    public Fecha() {
    }

    public Fecha(int dia, int mes, int anyo) {
        this.dia = dia;
        this.mes = mes;
        this.anyo = anyo;
    }

}
```

Gráfico 53. Clase sin sobreescritura método toString

En la clase principal, creamos una instancia de Fecha e imprimimos el objeto creado.

```
public class Main {

    public static void main(String[] args) {
        Fecha fecha = new Fecha(12, 4, 2012);
        System.out.println(fecha);

        FechaConToString otraFecha = new FechaConToString(25, 8, 2012);
        System.out.println(otraFecha);
    }

}
```

Gráfico 54. Clase Principal donde creamos e imprimimos los objetos

Al ejecutar la clase vemos como se imprime el objeto fecha. Es una representación que no nos aporta ninguna información de utilidad, sólo se muestra el paquete, el nombre de la clase y el código del objeto.

```
ejemplotostring.Fecha@19821f
25/8/2012
```

Gráfico 55. Resultados tras la impresión de los objetos

A continuación creamos la clase FechaConToString donde sobrescribimos el método toString para obtener una representación textual del objeto.

```
public class FechaConToString {

    private int dia;
    private int mes;
    private int anyo;

    public FechaConToString() {
    }

    public FechaConToString(int dia, int mes, int anyo) {
        this.dia = dia;
        this.mes = mes;
        this.anyo = anyo;
    }

    @Override
    public String toString() {
        return dia + "/" + mes + "/" + anyo;
    }

}
```

Gráfico 56. Clase con sobreescritura del método toString

Al generar la instancia de esta última clase e imprimirlo, vemos que esta vez muestra la representación textual del objeto, aportándonos una mayor información.

Todo el código de este ejemplo lo encontrareis en **Ejemplo10_toString.zip**

CLASES ABSTRACTAS

Hemos visto como crear una subclase extendiendo de otra clase utilizando herencia. También hemos visto como podemos sobreescribir un método que hemos heredado.

Muchas veces nos vamos a encontrar con el siguiente caso: Creamos la superclase con un método que ya sabemos de antemano que se debe sobreescribir en la subclase porque su implementación debe variar. Entonces, qué sentido tiene implementar el método?

El lenguaje Java permite diseñar las clases de modo que los métodos declarados en las superclases no proporcionen ninguna implementación. Este tipo de métodos se denominan métodos abstractos. La implementación del método viene proporcionada por las subclases. Una clase que tenga uno o varios métodos abstractos se denomina clase abstracta.

Veamos el siguiente ejemplo.

Hemos creado una clase Figura donde hemos declarado dos métodos: El método mostrarPosición está implementado. Este método lo heredarán todas las subclases y su implementación no cambiará ya que todas las figuras tienen una posición en el espacio representada por los puntos x e y.

El método calcularArea es un método abstracto y por supuesto no está implementado. Observemos que los métodos abstractos terminan en punto y coma (;), si un método tiene llave de apertura y cierre ya está implementado aunque no tenga código en su interior.

El motivo por el cual el método calcularArea es abstracto es porque sabemos que las subclases han de sobreescribir este método ya que la forma de calcular el área de cada figura es diferente.

También observamos que la clase está declarada como abstracta, esto nos los indica el compilador. Si una clase tiene uno o más métodos abstractos, se debe declarar como abstracta. En caso contrario genera un error de compilación.

```java
public abstract class Figura {
    private int x;
    private int y;

    public Figura() {
    }

    public Figura(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public abstract double calcularArea();

    public String mostrarPosicion() {
        return  "[" + x + " , " + y + "]";
    }

}
```

Gráfico 57. Ejemplo de clase abstracta

A continuación vamos a ver las subclases creadas. Como el método heredado calcularArea es abstracto por definición las subclases también lo son hasta que se implemente dicho método.

No estamos obligados a implementarlo, pero si no lo hacemos debemos declarar las subclases como abstractas. Qué problema tendremos entonces? Pues que una clase abstracta realmente es una clase inacabada por lo cual no se puede instanciar.

```
public class Rectangulo extends Figura {

    private double base;
    private double altura;

    public Rectangulo(int x, int y) {
        super(x, y);
    }

    public Rectangulo(int x, int y, double base, double altura) {
        super(x, y);
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }

}
```

Gráfico 58. Implementación del método abstracto en la clase Rectángulo

En la clase Rectangulo implementamos el método calcularArea con el algoritmo adecuado para calcular el área de los rectángulos.

A continuación vemos la implementación del mismo método en la clase Circulo.

```
public class Circulo extends Figura{

    private double radio;

    public Circulo(int x, int y) {
        super(x, y);
    }

    public Circulo(int x, int y, double radio) {
        super(x, y);
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }

}
```

Gráfico 59. Implementación del método abstracto en la clase Circulo

Desde la clase principal creamos tres instancias:

La primera de ellas, es una instancia de la clase Figura que genera un error de compilación puesto que dicha clase es abstracta y no se puede instanciar.

Creamos una instancia de la clase Rectangulo donde pasamos los siguientes argumentos al constructor: puntos x e y, base y altura del rectángulo.

También se crea una instancia de la clase Circulo con los siguientes datos: puntos x e y además del radio del circulo.

```
public class Main {

    public static void main(String[] args) {
        // Error de compilación
        // Figura figura = new Figura(4, 6);

        Rectangulo rectangulo = new Rectangulo(4, 6, 12, 15);
        rectangulo.mostrarPosicion();
        System.out.println("El area del rectangulo es: " +
                           rectangulo.calcularArea());

        Circulo circulo = new Circulo(3, 8, 5);
        circulo.mostrarPosicion();
        System.out.println("El area del circulo es: " +
                           circulo.calcularArea());
    }

}
```

Gráfico 60. Clase principal donde generamos las instancias

Todo el código de este ejemplo lo encontrareis en **Ejemplo11_ClasesAbstractas.zip**

UML utiliza letras en cursiva para indicar elementos abstractos en los diagramas de clases. También se pueden marcar las clases abstractas con el indicador {abstract} en la sección de nombre.

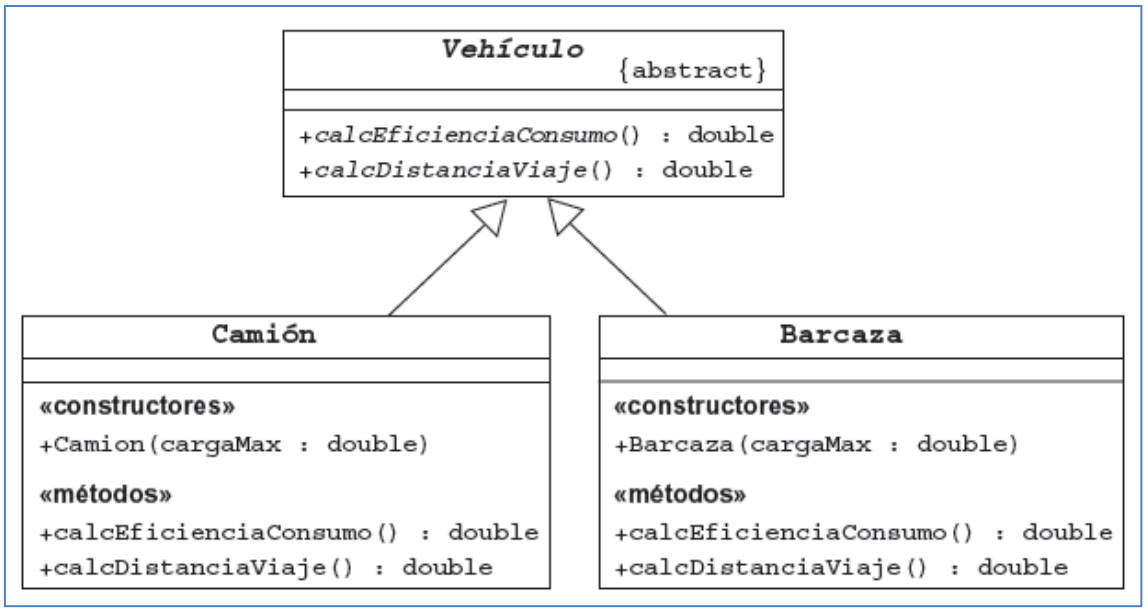


Gráfico 61. Diagrama UML clases abstractas

INTERFACES

Hemos visto que en Java solo es posible la herencia simple, el hecho de que una clase tan sólo puede heredar de otra clase, nunca de varias.

La forma de simular una herencia múltiple la tenemos en el uso de interfaces.

Una interface es una clase abstracta llevada al extremo. Una clase abstracta puede contener métodos implementados y métodos abstractos, así como propiedades y constructores. Pues bien, en una interface todos los métodos declarados son abstractos, nunca puede haber métodos implementados.

Todos los métodos declarados en una interfaz son públicos (public) y abstractos (abstract), con independencia de que mencionemos estos modificadores en el código o no. Igualmente, todos los atributos son public, static y final. En otras palabras, sólo se pueden declarar atributos constantes.

Al igual que los nombres de clases abstractas, los nombres de interfaz se utilizan como tipos de variables de referencia.

Imagine un grupo de objetos que comparten la misma habilidad: pueden volar. Puede construir una interfaz pública llamada ObjetoVolador que admite tres operaciones: despegar, aterrizar y volar.

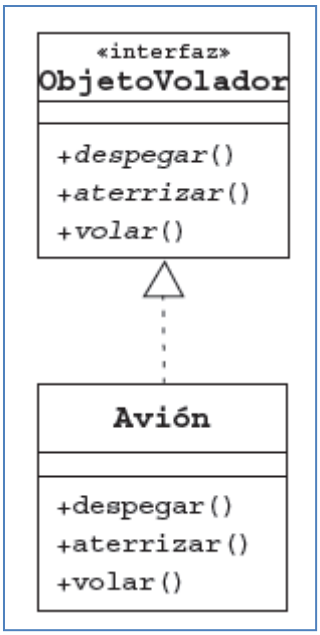


Gráfico 62. Diagrama UML con interface

Como podemos observar en el diagrama UML, las interfaces se marcan con el indicador <<interface>> en la sección de nombre.

Por otro lado, utilizamos una flecha discontinua para indicar la clase que implementa una interface.

A continuación vemos el código fuente de la interface y la clase.

```
public interface ObjetoVolador {

    public void despegar();

    public void aterrizar();

    public void volar();
}
```

Gráfico 63. Código interface ObjetoVolador

```java
public class Avion implements ObjetoVolador {

    public void despegar() {
        // acelerar hasta despegar
        // subir el tren de aterrizaje
    }

    public void aterrizar() {
        // bajar el tren de aterrizaje
        // decelerar y desplegar flaps hasta tocar tierra
        // frenar
    }

    public void volar() {
        // mantener los motores en marcha
    }
}
```

Gráfico 64. Implementación de la interface ObjetoVolador en la clase Avion

Puede haber muchas clases que implementen la interfaz ObjetoVolador. Un Avion, un ave o Superman pueden volar.

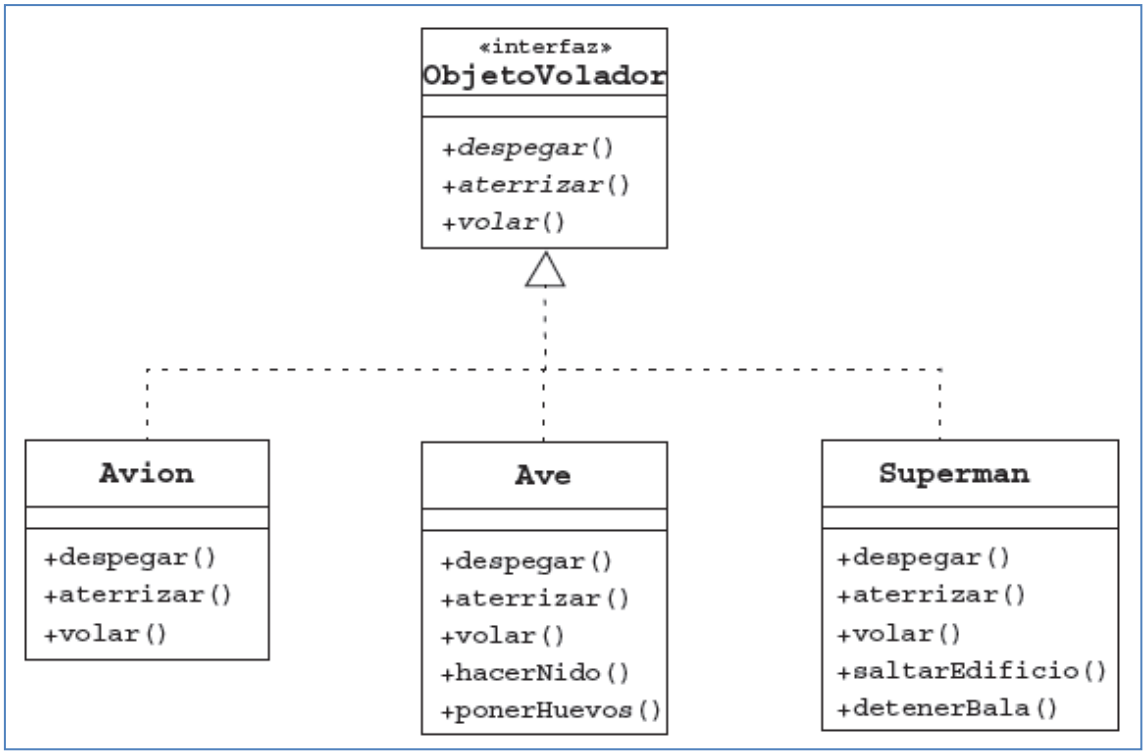


Gráfico 65. Varias clases implementan la misma interface

Todo el código de este ejemplo lo encontrareis en **Ejemplo12_Interfaces.zip**

HERENCIA MULTIPLE

Suena como una herencia múltiple, pero no lo es. El peligro de la herencia múltiple es que una clase podría heredar dos implementaciones diferentes del mismo método. Esto no es posible con las interfaces, porque una declaración de método de una interfaz no proporciona ninguna implementación.

El secreto para similar la herencia múltiple es porque una clase sólo puede heredar de una sola clase pero puede implementar varias interfaces.

A continuación mostramos un diagrama UML donde se puede apreciar una herencia múltiple.

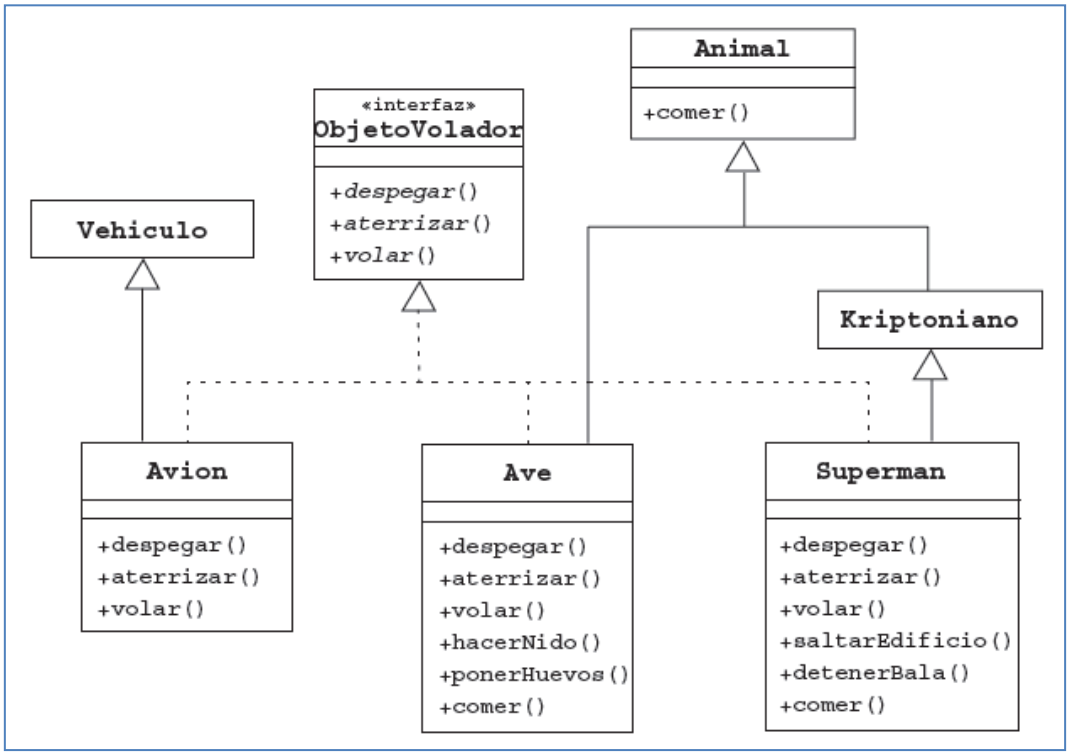


Gráfico 66. Diagrama UML con herencia múltiple

Veamos una implementación de herencia múltiple, según el diagrama la clase Ave hereda de la clase Animal y a su vez implementa la interface ObjetoVolador.

```
public class Animal {

    public void comer(){
        // los animales comen
    }

}
```

Gráfico 67. Implementación de la clase Animal

```
public class Ave extends Animal implements ObjetoVolador {

    public void despegar() { /* implementación de despegar */ }

    public void aterrizar() { /* implementación de aterrizar */ }

    public void volar() { /* implementación de volar */ }

    public void hacerNido() { /* comportamiento de nidificación */ }

    public void ponerHuevos() { /* implementación de puesta de huevos */ }

    @Override
    public void comer() { /* sobrescritura de la acción de comer */ }
}
```

Gráfico 68. Implementación de la clase Ave simulando herencia múltiple

La cláusula extends debe especificarse antes que la cláusula implements. La clase Ave puede suministrar sus propios métodos (hacerNido y ponerHuevos), así como sobrescribir los métodos de la clase Animal (comer).

MULTIPLES INTERFACES

Una clase puede implementar varias interfaces. El Hidroavion no sólo puede volar, sino también navegar. La clase Hidroavion amplía la clase Avion, con lo que hereda esa implementación de la interfaz ObjetoVolador. La clase Hidroavión también implementa la interfaz Nautico.

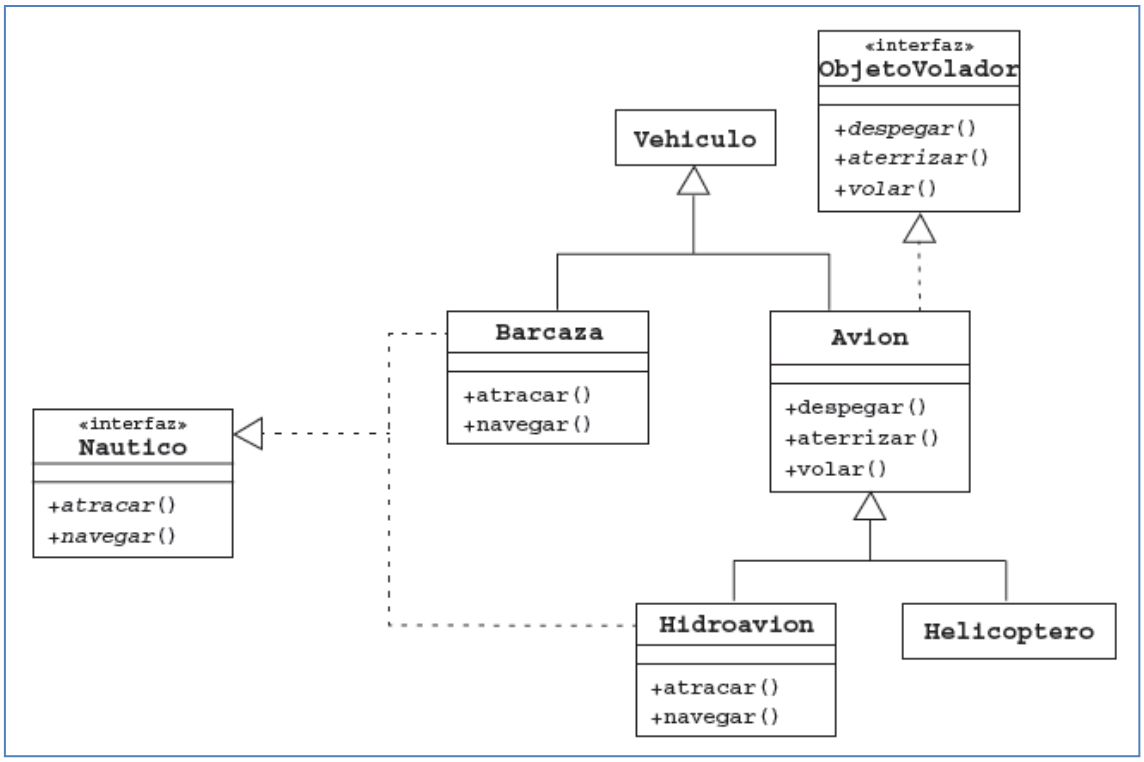


Gráfico 69. Diagrama UML con varias interfaces

USOS DE LAS INTERFACES

Las interfaces se utilizan para:

- Declarar métodos que serán implementados por una o varias clases.
- Dar a conocer la interfaz de programación de un objeto sin revelar el verdadero cuerpo de la clase (esto puede ser útil al distribuir un paquete de clases a otros desarrolladores).
- Identificar las similitudes entre clases no relacionadas sin tener que establecer ninguna relación entre ellas.
- Simular la herencia múltiple declarando una clase que implemente varias interfaces.

POLIMORFISMO

Según el siguiente diagrama de clases. La clase Gerente hereda de la clase Empleado y la clase Director hereda de la clase Gerente. También podemos decir que la clase Director hereda indirectamente de Empleado.

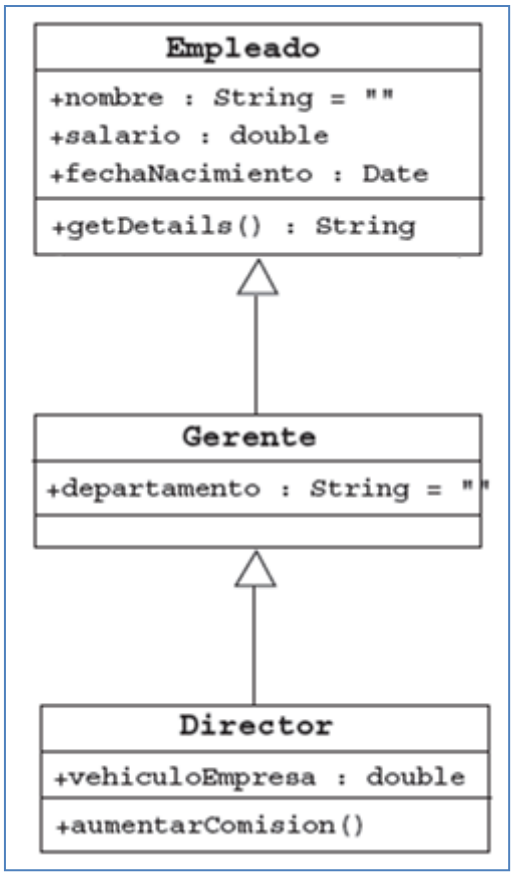


Gráfico 70. Diagrama de clases para aplicar polimorfismo

RELACION IS-A

Cuando utilizamos herencia se establece una relación is-a (traducido es-un). Según el diagrama anterior se establecen las siguientes relaciones:

- Gerente es un Empleado
- Director es un Empleado
- Director es un Gerente

Además de forma implícita, sabemos que Empleado hereda de Object por lo cual también se establecen las siguientes relaciones:

- Empleado es un Object
- Gerente es un Object
- Director es un Object

REPRESENTACION DE LOS OBJETOS CREADOS

Habéis oído hablar de las muñecas matrioskas? Son esas muñecas rusas que va una dentro de otra. Pues bien, vamos a pensar que cada objeto creado anteriormente es una muñeca. Por lo cual lo podríamos representar así.

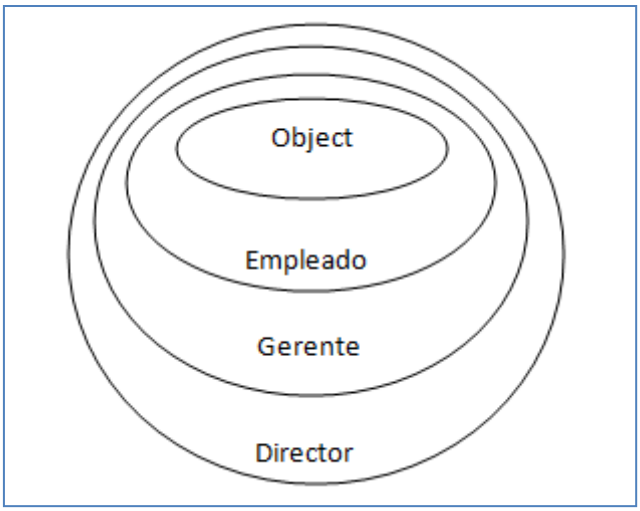


Gráfico 71. Representación de objetos por herencia

Es como decir que dentro del objeto Director hay un objeto Gerente y dentro de este un objeto Empleado y dentro de este a su vez un objeto de tipo Object.

Esta representación nos va a ayudar a entender otra de las características de la Programación Orientada a Objetos. Nos referimos al polimorfismo.

Un objeto sólo tiene una forma (aquella que se le asigna cuando se construye). Sin embargo, una variable es polimórfica porque puede hacer referencia a objetos de distintas formas. Lo que queremos decir, es que el objeto no cambia. Lo que cambia es la forma como lo vemos.

El lenguaje Java, como la mayoría de los lenguajes de programación OO, en realidad permite hacer referencia a un objeto con una variable que es uno de los tipos de una superclase. Por tanto, es posible decir:

```
Empleado e = new Gerente(); //válido
```

Gráfico 72. Ejemplo polimorfismo

Utilizando la variable e tal cual, puede acceder únicamente a las partes del objeto que son componentes de Empleado; las partes específicas de Gerente están ocultas. Esto es porque, por lo que respecta al compilador, e es un Empleado, no un Gerente. Por lo tanto, no se permite lo siguiente:

```
// Intento no permitido de asignar un atributo de Gerente
e.departamento = "Ventas";
// la variable se declara como un tipo de Empleado
// a pesar de que el objeto Gerente tiene ese atributo
```

Gráfico 73. Error en el uso de polimorfismo

LLAMADA A METODOS VIRTUALES

Supongamos que es cierto lo siguiente:

Empleado e = new Empleado();

Gerente m = new Gerente();

Si hace una llamada a e.getDetails() y a m.getDetails(), está llamando a diferentes comportamientos. El objeto Empleado ejecuta la versión de getDetails() asociada a la clase Empleado y el objeto Gerente ejecuta la versión de getDetails() asociada a la clase Gerente.

Lo que no resulta tan obvio es lo que ocurre si tenemos el código siguiente:

Empleado e = new Gerente();

e.getDetails();

O algo parecido, como un argumento de un método general o un elemento de una colección heterogénea.

De hecho, se obtiene el comportamiento asociado al objeto al que hace referencia la variable durante el tiempo de ejecución. El comportamiento no está determinado por el tipo de la variable en el momento de la compilación. Este es un aspecto del polimorfismo y un aspecto importante de los lenguajes orientados a objetos. Este comportamiento a menudo recibe el nombre de llamada a métodos virtuales.

En el ejemplo anterior, el método e.getDetails() ejecutado procede del tipo real del objeto, un Gerente.

COLECCIONES HETEROGÉNEAS

Es posible crear colecciones de objetos que tienen una clase común. Este tipo de colecciones se denominan **homogéneas**. Por ejemplo:

```
MyDate[]  dates = new MyDate[2];
dates[0] = new MyDate(22, 12, 1964);
dates[1] = new MyDate(22, 7, 1964);
```

Gráfico 74. Ejemplo de colecciones homogéneas

El lenguaje Java incluye una clase Object que permite crear colecciones de todos los tipos de elementos porque todas las clases amplían la clase Object. Estas colecciones se denominan **heterogéneas**.

Una colección heterogénea es aquella que se compone de elementos dispares. En los lenguajes OO, es posible crear colecciones de numerosos elementos. Todas ellas tienen una clase de origen común: la clase Object. Por ejemplo:

```
Empleado [] personal = new Empleado[1024];
personal[0] = new Gerente();
personal[1] = new Empleado();
personal[2] = new Ingeniero();
```

Gráfico 75. Ejemplo de colecciones heterogéneas

Es posible escribir incluso un método de ordenación que coloque los empleados por orden de edad o salario, con independencia de que algunos de esos empleados sean gerentes.

ARGUMENTOS POLIMÓRFICOS

Es posible escribir métodos que acepten un objeto genérico (en este caso, la clase Empleado) y funcionen adecuadamente con los objetos de cualquier subclase de dicho objeto. Por ejemplo, podría generarse un método en una clase de aplicación que tome un empleado y lo compare con un determinado umbral de salario para determinar las obligaciones fiscales de ese empleado. Si se utilizan las características polimórficas, es posible hacerlo como sigue:

```
public class ServicioImpuestos {
  public TipoImpositivo hallarTipoImpositivo(Empleado e) {
    // hace los cálculos y devuelve el tipo impositivo para e
  }
}

// Mientras tanto, en otra parte de la clase de aplicación
ServicioImpuestos svcImp = new ServicioImpuestos();
Gerente m = new Gerente();
TipoImpositivo t = svcImp.hallarTipoImpositivo(m);
```

Gráfico 76. Ejemplo de argumentos polimórficos

Esto es válido porque un Gerente es un Empleado. No obstante, el método hallarTipoImpositivo sólo tiene acceso a los miembros (variables y métodos) que estén definidos en la clase Empleado.

OPERADOR INSTANCEOF

Dado que es posible pasar objetos de un lugar a otro utilizando referencias a sus superclases, a veces conviene saber de qué objetos reales se dispone. Éste es el propósito del operador instanceof.

Si recibe un objeto utilizando una referencia del tipo Empleado, puede que resulte ser un Gerente o un Ingeniero. Puede comprobarlo utilizando instanceof de la forma siguiente:

```
public void hacerAlgo(Empleado e) {
  if ( e instanceof Gerente ) {
    // Procesar un Gerente
  } else if ( e instanceof Ingeniero ) {
    // Procesar un Ingeniero
  } else {
    // Procesar cualquier otro tipo de Empleado
  }
}
```

Gráfico 77. Ejemplo operador instanceof

CONVERSIÓN DE OBJETOS

En casos en los que se ha recibido una referencia a una clase de nivel superior y se ha comprobado mediante el operador instanceof que el objeto es una subclase concreta, es posible acceder a toda la funcionalidad del objeto convirtiendo la referencia.

```
public void hacerAlgo(Empleado e) {
  if ( e instanceof Gerente ) {
    Gerente m = (Gerente) e;
    System.out.println("Éste es el gerente de "
                       + m.getDepartamento());
  }
  // resto de la operación
}
```

Gráfico 78. Ejemplo conversión de objetos

Si no realiza la conversión, el intento de ejecutar e.getDepartamento() fracasará porque el compilador no puede localizar ningún método llamado getDepartamento en la clase Empleado.

Si no realiza la comprobación con instanceof, corre el riesgo de que falle la conversión. En general, cualquier intento de convertir una referencia a un objeto se somete a diferentes comprobaciones:

- Las conversiones en dirección ascendente dentro de la jerarquía de clases siempre están permitidas y, de hecho, no precisan el operador de conversión. Se pueden hacer mediante una simple asignación.
- En el caso de las conversiones en dirección descendente, el compilador debe poder considerar que la conversión es, al menos, posible. Por ejemplo, no se

permitirá ningún intento de convertir una referencia a Gerente en una referencia a Ingeniero, porque el Ingeniero no es un Gerente. La clase de destino de la conversión tiene que ser una subclase del tipo de referencia actual.

- Si el compilador permite la conversión, el tipo de objeto se comprueba durante el tiempo de ejecución. Por ejemplo, si se omite la comprobación instanceof en el código y el objeto que se va a convertir en realidad no es del tipo en el que se va a convertir, se producirá un error de tiempo de ejecución (excepción).

Todo el código de este ejemplo lo encontrareis en **Ejemplo13_Polimorfismo.zip**

CLASES ENVOLVENTES

El lenguaje Java no considera los tipos de datos primitivos como objetos. Por ejemplo, los datos numéricos, booleanos y de caracteres se tratan de forma primitiva para mantener la eficiencia. A fin de poder manejar los tipos primitivos como objetos, Java proporciona las llamadas clases envoltorio o clases envolventes. Estos elementos quedan envueltos por un objeto que se crea en torno a ellos. Cada tipo primitivo en Java tiene una clase envoltorio asociada en el paquete java.lang. Cada clase envoltorio encapsula un único valor primitivo.

Las clases envolventes implementan objetos inmutables. Esto significa que, una vez inicializado el valor primitivo en su envoltorio, no hay forma de cambiar ese valor.

A continuación se muestra una tabla en la cual se recogen las diferentes clases envolventes.

Tipo de datos primitivos	Clase envoltorio
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

Gráfico 79. Clases envolventes

Los objetos de clase envoltorio se construyen pasando el valor que debe encapsularse al constructor adecuado.

```
int pInt = 420;
Integer wInt = new Integer(pInt); // esto se denomina
boxing
int p2 = wInt.intValue(); // esto se denomina unboxing
```

Gráfico 80. Ejemplos de conversión de primitivos en clases envoltorio

Las clases envoltorio son útiles para convertir datos primitivos debido a la gran cantidad de métodos disponibles con este tipo de clases, por ejemplo:

```
int x = Integer.valueOf(str).intValue();

or:

int x = Integer.parseInt(str);
```

Gráfico 81. Ejemplo de uso de la clase envolvente Integer

AUTOBOXING

Si necesita cambiar los tipos de datos primitivos en su objeto equivalente (operación denominada boxing), necesita utilizar las clases envoltorio.

Igualmente, para obtener el tipo de dato primitivo a partir de la referencia del objeto (lo que se denomina unboxing), también necesita usar los métodos de las clases envoltorio. Todas estas operaciones de conversión pueden complicar el código y dificultar su comprensión. En la versión 5.0 de J2SE se ha introducido una función de conversión automática denominada **autoboxing** que permite asignar y recuperar los tipos primitivos sin necesidad de usar clases envoltorio.

El siguiente ejemplo contiene dos casos sencillos de conversión y recuperación automática de primitivos (autoboxing y autounboxing).

```
int pInt = 420;
Integer wInt = pInt; // esto se denomina autoboxing
int p2 = wInt; // esto se denomina autounboxing
```

Gráfico 82. Ejemplo de autoboxing

El compilador de J2SE versión 5.0 ahora creará el objeto envoltorio automáticamente cuando se asigne un primitivo a una variable del tipo de clase envoltorio. Asimismo, el

compilador extraerá el valor primitivo cuando realice la asignación de un objeto envoltorio a una variable de tipos primitivos.

Esto puede hacerse al pasar parámetros a métodos o, incluso, dentro de las expresiones aritméticas.

Todo el código de este ejemplo lo encontrareis en **Ejemplo14_Autoboxing.zip**

RECURSOS ESTÁTICOS

La palabra **static** declara miembros (atributos, métodos y clases anidadas) que están asociados a una clase en vez de a una instancia de la clase.

ATRIBUTOS DE CLASE

A veces resulta útil tener una variable compartida por todas las instancias de una clase. Esta variable podría utilizarse, por ejemplo, como base para la comunicación entre instancias o para llevar el control del número de instancias que se han creado.

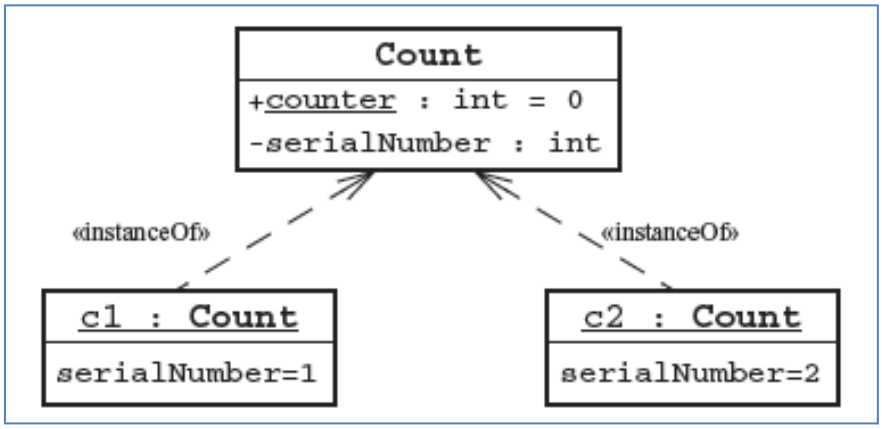


Gráfico 83. Diagrama UML de la clase Count y dos instancias únicas

El modo compartido se consigue marcando la variable con la palabra static. Este tipo de variables a veces se denominan variables de clase para distinguirlas de los miembros o las variables de instancia, que no se comparten.

```
public class Counter {

    private int serialNumber;
    public static int counter = 0;

    public Counter() {
        counter++;
        serialNumber = counter;
    }
}
```

Gráfico 84. Ejemplo de variable estática

En este ejemplo, a cada objeto que se crea se le asigna un número de serie exclusivo que empieza desde 1 y continúa en sentido ascendente. Todas las instancias comparten la variable counter por lo que, cuando el constructor de un objeto incrementa counter, el siguiente objeto que se crea recibe el valor incrementado.

Si una variable static no se marca como private, es posible acceder a ella desde fuera de la clase. Para hacerlo, no se necesita ninguna instancia de la clase, basta hacer referencia a ella mediante el nombre de la clase.

```
public class OtraClase {

    public void incrementNumber() {
        Counter.counter++;
    }
}
```

Gráfico 85. Acceso a una variable estática fuera de la clase

MÉTODOS DE CLASE

Cuando no hay disponible ninguna instancia de un determinado objeto, es necesario acceder al código del programa. Los métodos marcados con la palabra static pueden utilizarse de esta forma y algunas veces se denominan métodos de clase.

```
public class Count2 {

    private int serialNumber;
    private static int counter = 0;

    public static int getTotalCount() {
        return counter;
    }

    public Count2() {
        counter++;
        serialNumber = counter;
    }
}
```

Gráfico 86. Ejemplo método estático

Debe acceder a los métodos estáticos utilizando el nombre de la clase en lugar de una referencia al objeto, de la forma siguiente:

```
public class Main {

    public static void main(String[] args) {
        System.out.println("El número del contador es "
                + Count2.getTotalCount());
        Count2 count1 = new Count2();
        System.out.println("El número del contador es "
                + Count2.getTotalCount());
    }
}
```

Gráfico 87. Ejemplo invocación a un método estático

Dado que puede llamar a un método static sin necesidad de tener una instancia de la clase a la que pertenece, no existe ningún valor this. Como consecuencia, un método static no puede acceder a ninguna variable salvo a las variables locales, los atributos static y sus parámetros. Cualquier intento de acceder a atributos que no sean estáticos provoca un error de compilación.

Los atributos no estáticos se limitan a una instancia y sólo se puede acceder a ellos mediante referencias a esa instancia.

```
public class Count3 {
  private int serialNumber;
  private static int counter = 0;

  public static int getSerialNumber() {
    return serialNumber;  // ERROR DE COMPILACIÓN
  }
}
```

Gráfico 88. Error al intentar acceder a un recurso no estático

Cuando use métodos estáticos, debe tener en cuenta lo siguiente:

- No es posible sobrescribir un método static pero sí puede ocultarse. Para poder sobrescribir un método, tiene que ser no estático. La existencia de dos métodos estáticos con la misma firma en una jerarquía de clases simplemente significa que son dos métodos de clase independientes. Si se aplica un método de clase a una referencia de objeto, el método llamado será el correspondiente a la clase para la que se haya declarado la variable.
- El método main() es un método static porque JVM no crea ninguna instancia de la clase cuando lo ejecuta. Por tanto, si tiene datos de miembros, debe crear un objeto para acceder a ellos.

INICIALIZADORES ESTÁTICOS

Una clase puede contener código en bloques estáticos que no forman parte de los métodos normales. El código de los bloques estáticos se ejecuta una vez que se ha cargado la clase. Si una clase contiene varios bloques estáticos, éstos se ejecutan por orden de aparición en la clase.

```
public class Count4 {

    public static int counter;

    static {
        counter = Integer.getInteger("myApp.Count4.counter").intValue();
    }
}
```

Gráfico 89. Ejemplo inicializador estático

En el código de la clase Count4 se ha utilizado un método estático de la clase Integer getInteger(String), el cual devuelve un objeto Integer que representa el valor de una propiedad del sistema. Esta propiedad, denominada myApp.Count4.counter, se define en la línea de comandos con la opción -D. El método intValue del objeto Integer devuelve el valor como un tipo int.

```
public class TestStaticInit {

    public static void main(String[] args) {
        System.out.println("counter = " + Count4.counter);
    }
}
```

Gráfico 90. Clase principal que muestra la variable estática

El resultado es el siguiente:

```
java -DmyApp.Count4.counter=47 TestStaticInit
counter = 47
```

Gráfico 91. Ejemplo de ejecución

Todo el código de este ejemplo lo encontrareis en **Ejemplo15_RecursosEstaticos.zip**

PALABRA CLAVE FINAL

La palabra clave final puede tener diferentes significados según el contexto en el cual se utilice.

PROPIEDAD FINAL

Cuando declaramos una propiedad final estamos creando una constante. Una constante es una variable que una vez asignado un valor, este no se puede modificar.

En el siguiente ejemplo, vemos la propiedad id declarada como final, esta se inicializa en el constructor a partir de un contador estático.

Una vez que hemos asignado un valor a la propiedad id este no podrá ser modificado.

```
public class Persona {

    final int id;
    String nombre;
    String apellido;

    static int contador = 0;

    public Persona() {
        id = ++contador;
    }

    public Persona(String nombre, String apellido) {
        id = ++contador;
        this.nombre = nombre;
        this.apellido = apellido;
    }

}
```

Gráfico 92. Ejemplo propiedad final

METODO FINAL

Cuando declaramos un método como final, este no se puede sobrescribir.

En el siguiente ejemplo hemos agregado el método mostrarInformacion a la clase Persona. Dicho método lo hemos declarado como final lo que significa que las subclases de Persona no podrán redefinir este método.

```
public final void mostrarInformacion(){
    System.out.println("ID: " + id +
                       " NOMBRE: " + nombre +
                       " APELLIDO: " + apellido);
}
```

Gráfico 93. Ejemplo método final

CLASE FINAL

Una clase final es una clase que no soporta herencia, esto es, no se puede crear una subclase de ella.

```
public final class Alumno {

    private int id;
    private String nombre;
    private float notaMedia;

    public Alumno() {
    }

    public Alumno(int id, String nombre, float notaMedia) {
        this.id = id;
        this.nombre = nombre;
        this.notaMedia = notaMedia;
    }
```

Gráfico 94. Ejemplo fragmento de clase final

Todo el código de este ejemplo lo encontrareis en **Ejemplo16_RecursosFinales.zip**

TIPOS ENUMERADOS

Una práctica habitual en programación es tener un número finito de nombres simbólicos que representan los valores de un atributo. Por ejemplo, sería posible crear un conjunto de símbolos que representasen los palos de una baraja de cartas: PICAS, CORAZONES, TREBOLES y DIAMANTES. Esta forma de representación se suele denominar tipo enumerado.

La versión 5.0 de Java SE incluye una modalidad de tipos enumerados que mantiene la seguridad de los tipos (typesafe enum). En el siguiente ejemplo se muestra un tipo enumerado para representar los palos de una baraja de cartas. Piense en el tipo Palo como en una clase con un conjunto finito de valores que reciben los nombres simbólicos incluidos en la definición del tipo. Por ejemplo, Palo.PICAS es del tipo Palo.

```
public enum Palo {

    PICAS,
    CORAZONES,
    TREBOLES,
    DIAMANTES
}
```

Gráfico 95. Ejemplo de tipo enumerado

El siguiente código muestra cómo la clase NaipeBaraja utiliza el tipo Palo para el tipo de datos del atributo palo.

```
public class NaipeBaraja {
    private Palo palo;
    private int rango;

    public NaipeBaraja(Palo palo, int rango) {
        this.palo = palo;
        this.rango = rango;
    }

    public String getNamePalo() {
        String name = "";
        switch (palo) {
            case PICAS:
                name = "Picas";
                break;
            case CORAZONES:
                name = "Corazones";
                break;
            case TREBOLES:
                name = "Treboles";
                break;
            case DIAMANTES:
                name = "Diamantes";
                break;
            default:
            // No es necesario comprobar si hay errores ya que
            // el enum Palo es finito.
        }
        return name;
    }
```

Gráfico 96. Fragmento de clase que utiliza el tipo enumerado

```
public class Main {

    public static void main(String[] args) {
        NaipeBaraja naipe1 = new NaipeBaraja(Palo.PICAS, 2);
        System.out.println("el naipe1 es: " + naipe1.getRango()
                + " de " + naipe1.getNamePalo());

    }
}
```

Gráfico 97. Clase principal donde se crea una carta

TIPOS ENUMERADOS AVANZADOS

Lamentablemente, la clase NaipeBaraja sigue necesitando un método getNamePalo para mostrar en la interfaz gráfica un nombre que resulte fácil de leer al usuario. Si el programa mostrarse el verdadero valor de Palo, mostraría el nombre simbólico del valor del tipo. Por ejemplo, Palo.PICAS aparecería como PICAS. Esto resulta más reconocible para el usuario que “0”, pero no tanto como Picas.

Por otro lado, el nombre del palo no debería formar parte de la clase NaipeBaraja sino, más bien, del tipo Palo. La nueva modalidad de tipos enumerados permite usar atributos y métodos, igual que en las clases normales.

El siguiente código muestra una versión mejorada del primer tipo Palo con un atributo nombre y un método getName. Observe cómo se oculta la información de la forma adecuada con el atributo “private” y el método de acceso “public”.

```
public enum Palo {

    PICAS("Picas"),
    CORAZONES("Corazones"),
    TREBOLES("Treboles"),
    DIAMANTES("Diamantes");

    private final String name;

    private Palo(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Gráfico 98. Tipo enumerado avanzado

Un constructor enum siempre debería utilizar acceso privado. Los argumentos del constructor se suministran después de cada valor declarado. Por ejemplo, en la línea 4, la cadena "Picas" es el argumento del constructor enum para el valor PICAS. Los tipos enumerados pueden tener cualquier cantidad de atributos y métodos.

Por último, se muestra el programa de prueba modificado, que utiliza el nuevo método getName en el tipo Palo. El método getPalo devuelve un valor de Palo y el método getName devuelve el atributo nombre de ese valor de Palo.

```
public class Main {

    public static void main(String[] args) {
        NaipeBaraja naipe1 = new NaipeBaraja(Palo.PICAS, 2);
        System.out.println("el naipe1 es: " + naipe1.getRango()
                + " de " + naipe1.getPalo().getName());

    }
}
```

Gráfico 99. Clase principal donde se usa el tipo enumerado

Todo el código de este ejemplo lo encontrareis en **Ejemplo17_Enumerados.zip**

CONSULTA API

El API es el conjunto de todos los recursos (clases, interfaces, tipos enumerados, ...etc.) que podemos utilizar en una aplicación. Todo el API está debidamente documentado y debemos aprender como consultarlo.

Para acceder a la documentación del API abrimos un navegador y en cualquier buscador escribimos API Java 6 y nos llevará a la siguiente página:

docs.oracle.com/javase/6/docs/api/

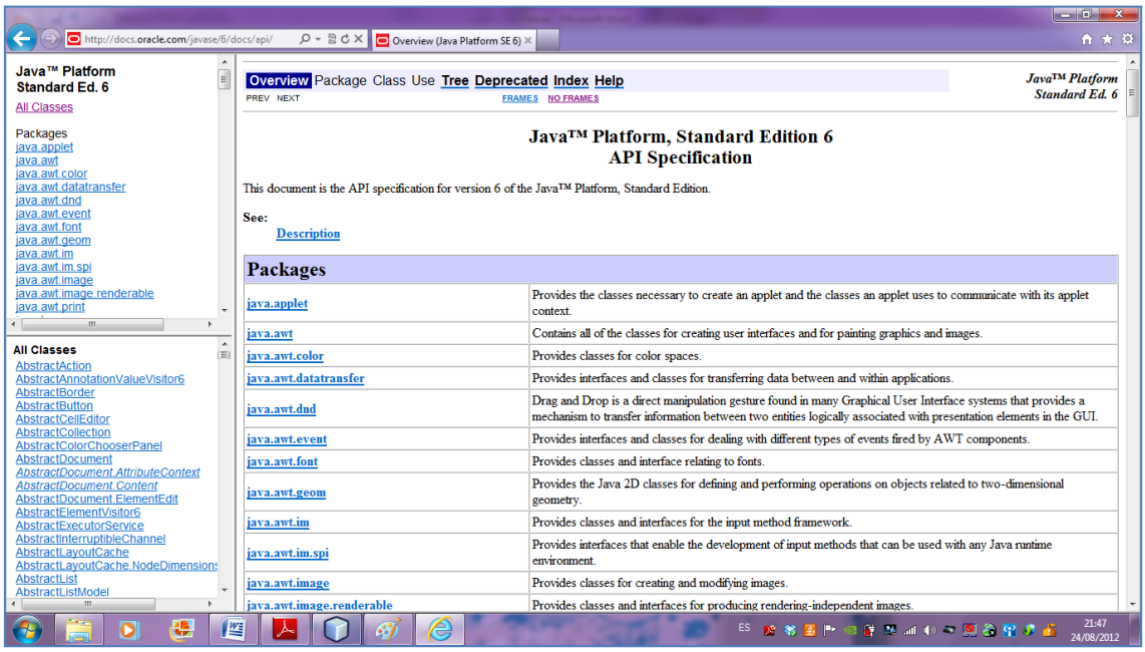


Gráfico 100. Consulta API

En la pagina vista observamos tres marcos o frames.

- En el superior izquierda podremos elegir ver todas las clases o seleccionar un paquete.
- En el marco situado en la parte inferior izquierda vemos todas las clases del paquete seleccionado anteriormente o todas las clases ordenadas alfabéticamente.
- En el marco de la derecha encontraremos toda la información de la clase seleccionada como herencia aplicada, constructores, propiedades, métodos, ...etc.



RECUERDA QUE...

- La herencia me permite heredar los recursos de una superclase siempre y cuando estos no sean constructores, miembros privados o miembros estáticos.
- El polimorfismo se basa en la capacidad de ver un objeto de múltiples formas. El objeto tiene una sola forma, un solo tipo, que será el de la clase instanciada.
- La sobreescritura de métodos me permite redefinir un método heredado.
- La sobrecarga de métodos es la capacidad de tener varios métodos con el mismo nombre. Lo importante es que el compilador pueda distinguir uno de otro.
- La sobrecarga de constructores nos permite crear un objeto con diferente numero de parámetros iniciales.
- Las clases envolventes o envoltorio permiten envolver un tipo primitivo en un objeto.
- Los recursos estáticos o también llamados recursos de clase permiten almacenar datos en la propia clase y no en el objeto.
- Los tipos enumerados permiten crear una variable para un conjunto de datos acotados.
- Una clase abstracta es aquella que tiene uno o varios métodos abstractos, esto es no implementados.
- Una interface define todos sus métodos como abstractos y publicos.
- No se puede crear una instancia de una clase abstracta o interface, pero sí que se pueden utilizar como tipo de una variable.

7.- EXCEPCIONES Y ASERCIONES

Una excepción es un error que ocurre en tiempo de ejecución haciendo que el programa termine de forma inesperada. Veamos un ejemplo:

```
public class AddArguments {
  public static void main(String args[]) {
    int sum = 0;
    for ( String arg : args ) {
      sum += Integer.parseInt(arg);
    }
    System.out.println("Sum = " + sum);
  }
}
```

Gráfico 101. Ejemplo que suma números introducidos como argumentos iniciales

Si se introducen los datos adecuados al ejecutar la clase, el resultado será el esperado.

```
java AddArguments 1 2 3 4
Sum = 10
```

Gráfico 102. Resultado con datos correctos

Ahora bien, que ocurre si no se introducen en el formato esperado? Se produce una excepción y la ejecución del programa se detiene sin llegarse a completar.

```
java AddArguments 1 two 3.0 4
Exception in thread "main" java.lang.NumberFormatException: For input string: "two"
    at java.lang.NumberFormatException.forInputString(NumberFormatException.java:48)
    at java.lang.Integer.parseInt(Integer.java:447)
    at java.lang.Integer.parseInt(Integer.java:497)
    at AddArguments.main(AddArguments.java:5)
```

Gráfico 103. Resultado con datos erróneos

CAPTURAR EXCEPCIONES

Para evitar que el programa termine al generarse una excepción, tenemos la opción de capturarla y de esta forma la ejecución continúa.

Realmente una excepción es un objeto de tipo Exception. Cuando la máquina virtual se encuentra con un error que no puede resolver genera una instancia de una clase que hereda de Exception. El tipo de clase elegido será el del tipo de problema encontrado.

Si utilizamos un bloque **try-catch** para capturar excepciones, lo que realmente pretendemos es capturar ese objeto generado por la maquina virtual. De esta forma evitamos que se interrumpa la ejecución.

Utilizamos el bloque **try** para encerrar aquellas instrucciones "peligrosas", nos referimos a las sentencias donde se podría generar una excepción.

El bloque **catch** es donde capturamos la excepción. Veamos el ejemplo anterior mejorado.

```
public class AddArguments2 {
  public static void main(String args[]) {
    try {
      int sum = 0;
      for ( String arg : args ) {
        sum += Integer.parseInt(arg);
      }
      System.out.println("Sum = " + sum);
    } catch (NumberFormatException nfe) {
      System.err.println("One of the command-line "
                         + "arguments is not an integer.");
    }
  }
}
```

Gráfico 104. Ejemplo try-catch

La ejecución daría el siguiente resultado:

```
java AddArguments2 1 two 3.0 4
One of the command-line arguments is not an integer.
```

Gráfico 105. Resultado tras capturar excepciones

Qué está ocurriendo? Por qué no muestra el resultado de la suma? El problema es el siguiente:

Cuando se lee el primer argumento ("1") se realiza la conversión a tipo entero y se acumula a la suma. Cuando se lee el segundo argumento ("two"), al intentar hacer la conversión a tipo entero se genera una excepción de tipo NumberFormatException, en este momento se ejecuta el bloque catch para capturar dicha excepción.

El resto de los argumentos ya no se evalúan y tampoco se imprime el resultado final de la suma.

Para evitar esto, todavía podríamos mejorar más nuestro código haciendo que sólo la instrucción peligrosa forme parte del bloque try. De esta forma, se evaluarán todos los argumentos y se mostrará la suma. Veamos cómo hacerlo.

```
public class AddArguments3 {
  public static void main(String args[]) {
    int sum = 0;
    for ( String arg : args ) {
       try {
         sum += Integer.parseInt(arg);
       } catch (NumberFormatException nfe) {
         System.err.println("[" + arg + "] is not an integer"
                            + " and will not be included in the sum.");
       }
    }
    System.out.println("Sum = " + sum);
  }
}
```

Gráfico 106. Ejemplo mejorado de try-catch

La ejecución dará ahora el siguiente resultado:

```
java AddArguments3 1 two 3.0 4
[two] is not an integer and will not be included in the sum.
[3.0] is not an integer and will not be included in the sum.
Sum = 5
```

Gráfico 107. Resultado del ejemplo mejorado

Podemos utilizar múltiples bloques catch pero debemos seguir la regla de declararlos del más especifico al más genérico. En caso contrario se genera un error de compilación.

También podemos utilizar **try-catch-finally**. El bloque finally nos asegura que se ejecutará siempre haya excepción o no.

Un buen ejemplo de uso del bloque **finally** es cuando abrimos una conexión a la base de datos y siempre se debería de cerrar se haya producido una excepción o no. Pues bien, el cierre de la conexión iría dentro del bloque finally.

MANEJO DE EXCEPCIONES MÚLTIPLES

Se ha mejorado también el manejo de excepciones múltiples, que ahora se pueden atrapar usando un único bloque. Hasta ahora, si tenías un método con, por ejemplo, tres excepciones, tenías que atraparlas de forma individual. Con Java 7 se puede hacer lo siguiente:

```
public void newMultiCatch( ){
        try{
                methodThatThrowsThreeExceptions( );
        }catch(ExceptionOne | ExceptionTwo | ExceptionThree e){
                //maneja las excepciones
        }
}
```

Sin embargo, si tienes excepciones que pertenecen a distintos tipos que deben manejarse de distinta manera, se puede hacer lo siguiente:

```
public void newMultiMultiCatch( ){
        try{
                methodThatThrowsThreeExceptions( );
        }catch (ExceptionOne e){
                //maneja ExceptionOne
        }catch (ExceptionTwo | ExceptionThree e){
                //maneja ExceptionTwo y ExceptionThree
        }
}
```

API EXCEPTION

El API Exception nos permite ver las diferentes clases de las excepciones y la forma en que están organizadas.

Como vemos en el diagrama, la clase **Exception** hereda de la clase **Throwable** y hermana de ella tenemos la clase **Error**.

Los errores no se pueden capturar. Cuando ocurren el programa termina sin que podamos hacer nada.

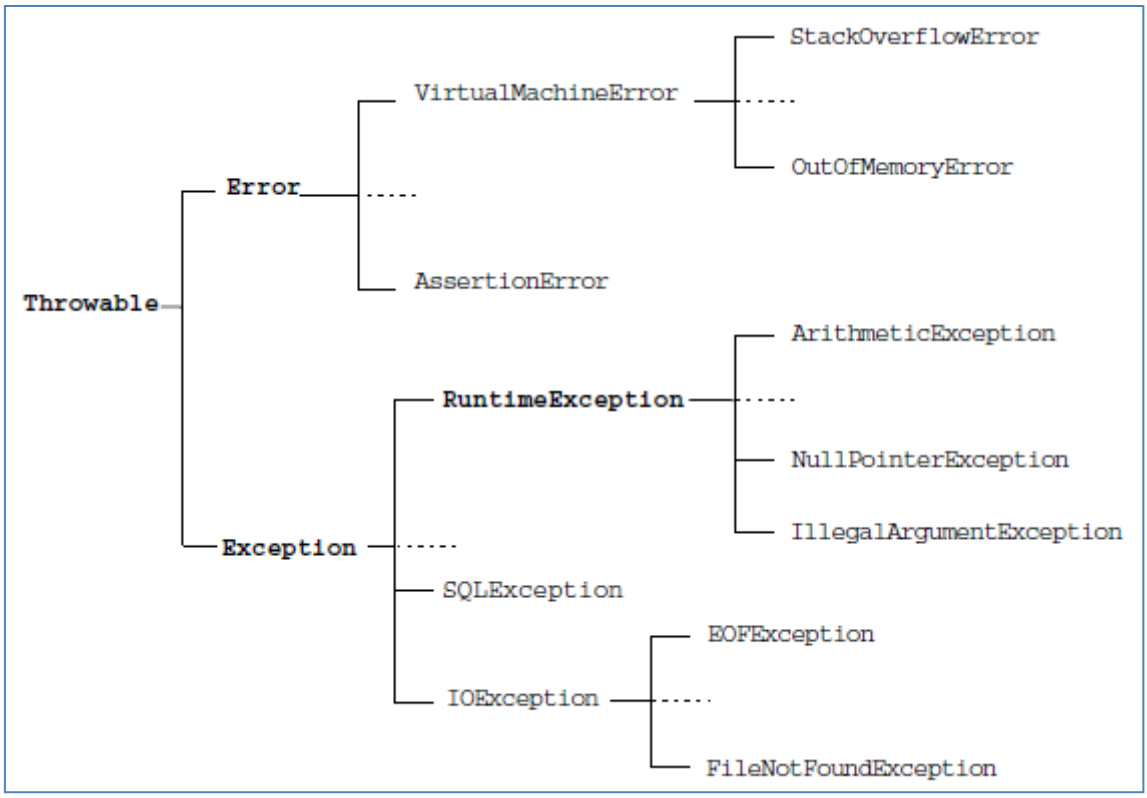


Gráfico 108. Categorías de Excepciones

PROPAGAR EXCEPCIONES

Si no queremos capturar una excepción generada tenemos otra opción que es lanzarla.

Para lanzar una excepción añadimos la clausula **throws** en el método donde se puede generar la excepción.

En este caso, la excepción se propaga a la instrucción donde se ha efectuado la llamada al método.

Veamos ejemplos:

```
void trouble() throws IOException { ... }
void trouble() throws IOException, MyException { ... }
```

Gráfico 109. Métodos que lanzan excepciones

REGLAS DE SOBREESCRITURA DE METODOS QUE LANZAN EXCEPCIONES

Si queremos sobrescribir o redefinir un método con clausula throws aquí tenemos las reglas a seguir:

- Se puede sobrescribir el método eliminado la cláusula throw

- No se puede sobrescribir para que lance más excepciones que las definidas
- Puede lanzar excepciones de una subclase de la definida originalmente.
- No puede lanzar excepciones que sean de la superclase de la definida en un principio.

EXCEPCIONES PERSONALIZADAS

A pesar de que el API Exception nos ofrecen muchas clases para manejar excepciones, muchas veces nos vamos a encontrar con la situación de querer crear nuestra propia excepción, a eso lo denominamos excepciones personalizadas o propias.

Gracias al mecanismo de herencia, para generar nuestra propia excepción es suficiente con crear una clase que herede de la clase Exception. Veamos un ejemplo:

```
public class ServerTimedOutException extends Exception {
  private int port;

  public ServerTimedOutException(String message, int port) {
    super(message);
    this.port = port;
  }

  public int getPort() {
    return port;
  }
}
```

Gráfico 110. Excepción personalizada

La maquina virtual no reconoce nuestra nueva excepción por lo cual no va a generar objetos de este tipo, lo tendremos que hacer nosotros manualmente.

Para generar una excepción personalizada es tan fácil como crear una instancia de la clase y lanzarla con la cláusula throw. Veamos como:

```
public void connectMe(String serverName)
       throws ServerTimedOutException {
  boolean successful;
  int portToConnect = 80;

  successful = open(serverName, portToConnect);

  if ( ! successful ) {
    throw new ServerTimedOutException("Could not connect",
                                      portToConnect);
  }
}
```

Gráfico 111. Creación de excepción personalizada

En este ejemplo estamos propagando la excepción con throws. También podríamos haberla capturado con un bloque try-catch o try-catch-finally.

Todo el código de este ejemplo lo encontrareis en **Ejemplo18_Excepciones.zip**

GESTIÓN DE ASERCIONES

Una aserción es una comprobación que se hace en un punto del programa para verificar que realmente estamos utilizando los datos correctos.

La sintaxis para incluir una aserción es cualquiera de las siguientes:

assert <expresion_booleana> ;

assert < expresion_booleana > : <mensaje_error> ;

Si la expresión booleana se evalúa a false, entonces se lanza un AssertionError. Como vimos anteriormente los errores no se pueden capturar y el programa finaliza.

Veamos un ejemplo: Estamos ejecutando un código y llegados a este punto pensamos que x siempre toma un valor mayor o igual a 0. Por lo cual si la condición devuelve false pensamos que x vale 0.

```
if (x > 0) {
  // do this
} else {
  // do that
}
```

Gráfico 112. Ejemplo sin utilizar aserciones

Pues bien, el uso de las aserciones nos permite verificar esta suposición. Si x vale 0, la expresión de la aserción se evaluará como true y el programa continuará según lo previsto. Pero si x toma un valor negativo entonces la expresión devolverá false y se

generará un AssertionError haciendo que el programa se detenga. Esto es lo que realmente queremos, que el programa lance el error para poder depurarlo.

Veamos el ejemplo anterior mejorado.

```
if (x > 0) {
  // do this
} else {
  assert ( x == 0 );
  // do that, unless x is negative
}
```

Gráfico 113. Ejemplo aserciones

La ventaja de utilizar aserciones es evitar el uso de condicionales innecesarios que una vez depurado todo el código podremos desactivar y no llevarlas a producción.

BUENAS PRÁCTICAS CON ASERCIONES

Debemos saber exactamente qué consecuencias tenemos al desactivar las aserciones. No podemos comprobar mediante aserciones aquellos procesos necesarios en producción.

Se pueden utilizar aserciones para:

- Controlar los invariantes del control de flujo
- Post condiciones e invariantes de la clase
- Invariantes internas

No se deberían utilizar aserciones en los siguientes casos:

- Para chequear los parámetros de un método público.
- No usar métodos en la aserción que puedan causar efectos secundarios, como por ejemplo modificar el valor de una variable.

HABILITAR Y DESHABILITAR ASERCIONES

Por defecto, las aserciones están deshabilitadas. Para habilitarlas recurrimos a las siguientes opciones:

```
java -enableassertions MyProgram
```

o también:

```
java -ea MyProgram
```

Todo el código de este ejemplo lo encontrareis en **Ejemplo19_Aserciones.zip**



RECUERDA QUE...

- Las excepciones son errores que surgen tiempo de ejecución.
- Estas se pueden capturar o lanzar.
- Una aserción es una comprobación que utilizamos únicamente en modo desarrollo, pudiendo habilitarlas o no antes de llevar nuestro código a producción.

8.- COLECCIONES Y GENERICOS

Al trabajar con arrays nos encontrado con una serie de inconvenientes:

- Todos los elementos del array deben ser del mismo tipo
- Al crear el array debemos poner un tamaño inicial, me refiero a especificar el número de elementos que va a contener.
- Los arrays no se pueden redimensionar ya que son estructuras estáticas.

Las colecciones son otra alternativa a los arrays que me permiten almacenar varios elementos en la misma estructura y soluciona los inconvenientes anteriores:

- Los elementos de las colecciones se almacenan como Object por lo cual me permite guardar cualquier tipo de elemento.
- No necesitamos especificar de inicio el número de elementos que va a contener.
- Son estructuras dinámicas por lo cual puedo ir añadiendo tantos elementos como quiera que la colección los admite sin problemas.

API COLLECTIONS

El API Collections contiene interfaces que permiten agrupar objetos en una de las siguientes colecciones:

- **Collection**; Un grupo de objetos denominados elementos, la implementación determina si guardan un orden específico y si se permiten elementos duplicados.
- **Set**; Es un tipo de colección donde no se garantiza que se conserve el orden de entrada de los elementos. Tampoco permite elementos duplicados.
- **List** – En esta colección si se garantiza el orden de entrada y si que se permiten los elementos duplicados.

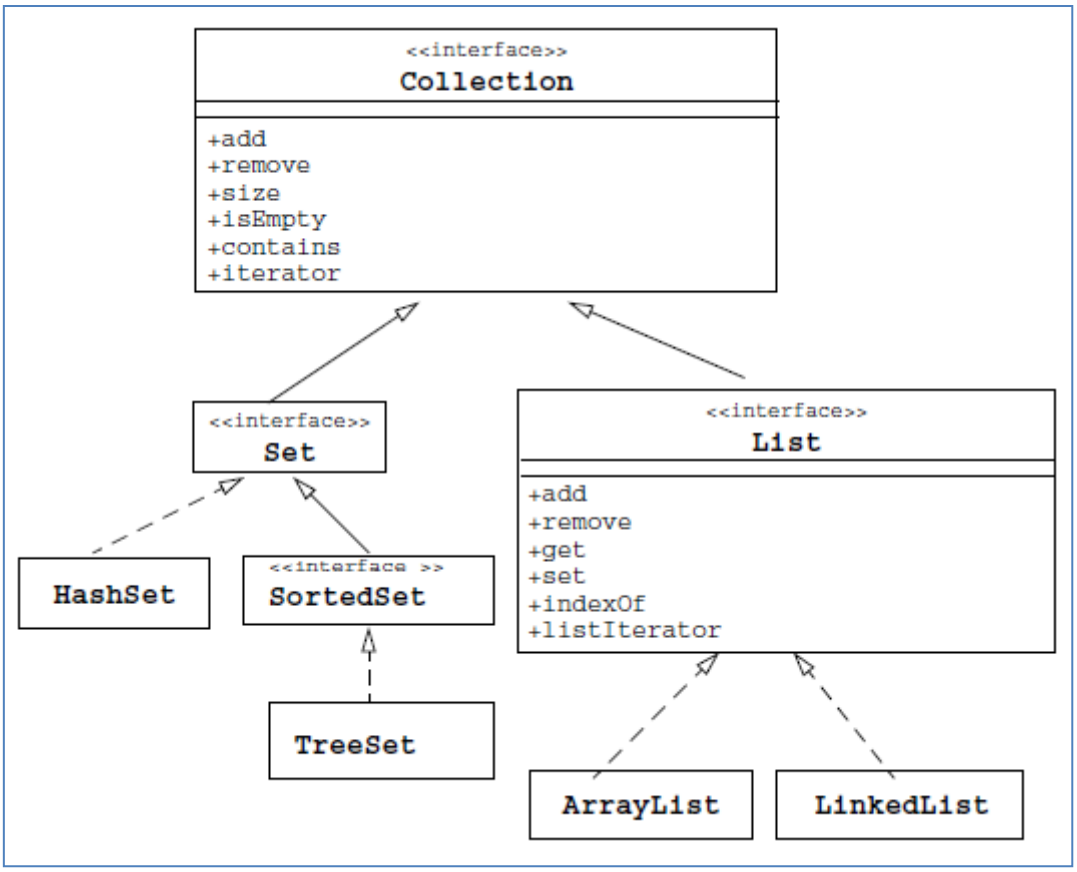


Gráfico 114. API Collections

Las colecciones de tipo SortedSet son las únicas que me permiten almacenar los elementos ordenados. Veremos más adelante como hacerlo.

PRINCIPALES COLECCIONES

A continuación presentamos una tabla donde se recogen las principales clases que implementan las colecciones.

	Hash Table	Array redimensionable	Árbol balanceado	Linked List	Hash Table + Linked List
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

SET

Recordamos que una colección de tipo Set no garantiza el orden de entrada de los elementos y tampoco permite introducir elementos duplicados.

En el siguiente ejemplo vemos como crear una colección de este tipo y añadir unos cuantos elementos.

Si intentamos introducir un elemento repetido simplemente lo ignora sin generar una excepción o error de compilación.

```
// crear una coleccion de tipo Set
Set coleccionSet = new HashSet();
coleccionSet.add("uno");
coleccionSet.add("segundo");
coleccionSet.add(new Integer(4));
coleccionSet.add(new Float(3.15));
coleccionSet.add("segundo");  // los repetidos no se añaden
System.out.println(coleccionSet);
```

Gráfico 115. Ejemplo Set

La ejecución de este ejemplo podría mostrar la siguiente salida.

```
[4, 3.15, segundo, uno]
```

LIST

Una colección de tipo List si que conserva el orden de entrada de los elementos y además permite introducir elementos duplicados.

En este ejemplo vemos como crear una colección de tipo List.

```
// crear una coleccion de tipo List
List coleccionList = new ArrayList();
coleccionList.add("uno");
coleccionList.add("segundo");
coleccionList.add(new Integer(4));
coleccionList.add(new Float(3.15));
coleccionList.add("segundo");  // los repetidos SI se añaden
System.out.println(coleccionList);
```

Gráfico 116. Ejemplo List

Tras la ejecución del código este es el resultado que veremos en la consola.

```
[uno, segundo, 4, 3.15, segundo]
```

MAP

Un mapa no se considera una colección ya que no hereda de la interface Collection.

Los elementos de un mapa se forman como clave-valor. Además tienen las siguientes restricciones:

- Las claves duplicadas no están permitidas.
- Una clave solo puede referenciar un valor, no varios.

La interface Map define una serie de métodos para manipular el mapa, los más interesantes son:

- **entrySet**; Devuelve una colección de tipo Set con todos los elementos (clave-valor).
- **keySet**; Devuelve una colección de tipo Set con todas las claves del mapa.
- **values**; Devuelve un objeto de tipo Collection con todos los valores del mapa.

El API de la interface Map es el siguiente:

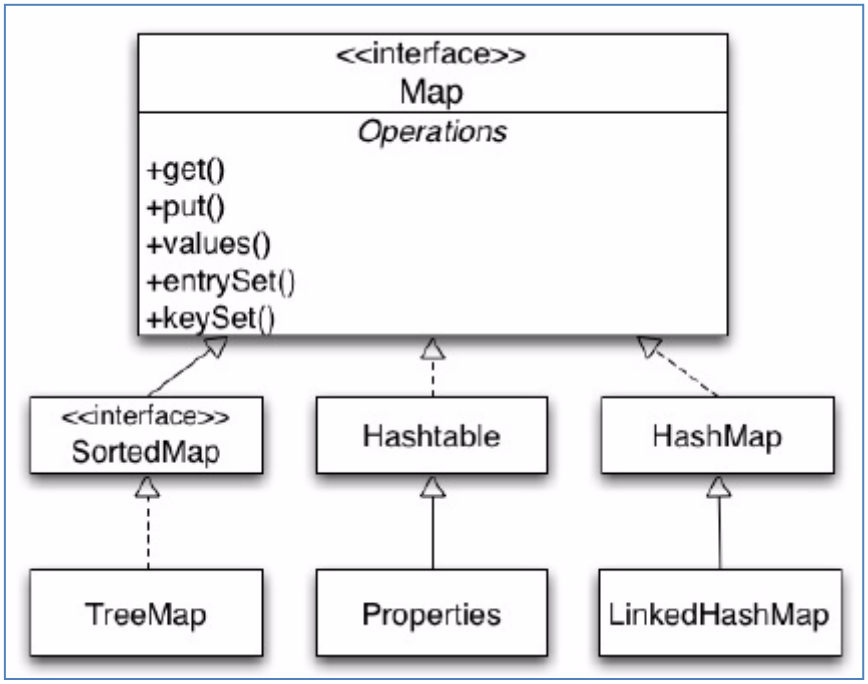


Gráfico 117. API interface Map

Vemos un ejemplo de cómo crear un mapa. Si intentamos introducir un elemento con su clave duplicada, se sobrescribe el valor del elemento.

```
// crear un Mapa
// un mapa no permite claves duplicadas, valores duplicados si.
Map mapa = new HashMap();
mapa.put("1", "uno");
mapa.put("2", "dos");
mapa.put("1", "tres"); // sobreescribe el elemento

// mostrar todas las claves (keys)
System.out.println(mapa.keySet());

// mostrar todos los valores (values)
System.out.println(mapa.values());

// mostrar todos los elementos como pares key-value
System.out.println(mapa.entrySet());
```

Gráfico 118. Ejemplo Map

El resultado tras la ejecución sería el siguiente:

```
[2, 1]
[dos, tres]
[2=dos, 1=tres]
```

Todo el código de este ejemplo lo encontrareis en **Ejemplo20_Colecciones.zip**

ORDENAR COLECCIONES

Cuando mencionamos el concepto ordenar colecciones nos estamos refiriendo a mantener los elementos clasificados dentro de la colección, por ejemplo, alfabéticamente, cronológicamente, ascendente, ...etc.

Para lograr este objetivo utilizamos las interfaces **Comparable** y **Comparator**.

INTERFACE COMPARABLE

La interface Comparable permite definir el orden natural de los elementos. Ejemplos de clases que implementan dicha interface son:

- **String**; En los elementos de este tipo se define un orden alfabético.
- **Date**; En los elementos de este tipo se define un orden cronológico

- **Integer**; En los elementos de este tipo se define un orden numérico

La interface Comparable define un único método:

```
int compareTo(Object o)
```

Este método lo debemos implementar y el valor entero a devolver tendrá el siguiente significado:

- 0; si los objetos a comparar son iguales
- 1; si la instancia en la cual estamos es mayor que el objeto pasado como argumento.
- -1; si la instancia en la cual estamos es menor que el objeto pasado como argumento.

En el siguiente ejemplo, vemos que queremos ordenar las instancias de tipo Cliente atendiendo a su cifra de ventas.

```
public class Cliente implements Comparable{

    String nombre;
    double cifraVentas;
    String nif;

    public int compareTo(Object o) {
        Cliente otroCliente = null;
        int resultado = 0;
        if (o instanceof Cliente){
            otroCliente = (Cliente) o;
        }
        // 0 --> los objetos son iguales
        if (cifraVentas == otroCliente.cifraVentas){
            resultado = 0;
        }
        // 1 --> es mayor
        else if(cifraVentas > otroCliente.cifraVentas)
        {
            resultado = 1;
        }
        // -1 --> es menor
        else if(cifraVentas < otroCliente.cifraVentas)
        {
            resultado = -1;
        }
        return resultado;
    }
```

Gráfico 119. Clase que implementa la interface Comparable

Para mantener los elementos ordenados necesitamos una colección que implemente la interface SortedSet que como vimos antes son las únicas que permiten este propósito.

```
public static void main(String[] args) {
    // crear una coleccion clasifique los objetos --> TreeSet
    TreeSet arbol = new TreeSet();

    arbol.add(new Cliente("Juan", 1500, "12345678-A"));
    arbol.add(new Cliente("Jose", 1200, "1245778-A"));
    arbol.add(new Cliente("Maria", 1800, "2222222-A"));
    arbol.add(new Cliente("Laura", 1100, "88888888-A"));

    Object[] array = arbol.toArray();

    for (int i=0; i<array.length; i++){
        System.out.println((Cliente)array[i]);
    }

}
```

Gráfico 120. Clase principal que añade elementos al árbol para ordenarlos

El resultado de la ejecución sería el siguiente:

```
Cliente{ nombre=Laura cifraVentas=1100.0 nif=88888888-A}
Cliente{ nombre=Jose cifraVentas=1200.0 nif=1245778-A}
Cliente{ nombre=Juan cifraVentas=1500.0 nif=12345678-A}
Cliente{ nombre=Maria cifraVentas=1800.0 nif=2222222-A}
```

Gráfico 121. Resultado de ejecución

INTERFACE COMPARATOR

Al implementar la interface Comparable en la clase Cliente, estamos dando por hecho que todos los objetos Cliente siempre los vamos a ordenar por su cifra de ventas.

Ahora bien. Y si resulta que unas veces quiero ordenarlos por el nombre y otras veces por el nif por ejemplo. La solución nos la aporta la interface Comparator.

La interface Comparator define un único método:

```
int compare(Object o1, Object o2)
```

Gráfico 122. Sintaxis del método compare

También devolvemos un numero entero con el mismo significado que antes.

- 0; si los objetos a comparar son iguales

- 1; si el primer objeto es mayor que el segundo.
- -1; si el primer objeto es menor que el segundo.

Veamos un ejemplo: Ahora la clase Cliente no implementa ninguna interface.

```
public class ClienteSimple{

    String nombre;
    double cifraVentas;
    String nif;

    public ClienteSimple(String nombre, double cifraVentas, String nif) {
        this.nombre = nombre;
        this.cifraVentas = cifraVentas;
        this.nif = nif;
    }
```

Gráfico 123. Clase ClienteSimple

Generamos un comparador en una clase aparte. En este caso para ordenar por nombre.

```
public class ComparadorNombre implements Comparator{

    public int compare(Object o1, Object o2) {
        return (((ClienteSimple)o1).nombre.compareTo(((ClienteSimple)o2).nombre) );
    }

}
```

Gráfico 124. Clase ComparadorNombre

También nos podemos generar otro comparador para ordenar por el nif.

```
public class ComparadorNif implements Comparator{

    public int compare(Object o1, Object o2) {
        return (((ClienteSimple)o1).nif.compareTo(((ClienteSimple)o2).nif) );
    }

}
```

Gráfico 125. Clase ComparadorNif

A la hora de crear la colección que nos permite tener ordenados los objetos, debemos pasarle una instancia del comparador elegido. Esto especifica si queremos ordenar los elementos por el nombre o por el nif. Veamos como:

```
public static void main(String[] args) {
    // crear una coleccion clasifique los objetos --> TreeSet
    ComparadorNombre c = new ComparadorNombre();
    ComparadorNif nif = new ComparadorNif();

    // Elegimos ordenar los clientes por su nombre
    TreeSet arbol = new TreeSet(c);
    //TreeSet arbol = new TreeSet(nif);

    arbol.add(new ClienteSimple("Juan", 1500, "1111-A"));
    arbol.add(new ClienteSimple("Jose", 1200, "3333-B"));
    arbol.add(new ClienteSimple("Maria", 1800, "2222-C"));
    arbol.add(new ClienteSimple("Laura", 1100, "0000-D"));

    Object[] array = arbol.toArray();

    for (int i=0; i<array.length; i++){
        System.out.println((ClienteSimple)array[i]);
    }
}
```

Gráfico 126. Clase principal que añade elementos al árbol para luego ordenarlos

Todo el código de este ejemplo lo encontrareis en
Ejemplo21_OrdenarColecciones.zip

GENÉRICOS

Como hemos mencionado una colección almacena todos sus elementos como Object, por lo cual a la hora de recuperar dichos elementos debemos efectuar el casting apropiado.

```
ArrayList list = new ArrayList();
list.add(0, new Integer(42));
int total = ((Integer)list.get(0)).intValue();
```

Gráfico 127. Recuperar elementos de una colección

Utilizar un tipo genérico a la hora de crear la colección nos permite:

- Eliminar la necesidad de casting
- Controlar en tiempo de compilación que todos los elementos son del tipo genérico.

```
ArrayList<Integer> list = new ArrayList<Integer>();
list.add(0, new Integer(42));
int total = list.get(0).intValue();
```

Gráfico 128. Recuperar elementos de una colección usando genéricos

Veamos un ejemplo: Creamos una colección y definimos como tipo genérico String, esto nos permite controlar que todos los elementos introducidos en la colección son de tipo String. Si no fuese así e intentásemos introducir instancias de otro tipo se produciría un error de compilación.

Como observamos en el bucle for-each ahora recuperamos todos los elementos como String y no como Object en el caso de no utilizar genéricos.

```
public static void main(String[] args) {
    Set<String> coleccionSet = new HashSet<String>();
    coleccionSet.add("uno");
    coleccionSet.add("segundo");

    // si no es de tipoString genera un error de compilacion
    // coleccionSet.add(new Integer(4));
    // coleccionSet.add(new Float(3.15));
    coleccionSet.add("segundo");  // los repetidos no se añaden
    System.out.println(coleccionSet);

    for (String elemento : coleccionSet)
            System.out.println(elemento);
}
```

Gráfico 129. Recuperar elementos de una colección usando el método for-each

Aquí tenemos otro ejemplo de cómo utilizar genéricos con mapas.

```
import java.util.*;

public class MapPlayerRepository {
  HashMap<String, String> players;

  public MapPlayerRepository() {
    players = new HashMap<String, String> ();
  }

  public String get(String position) {
    String player = players.get(position);
    return player;
  }

  public void put(String position, String name) {
    players.put(position, name);
  }
```

Gráfico 130. Genéricos con mapas

La siguiente tabla recoge como utilizar los tipos genéricos:

Category	Non Generic Class	Generic Class
Class declaration	`public class ArrayList extends AbstractList implements List`	`public class ArrayList<E> extends AbstractList<E> implements List <E>`
Constructor declaration	`public ArrayList (int capacity);`	`public ArrayList<E> (int capacity);`

Method declaration	`public void add((Object o)`
	
	`public Object get(int index)`
		`public void add(E o)`
		
		`public E get(int index)`

Variable declaration examples	`ArrayList list1;`
	
	`ArrayList list2;`
		`ArrayList <String> list1;`
		
		`ArrayList <Date> list2;`

Instance declaration examples	`list1 = new ArrayList(10);`
	
	`list2 = new ArrayList(10);`
		`list1= new ArrayList<String> (10);`
		
		`list2= new ArrayList<Date> (10);`

Gráfico 131. Tabla que recoge los usos de los tipos genéricos

El API Collections con genéricos:

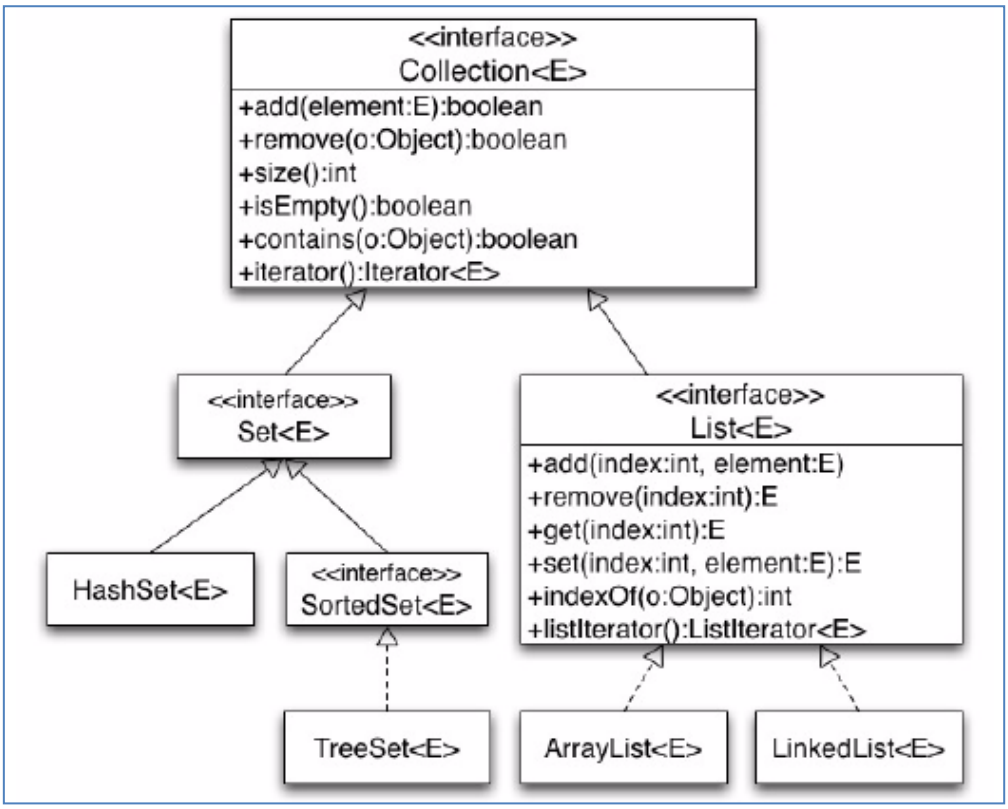


Gráfico 131. API Collections con tipos genéricos

USO DE GENERICOS CON JAVA 7

Una de las mejoras que se reclamaban en este proyecto era sobre el uso de Genéricos. Un ejemplo del uso de Genéricos hasta Java 7 sería como sigue:

```
Map<String, List<Trade>> trades = new TreeMap<String, List<Trade>>( );
```

Esto resulta un poco lioso, porque hay que declarar los tipos a ambos lados. Observad ahora como quedaría usando Java 7:

```
Map<String, List<Trade>> trades = new TreeMap<>( );
```

Mucho mejor, ¿verdad?. Ya no hay que declarar los tipos a la derecha, porque el compilador infiere de qué tipos se trata viendo los que hay a la izquierda. Incluso sería legal omitir el operador de Genéricos (diamond operator), así:

```
trades = new TreeMap( );
```

pero el compilador lanzaría advertencias de seguridad.

ITERADORES

Para recorrer colecciones podemos utilizar iteradores. Veamos un ejemplo:

```
List<Student> list = new ArrayList<Student>();
// add some elements
Iterator<Student> elements = list.iterator();
while (elements.hasNext()) {
  System.out.println(elements.next());
}
```

Gráfico 132. Creación de un iterador con tipo genérico

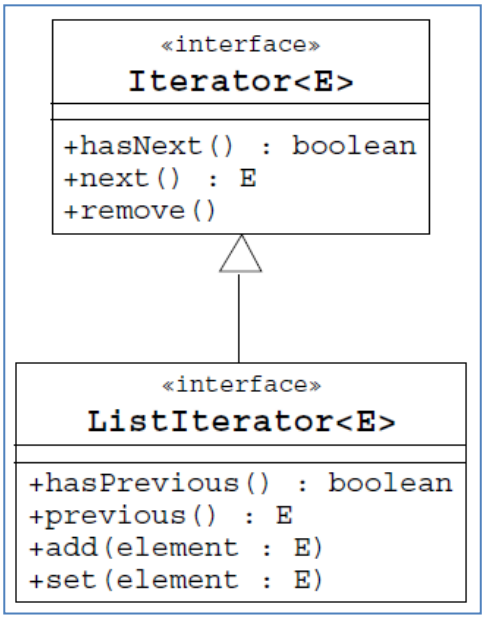


Gráfico 133. Diagrama UML de Iterator

Todo el código de este ejemplo lo encontrareis en **Ejemplo22_Genericos.zip**



RECUERDA QUE...

- Las colecciones es la forma de poder almacenar varios objetos de forma dinámica
- Podemos ordenar los objetos de una colección con las interfaces: Comparable y Comparator.
- Los tipos genéricos ayudan a comprobar el tipo de los objetos agregados en una colección en tiempo de compilación.

9.- MECANISMOS DE ENTRADA / SALIDA

ENTRADA DE ARGUMENTOS

En Java podemos introducir argumentos por línea de comandos en el momento de ejecutar la clase.

Estos argumentos se almacenan como elementos de tipo String en el array que se pasa al método estático **main**.

Veamos un ejemplo. En este código recorremos el array de argumentos pasado al método main y mostramos el valor de cada uno de ellos. Como vemos el array está declarado como tipo String, esto no se puede cambiar por lo cual todos los argumentos son de tipo texto.

```
public class TestArgs {
  public static void main(String[] args) {
    for ( int i = 0; i < args.length; i++ ) {
      System.out.println("args[" + i + "] is '" + args[i] + "'");
    }
  }
}
```

Gráfico 134 . Código que procesa argumentos de entrada

```
java TestArgs arg0 arg1 "another arg"
args[0] is 'arg0'
args[1] is 'arg1'
args[2] is 'another arg'
```

Gráfico 135. Ejemplo de ejecución

Todo el código de este ejemplo lo encontrareis en **Ejemplo23_Argumentos.zip**

PROPIEDADES

Las propiedades son una utilidad que vienen a sustituir el concepto de variables de entorno. Podemos utilizarlas para añadir información adicional a nuestras clases.

La clase **Properties** es una implementación de Map por lo cual todos sus elementos se componen de clave-valor ambos de tipo String.

Esta clase expone los siguientes métodos útiles:

- **propertyNames**; devuelve un Enumeration con todos los nombres de las propiedades.
- **getProperty**; este método recibe el nombre de la propiedad y devuelve el valor asociada a esta como un String

El método **System.getProperties** devuelve una instancia de tipo Properties

Podemos utilizar la opción **-D** para incluir nuevas propiedades.

```
import java.util.Properties;
import java.util.Enumeration;

public class TestProperties {
  public static void main(String[] args) {
    Properties props = System.getProperties();
    props.list(System.out);
  }
}
```

Gráfico 136. Ejemplo que muestra las propiedades

```
java -DmyProp=theValue TestProperties
```

Gráfico 137. Creamos una nueva propiedad al ejecutar la clase

```
java.runtime.name=Java(TM) SE Runtime Environment
sun.boot.library.path=C:\jse\jdk1.6.0\jre\bin
java.vm.version=1.6.0-b105
java.vm.vendor=Sun Microsystems Inc.
java.vm.name=Java HotSpot(TM) Client VM
file.encoding.pkg=sun.io
user.country=US
myProp=theValue
```

Gráfico 138. Ejemplo de ejecución

Todo el código de este ejemplo lo encontrareis en **Ejemplo24_Propiedades.zip**

FUNDAMENTOS DE ENTRADA Y SALIDA

Un **stream** es un concepto que utilizamos para definir un flujo de datos desde un origen a un destino.

El stream de origen (**source stream**) es donde se inicia el flujo de datos. También se utiliza el término input stream.

El stream de destino (**sink stream**) es donde finaliza el flujo de datos. Podemos reconocerlos como output stream.

Tanto los stream de origen como los stream de destino se conocen como nodos (**node stream**). A lo largo de este tema veremos que hay muchos tipos de nodos: ficheros, memoria, pipes, ...etc.

Los streams pueden ser de dos tipos:

- A nivel de **bytes**; Tienen la ventaja de poder procesar datos de cualquier formato pero el inconveniente es que requieren ajustarse a otros formatos.
- A nivel de **caracteres**; La ventaja es que tenemos un formato textual pero está limitado únicamente a este formato.

La siguiente tabla recoge las clases principales para manejar streams.

Stream	Byte	Character
stream de origen	**InputStream**	**Reader**
stream de destino	**OutputStream**	**Writer**

FLUJOS DE BYTES

La clase **InputStream** tiene los siguientes métodos:

Métodos de lectura:

- int **read**(); Devuelve el numero de bytes leídos o -1 que indica fin de archivo.
- int **read**(byte[] buffer); Los bytes leídos los almacena en el array buffer. Igual que antes devuelve un entero con el numero de bytes leídos o el valor -1.
- int **read**(byte[] buffer, int offset, int length); Los dos argumentos indican la posición de inicio y la longitud dentro del array.

Otros métodos importantes:

- void **close**(); Cierra el flujo.
- int **available**(); Indica el numero de bytes disponibles para su lectura.

- long **skip**(long n); Este método descarta el numero de bytes especificado.

Los métodos de la clase **OutputStream** son:

Métodos de escritura:

- void **write**(int c)
- void **write**(byte[] buffer)
- void **write**(byte[] buffer, int offset, int length)

Otros métodos:

- void **close**()
- void **flush**(); permite forzar las escrituras.

FLUJOS DE CARACTERES

Los métodos de la clase **Reader** son:

Métodos de lectura:

- int **read**()
- int **read**(char[] cbuf)
- int **read**(char[] cbuf, int offset, int length)

Otros métodos:

- void **close**()
- boolean **ready**()
- long **skip**(long n)

Los métodos de la clase **Writer**:

Métodos de escritura:

- void **write**(int c)
- void **write**(char[] cbuf)
- void **write**(char[] cbuf, int offset, int length)
- void **write**(String string)
- void **write**(String string, int offset, int length)

Otros métodos:

- void **close**()
- void **flush**()

CLASES IMPORTANTES

A continuación presentamos una tabla con las clases más importantes para manejar distintos tipos de nodos:

Tipo	Character	Byte
Fichero	FileReader FileWriter	FileInputStream FileOutputStream
Array de memoria	CharArrayReader CharArrayWriter	ByteArrayInputStream ByteArrayOutputStream
String de memoria	StringReader StringWriter	N/A
Pipe	PipedReader PipedWriter	PipedInputStream PipedOutputStream

Un nodo Pipe o tubería es una conexión entre dos streams. Se suele utilizar para desviar el flujo de datos de un stream a otro.

Ejemplo 1:

```
import java.io.*;
public class TestNodeStreams {
  public static void main(String[] args) {
    try {
      FileReader input = new FileReader(args[0]);
      try {
        FileWriter output = new FileWriter(args[1]);
        try {
          char[] buffer = new char[128];
          int charsRead;

          // read the first buffer
          charsRead = input.read(buffer);
          while ( charsRead != -1 ) {
            // write buffer to the output file

            output.write(buffer, 0, charsRead);

            // read the next buffer
            charsRead = input.read(buffer);
          }

        } finally {
          output.close();}
      } finally {
        input.close();}
    } catch (IOException e) {
      e.printStackTrace();
    }
  }
}
```

```
java TestNodeStreams file1 file2
```

Gráfico 139. Ejemplo que lee un fichero para copiar su contenido en otro

En este ejemplo leemos el primer fichero pasado como argumento y escribimos su contenido en el segundo. Utilizamos un buffer manual. Ejemplo 2:

```
import java.io.*;
public class TestBufferedStreams {
  public static void main(String[] args) {
    try {
      FileReader input = new FileReader(args[0]);
      BufferedReader bufInput = new BufferedReader(input);
      try {
        FileWriter output = new FileWriter(args[1]);
        BufferedWriter bufOutput= new BufferedWriter(output);
        try {
          String line;
          // read the first line
          line = bufInput.readLine();
          while ( line != null ) {
            // write the line out to the output file

            bufOutput.write(line, 0, line.length());
            bufOutput.newLine();
            // read the next line
            line = bufInput.readLine();
          }
        } finally {
          bufOutput.close();
        }
      } finally {
        bufInput.close();
      }
    } catch (IOException e) {
      e.printStackTrace();
    }
  }
}
```

```
java TestBufferedStreams file1 file2
```

Gráfico 140. Ejemplo anterior mejorado utilizando buffers

Este ejemplo es similar al anterior pero en vez de utilizar un buffer manual utilizamos las clases BufferedReader y BufferedWriter.

Estas clases nos permiten acumular datos en la lectura y en la escritura. Gracias a esto podemos escribir línea a línea en vez de carácter a carácter.

Todo el código de este ejemplo lo encontrareis en **Ejemplo25_IO_Ficheros.zip**

PROCESAR STREAMS

Un flujo de caracteres lo podemos procesar directamente con el tipo char o String. Como hemos dicho antes, los flujos de bytes son un poco más complejos, necesitan adaptarlos a cualquiera de los tipos de datos que soporta Java, nos referimos a los tipos primitivos.

Las siguientes figuras detallan las clases necesarias para llevar a cabo este proceso.

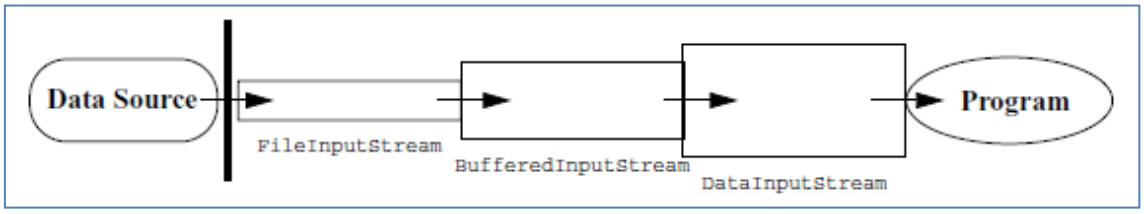


Gráfico 141. Lectura de datos a través de flujos de bytes

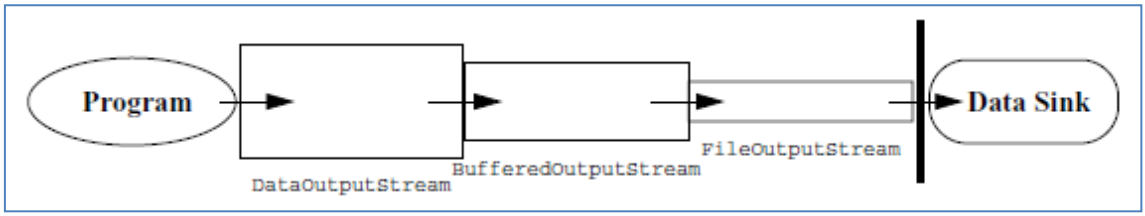


Gráfico 142. Escritura de datos a través de flujos de bytes

Tenemos otras clases para procesar streams, que se detallan en la siguiente tabla:

Tipo	Character	Byte
Buffers	BufferedReader BufferedWriter	BufferedInputStream BufferedOutputStream
Filtros	FilterReader FilterWriter	FilterInputStream FilterOutputStream
Conversión entre bytes y caracteres	InputStreamReader OutputStreamWriter	
Serialización de objetos		ObjectInputStream ObjectOutputStream
Conversión de datos		DataInputStream DataOutputStream
Contadores	LineNumberReader	LineNumberInputStream
Deshacer y Rehacer	PushbackReader	PushbackInputStream
Impresión	PrintWriter	PrintStream

JERARQUIA DE CLASES INPUTSTREAM

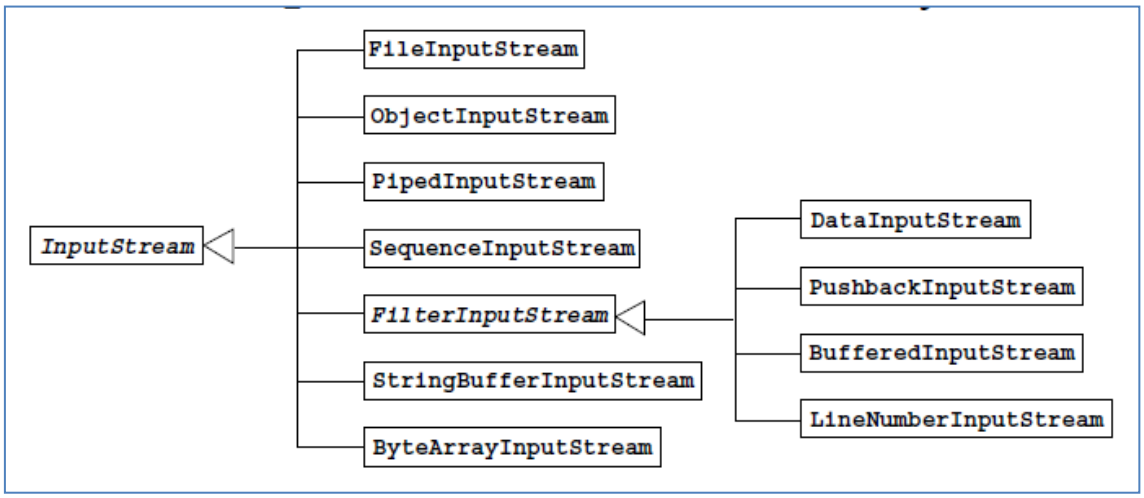


Gráfico 143. Herencia InputStream

JERARQUIA DE CLASES OUTPUTSTREAM

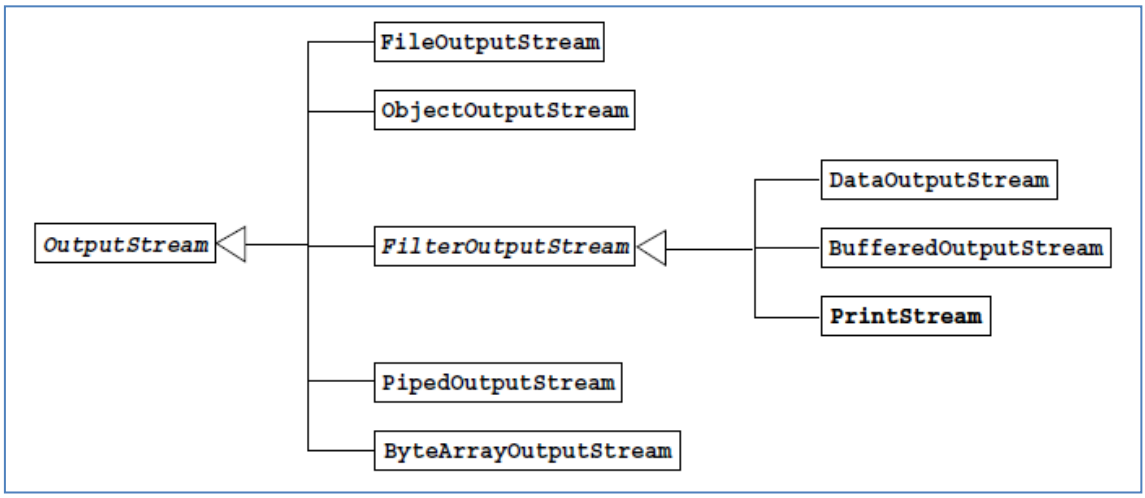


Gráfico 144. Herencia OutputStream

JERARQUIA DE CLASES READER

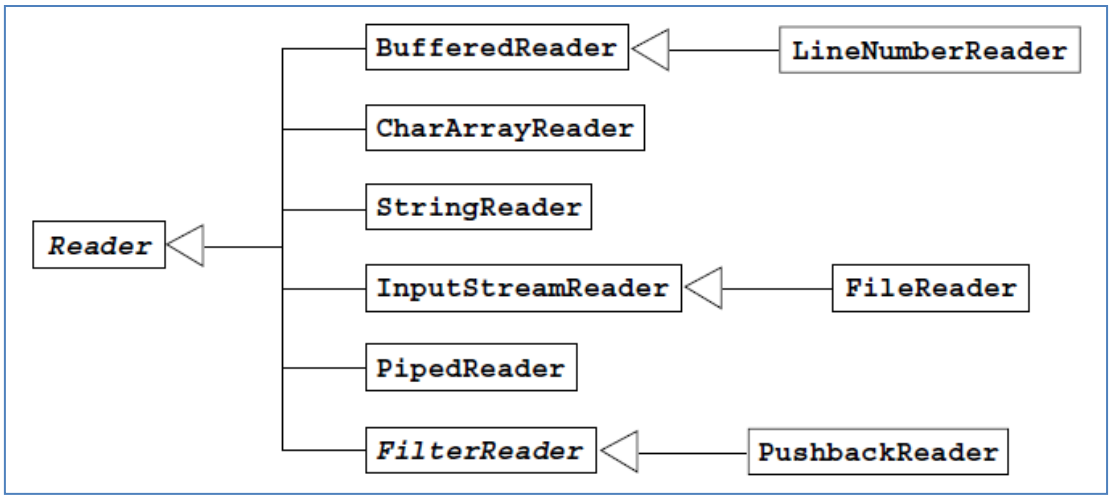


Gráfico 145. Herencia Reader

JERARQUIA DE CLASES WRITER

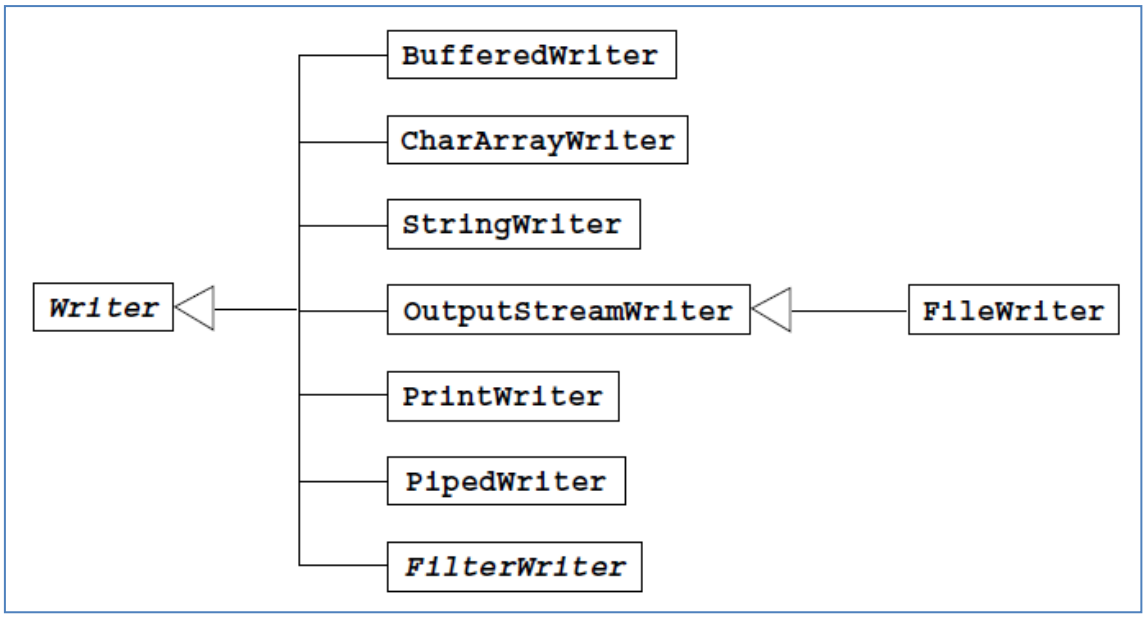


Gráfico 146. Herencia Writer

SERIALIZACIÓN

La **Serialización** es el mecanismo por el cual guardamos el estado de un objeto en un recurso no volátil como un fichero por ejemplo para luego poder volver a reconstruir este objeto, esto se denomina deserialización.

Cuando mencionamos el estado de un objeto nos referimos a las propiedades de este.

Si un objeto está referenciando a otro objeto este también debe ser serializable.

Si una de las propiedades del objeto no la queremos serializar entonces la definimos como **transient**.

Ejemplo de serialización:

```
import java.io.*;
import java.util.Date;

public class SerializeDate {

  SerializeDate() {
    Date d = new Date ();

    try {
      FileOutputStream f =
          new FileOutputStream ("date.ser");
      ObjectOutputStream s =
          new ObjectOutputStream (f);
      s.writeObject (d);
      s.close ();
    } catch (IOException e) {
      e.printStackTrace ();
    }
  }


  public static void main (String args[]) {
    new SerializeDate();
  }
}
```

Gráfico 147. Ejemplo de Serialización

En este ejemplo creamos un objeto de tipo Date y lo serializamos en un fichero con nombre date.ser

Ejemplo de deserialización:

```
import java.io.*;
import java.util.Date;

public class DeSerializeDate {

  DeSerializeDate () {
    Date d = null;

    try {
      FileInputStream f =
          new FileInputStream ("date.ser");
      ObjectInputStream s =
          new ObjectInputStream (f);
      d = (Date) s.readObject ();
      s.close ();
    } catch (Exception e) {
      e.printStackTrace ();
    }

    System.out.println(
      "Deserialized Date object from date.ser");
    System.out.println("Date: "+d);
  }

  public static void main (String args[]) {
    new DeSerializeDate();
  }
}
```

Gráfico 148. Ejemplo de deserialización

En este otro ejemplo deserializamos el objeto creado y guardado en el ejemplo anterior. Para ello leemos el objeto y hacemos un casting a Date. Todos los objetos deserializados se devuelven como tipo Object, por eso es necesario el casting.

Todo el código de este ejemplo lo encontrareis en **Ejemplo26_Serializacion.zip**

ENTRADA Y SALIDA DE CONSOLA

Java permite realizar operaciones de entrada / salida de consola a través de tres variables publicas y estáticas de la clase java.lang.System.

- **System.out**; es una objeto de tipo PrintStream. Es la salida estándar por consola

- **System.in**; es un objeto de tipo InputStream. Es la entrada estándar.
- **System.err**; es un objeto de tipo PrintStream. Es la salida estándar para mostrar errores.

Para realizar impresiones por consola utilizamos los métodos:

- **print**(); Imprime un argumento.
- **println**(); Imprime un argumento y añade un carácter de salto de línea ("\n").
- **printf**(); Imprime con formato. Los diferentes formatos a utilizar los mostramos en la siguiente tabla.

Código	Descripción
%s	Formatea el argumento como String. Normalmente se invoca al método toString().
%d %o %x	Formatea un entero como decimal, octal o hexadecimal respectivamente.
%f %g	Formatea en coma flotante. %g formato con notación científica
%n	Provoca un salto de línea.
%%	Lo utilizamos para añadir el carácter %

Ejemplo: Imprimimos los datos de un alumno, cada código de formato se sustituye por el valor devuelto por los métodos get respectivamente.

```
System.out.printf("Nombre = %s %s ID = %d Nota Media = %.1f\n",
                  s.getNombre(), s.getApellido(), s.alumnoID(), s.getMedia());
```

%.1f estamos expresando que queremos mostrar la nota media con un solo decimal.

El siguiente ejemplo nos permite leer datos por consola.

```
import java.io.*;

public class KeyboardInput {
  public static void main (String args[]) {
    String s;
    // Create a buffered reader to read
    // each line from the keyboard.
    InputStreamReader ir
      = new InputStreamReader(System.in);
    BufferedReader in = new BufferedReader(ir);

    System.out.println("Unix: Type ctrl-d to exit." +
                       "\nWindows: Type ctrl-z to exit");

    try {
      // Read each input line and echo it to the screen.
      s = in.readLine();
      while ( s != null ) {
        System.out.println("Read: " + s);
        s = in.readLine();
      }

      // Close the buffered reader.
      in.close();
    } catch (IOException e) { // Catch any IO exceptions.
      e.printStackTrace();
    }
  }
}
```

Gráfico 149. Solicitar datos por consola

A partir de Java 5, surge la nueva clase Scanner que nos permite leer datos por consola de una forma más rápida y con menos código.

```
import java.io.*;
import java.util.Scanner;
public class ScanTest {
  public static void main(String [] args) {
    Scanner s = new Scanner(System.in);
    String param = s.next();
    System.out.println("the param 1" + param);
    int value = s.nextInt();
    System.out.println("second param" + value);
    s.close();
  }
}
```

Gráfico 150. Lectura de datos por consola con Scanner

Todos los datos capturados por consola siempre son de tipo String, la clase Scanner nos proporciona los métodos nextXXX() para poder convertir el dato al tipo adecuado.

Todo el código de este ejemplo lo encontrareis en **Ejemplo27_Consola.zip**

ENTRADA Y SALIDA DE ARCHIVOS

La clase **File** perteneciente al paquete java.io nos permite manejar ficheros y directorios.

- File myFile = new File("myfile.txt"); Crea un fichero
- File myFile = new File("MyDocs", "myfile.txt"); Crea un fichero en el directorio especificado

Los métodos más utilizados de la clase **File** son los siguientes:

Métodos que proporcionan información del fichero:

- String **getName**(); Devuelve el nombre del fichero
- String **getPath**(); Devuelve la ruta relativa del fichero.
- String **getAbsolutePath**(); Devuelve la ruta absoluta del fichero;
- String **getParent**(); Devuelve el nombre del directorio donde se encuentra el fichero.
- long **lastModified**(); Devuelve la fecha de la última modificación.
- long **length**(); Devuelve la longitud del fichero.

Modificación de ficheros:

- boolean **renameTo**(File newName); Renombra el fichero con el nuevo nombre pasado como argumento.
- boolean **delete**(); Elimina el fichero.

Utilidades de directorio:

- boolean **mkdir**(); Crea un nuevo directorio y devuelve un valor booleano especificando si ha sido posible o no.
- String[] **list**(); Devuelve la lista de todo el contenido de un directorio. Equivalente a dir en MS-DOS.

Test de ficheros:

- boolean **exists**(); Devuelve un valor booleano especificando si existe el fichero o no.
- boolean **canWrite**(); Devuelve un valor booleano indicando si se puede escribir en el fichero o no.
- boolean **canRead**();Devuelve un valor booleano indicando si se puede leer el fichero o no.
- boolean **isFile**(); indica si es un fichero o no.
- boolean **isDirectory**(); indica si es un directorio o no.
- boolean **isHidden**(); indica si es un fichero oculto.

Ejemplo:

```
import java.io.*;
public class ReadFile {
  public static void main (String[] args) {
    // Create file
    File file = new File(args[0]);

    try {
      // Create a buffered reader
      // to read each line from a file.
      BufferedReader in
        = new BufferedReader(new FileReader(file));
      String s;

      // Read each line from the file and echo it to the screen.
      s = in.readLine();
      while ( s != null ) {
        System.out.println("Read: " + s);
        s = in.readLine();
      }
      // Close the buffered reader
      in.close();

    } catch (FileNotFoundException e1) {
      // If this file does not exist
      System.err.println("File not found: " + file);

    } catch (IOException e2) {
      // Catch any other IO exceptions.
      e2.printStackTrace();
    }
  }
}
```

Gráfico 151. Lectura de fichero

Ejemplo:

```
import java.io.*;

public class WriteFile {
  public static void main (String[] args) {
    // Create file
    File file = new File(args[0]);

    try {
      // Create a buffered reader to read each line from standard in.
      InputStreamReader isr
        = new InputStreamReader(System.in);
      BufferedReader in
        = new BufferedReader(isr);
      // Create a print writer on this file.
      PrintWriter out
        = new PrintWriter(new FileWriter(file));
      String s;

      System.out.print("Enter file text.  ");
      System.out.println("[Type ctrl-d to stop.]");

      // Read each input line and echo it to the screen.
      while ((s = in.readLine()) != null) {
        out.println(s);
      }

      // Close the buffered reader and the file print writer.
      in.close();
      out.close();

    } catch (IOException e) {
    // Catch any IO exceptions.
      e.printStackTrace();
    }
  }
}
```

Gráfico 152. Escritura en fichero

Todo el código de este ejemplo lo encontrareis en **Ejemplo28_File.zip**

CIERRE DE RECURSOS

El manejo automático de recursos tales como Files, Input/OutputStreams, etc., que los desarrolladores tienen que cerrar manualmente, pero que ahora Java 7 puede realizar de forma automática. Hasta ahora el proceso era el siguiente:

```
public void oldTry( ){
    try{
```

```
                fos = new FileOutputStream("movies.txt");
                dos = new DataOutputStream(fos);
                dos.writeUTF("Java 7 Block Buster");
            }catch (IOException e){
                e.printStackTrace( );
            }finally{
                try{
                    fos.close( );
                    dos.close( );
                }catch (IOException e){
                    //maneja la excepción
                }
            }
        }
```

Y así es como se haría con Java 7:

```
    public void newTry( ){
        try(FileOutputStream fos = new FileOutputStream("movies.txt");
            DataOutputStream dos = new DataOuputStream(fos)) {
                dos.writeUTF("Java 7 Block Buster");
        }catch (IOException e){
                //maneja la excepción
        }
    }
```

Eso sí, para que esto funcione, los recursos que queremos que se cierren automáticamente tienen que implementar la interfaz java.lang.AutoCloseable.



RECUERDA QUE...

- Podemos ejecutar un programa con una serie de argumentos iniciales que se recogerán en el método main.
- La entrada y salida permite manejar flujos a nivel de byte y a nivel de caracteres.
- Gracias a la clase File podemos manejar ficheros desde una aplicación Java.

10.- INTERFACE GRÁFICA CON SWING

QUE ES SWING

Swing es un API formado por un conjunto de componentes mejorados que sustituyen a los de las herramientas AWT originales más una serie de componentes avanzados. Estos componentes permiten crear interfaces de usuario con el tipo de funcionalidad que se espera de las aplicaciones modernas. Entre ellos se incluyen árboles de elementos, tablas, editores de texto avanzados y barras de herramientas flotantes.

Todo el api de swing queda recogido en el paquete javax.swing, mientras que el api awt está en el paquete java.awt.

Para poder desarrollar aplicaciones gráficas necesitamos de estos tres elementos:

- **Contenedores**; Se sitúan en la parte superior de las jerarquías de elementos de la interfaz gráfica. Todos los componentes de una GUI se agregan a estos contenedores. JFrame, JDialog, JWindow y JApplet son los contenedores de nivel superior.
- **Administradores de diseño**; Son los responsables de determinar la posición de los componentes en un contenedor. BorderLayout, FlowLayout y GridLayout son algunos ejemplos de estos administradores. Existen otros más complejos y sofisticados que proporcionan mayor control sobre la interfaz.
- **Componentes**; Todos los componentes de la interfaz gráfica se derivan de la clase JComponent, por ejemplo, JComboBox, JAbstractButton y JTextComponent.

En la siguiente imagen vemos un ejemplo de interface gráfica donde observamos cada uno de los elementos descritos.

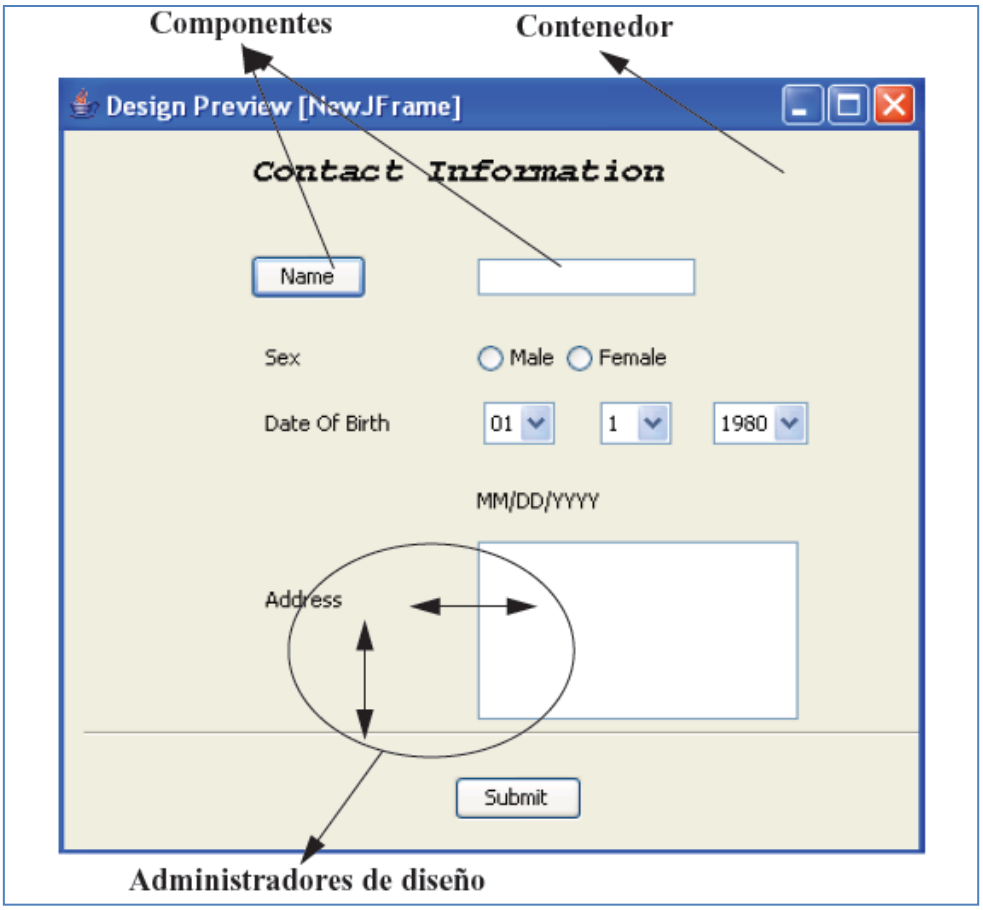


Gráfico 153. Elementos de una interface gráfica

CONTENEDORES SWING

Los contenedores Swing pueden clasificarse en dos grandes categorías:

- Contenedores de nivel superior
- Contenedores genéricos

CONTENEDORES DE NIVEL SUPERIOR

Son los que se sitúan en la parte superior de la jerarquía de contenedores de Swing. Existen tres contenedores de nivel superior: JFrame, JWindow y JDialog. Existe también una clase especial, JApplet, que, aunque no es estrictamente un contenedor de nivel superior, merece una mención porque debería utilizarse como elemento de nivel superior de cualquier applet que utilice componentes de Swing.

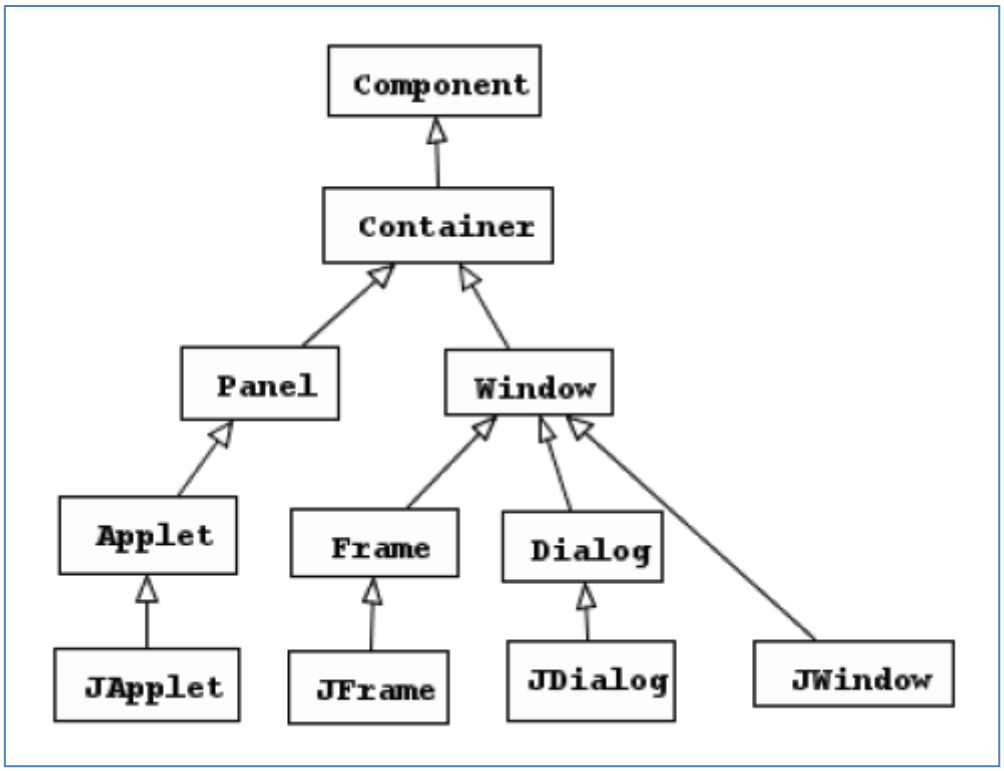


Gráfico 154. Jerarquía de los contenedores de nivel superior

Las clases JApplet, JFrame, JDialog y JWindow derivan respectivamente de las clases Applet, Frame, Dialog y Window. Es importante tenerlo en cuenta porque el resto de contenedores y componentes de Swing derivan de clases JComponent.

JApplet; Un applet es una aplicación Java que se incrusta en una página Web. Este contenedor sirve para desarrollar Applets. Todos los elementos que forman parte de la interfaz gráfica de un Applet están contenidos dentro de una clase que extiende a la clase JApplet, de ahí que en última instancia herede propiedades y métodos de la clase Container.



Gráfico 155. Representación de un contenedor JApplet

JFrame; Un contenedor de este tipo se muestra como una ventana de Windows con bordes, barra de títulos, botones para minimizar, maximizar y cerrar la ventana. También admite barra de menús.

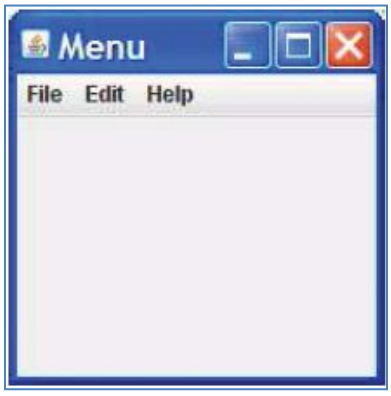


Gráfico 156. Representación de un contenedor JFrame

JDialog; Sirve para crear ventanas de diálogo. Este tipo de ventanas se integran en las aplicaciones y Applets y se utilizan para mostrar al usuario mensajes. Los diálogos más conocidos son aquellos que indican al usuario algún tipo de alerta.

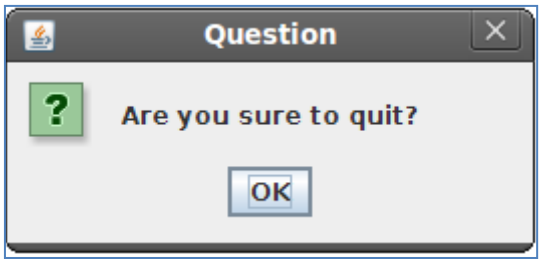


Gráfico 157. Representación de un contenedor JDialog

JWindow; Un contenedor de este tipo es muy similar a un JFrame pero este no tiene bordes y tampoco soporta barra de menús.



Gráfico 158. Representación de un contenedor JWindow

CONTENEDORES GENÉRICOS

Los contenedores genéricos se emplean para encapsular partes de una aplicación o un Applet. Se podrían ver como subcontenedores o un contenedor embebido en otro contenedor de nivel superior.

Los contenedores genéricos heredan de javax.swing.JComponent. Veamos algunos de ellos:

JPanel; Éste es sin duda alguna uno de los componentes más empleados en la creación de interfaces gráficas. Un JPanel describe un área que agrupa a otros componentes y se integra dentro de una aplicación de la misma forma que lo hacen otros controles básicos como por ejemplo un botón (JButton) o una etiqueta (JLabel).

Los componentes JPanel son muy fáciles de manejar y permiten organizar la interfaz no sólo desde un punto de vista visual sino además para agrupar funcionalidades. Cuando se crean interfaces gráficas complejas una de las prácticas más comunes consiste en desarrollar clases específicas que extienden a la clase JPanel para aglutinar partes de la aplicación que comparten un espacio común y una función concreta dentro de la aplicación o el Applet.

El componente JPanel se emplea para agrupar dos botones.

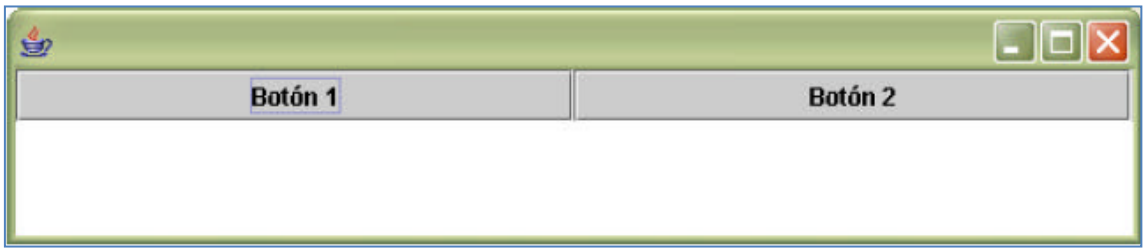


Gráfico 159. Representación de un contenedor JPanel

JScrollPane; Prácticamente cualquier elemento dentro de una interfaz gráfica puede tener barras de desplazamiento de forma que el usuario pueda ver aquellas partes que quedan ocultas debido a la limitación de espacio.

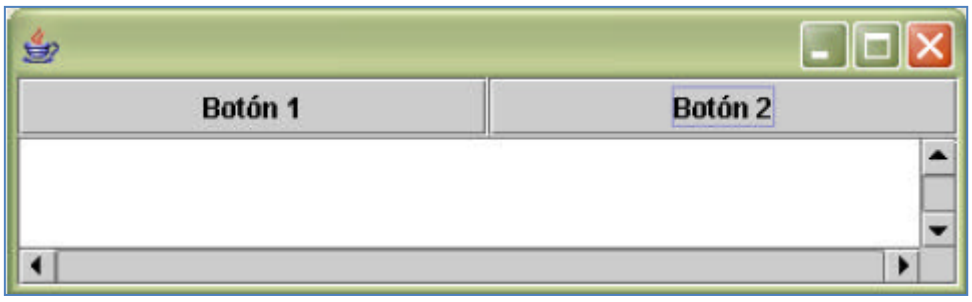


Gráfico 160. Representación de un contenedor JScrollPane

JSplitPane; Este tipo de contenedor permite agrupar dos componentes uno al lado del otro, o uno encima del otro. Entre ambos se muestra una barra de forma que cuando el usuario hace clic y desplaza la barra, las áreas de los dos componentes mostrados cambian de tamaño.

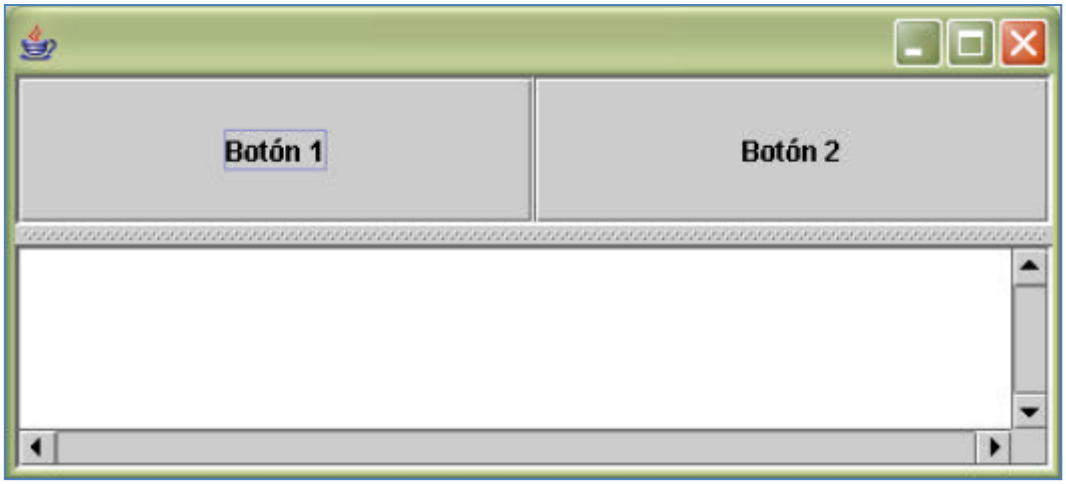


Gráfico 161. Representación de un contenedor JSplitPane

JTabbedPane; Este contenedor se emplea para distribuir otros componentes en pestañas.

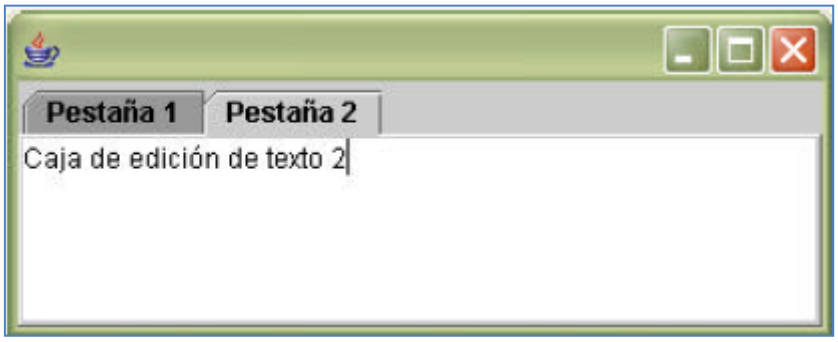


Gráfico 162. Representación de un contenedor JTabbedPane

JToolBar; La clase JToolBar se emplea para crear la barra de herramientas. Normalmente éstas están formadas por botones, listas y otro tipo de controles básicos, aunque como contenedor un componente JToolBar puede agrupar cualquier tipo de componente.

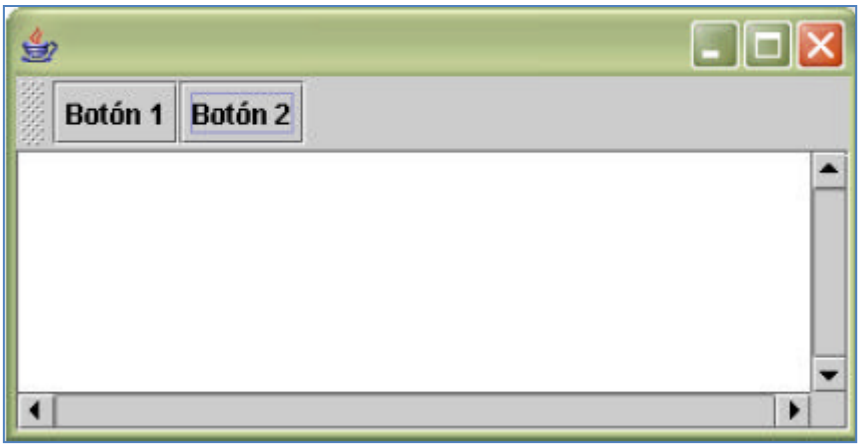


Gráfico 163. Representación de un contenedor JToolBar

ADMINISTRADORES DE DISEÑO

El administrador de diseño determina el tamaño y la posición de los componentes dentro de un contenedor. Como alternativa al uso de estos administradores, es posible asignar una posición absoluta mediante coordenadas de píxeles. La posición absoluta se obtiene definiendo la propiedad de diseño de un contenedor con el valor null. El posicionamiento absoluto no puede trasladarse de una plataforma a otra. Aspectos como el tamaño de los tipos de letra y las pantallas hacen que una posición que sea correcta según sus coordenadas, sea potencialmente inservible en otras plataformas.

Los distintos administradores de diseño son los siguientes:

- BorderLayout
- FlowLayout
- BoxLayout
- CardLayout
- GridLayout
- GridBagLayout
- GroupLayout

BORDERLAYOUT

BorderLayout organiza los componentes en cinco zonas diferentes: CENTER, NORTH, SOUTH, EAST y WEST (centro, norte, sur, este y oeste, respectivamente). Este administrador de diseño limita a uno el número de componentes que pueden agregarse a cada zona.

Es necesario especificar la posición del componente. Si no se especifica, se coloca automáticamente en el centro. Cualquier espacio sobrante lo usa el componente de la zona central.

BorderLayout es el diseño predeterminado para los objetos JFrame, JDialog y JApplet.

```
public class EjemploBorderLayout {

    private JFrame f;
    private JButton bn, bs, bw, be, bc;

    public EjemploBorderLayout() {
        f = new JFrame("Border Layout");
        bn = new JButton("Button 1");
        bc = new JButton("Button 2");
        bw = new JButton("Button 3");
        bs = new JButton("Button 4");
        be = new JButton("Button 5");
    }

    public void launchFrame() {
        f.add(bn, BorderLayout.NORTH);
        f.add(bs, BorderLayout.SOUTH);
        f.add(bw, BorderLayout.WEST);
        f.add(be, BorderLayout.EAST);
        f.add(bc, BorderLayout.CENTER);
        f.setSize(400, 200);
        f.setVisible(true);
    }

    public static void main(String args[]) {
        EjemploBorderLayout guiWindow2 = new EjemploBorderLayout();
        guiWindow2.launchFrame();
    }
}
```

Gráfico 164. Ejemplo BorderLayout

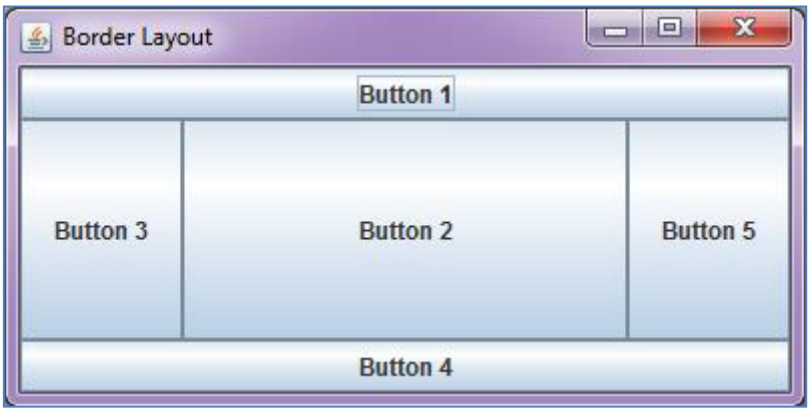


Gráfico 165. Resultado tras ejecutar el ejemplo BorderLayout

FLOWLAYOUT

FlowLayout coloca los componentes en una fila. En su forma predeterminada, coloca los componentes de izquierda a derecha (LEFT_TO_RIGHT). Esta orientación puede cambiarse con la propiedad ComponentOrientation para que sea RIGHT_TO_LEFT, es decir, de derecha a izquierda. Es posible especificar el espacio vertical y horizontal que debe haber entre los componentes. Si no se especifica, se utiliza una distancia predeterminada de cinco unidades en ambas direcciones.

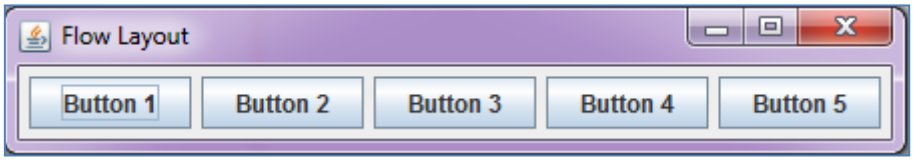


Gráfico 166. Resultado tras ejecutar el ejemplo FlowLayout

A continuación vemos un ejemplo en el cual se disponen los componentes en la misma fila.

```
public class EjemploFlowLayout {

    private JFrame f;
    private JButton b1, b2, b3, b4, b5;

    public EjemploFlowLayout() {
        f = new JFrame("Flow Layout");
        b1 = new JButton("Button 1");
        b2 = new JButton("Button 2");
        b3 = new JButton("Button 3");
        b4 = new JButton("Button 4");
        b5 = new JButton("Button 5");
    }

    public void launchFrame() {
        f.setLayout(new FlowLayout());
        f.add(b1);
        f.add(b2);
        f.add(b3);
        f.add(b4);
        f.add(b5);
        f.pack();
        f.setVisible(true);
    }

    public static void main(String args[]) {
        EjemploFlowLayout guiWindow2 = new EjemploFlowLayout();
        guiWindow2.launchFrame();
    }
}
```

Gráfico 167. Ejemplo FlowLayout

BOXLAYOUT

BoxLayout organiza los componentes en disposición vertical u horizontal.

El constructor de BoxLayout utiliza un parámetro llamado “axis” (eje) que debe utilizarse para indicar la dirección en la que se alinearán los componentes. Este parámetro puede adoptar los siguientes valores:

- X_AXIS: los componentes se organizan horizontalmente, de izquierda a derecha.
- Y_AXIS: los componentes se organizan verticalmente, de arriba abajo.
- LINE_AXIS: los componentes se colocan en la misma dirección que las palabras de una línea.
- PAGE_AXIS: los componentes se colocan en la misma dirección que las líneas de una página.

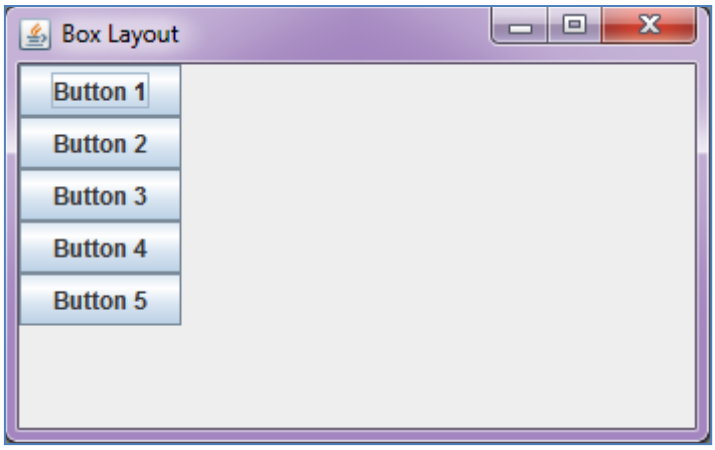


Gráfico 168. Resultado tras ejecutar el ejemplo BoxLayout

```
public class EjemploBoxLayout {

    private JFrame f;
    private JButton b1, b2, b3, b4, b5;

    public EjemploBoxLayout() {
        f = new JFrame("Box Layout");
        b1 = new JButton("Button 1");
        b2 = new JButton("Button 2");
        b3 = new JButton("Button 3");
        b4 = new JButton("Button 4");
        b5 = new JButton("Button 5");
    }

    public void launchFrame() {
        // añade los botones en la misma fila
        //f.setLayout(new BoxLayout(f.getContentPane(), BoxLayout.X_AXIS));

        // añade los botones en la misma columna
        f.setLayout(new BoxLayout(f.getContentPane(), BoxLayout.Y_AXIS));
        f.add(b1);
        f.add(b2);
        f.add(b3);
        f.add(b4);
        f.add(b5);
        f.pack();
        f.setVisible(true);
    }


    public static void main(String args[]) {
        EjemploBoxLayout guiWindow2 = new EjemploBoxLayout();
        guiWindow2.launchFrame();
    }
}
```

Gráfico 169. Ejemplo BoxLayout

CARDLAYOUT

CardLayout organiza los componentes como una baraja de cartas. Cada carta acepta un único componente para su representación en pantalla.

Este único componente se convierte en un contenedor, con lo que es posible representar múltiples componentes dentro de una carta. Sólo puede verse una carta cada vez.

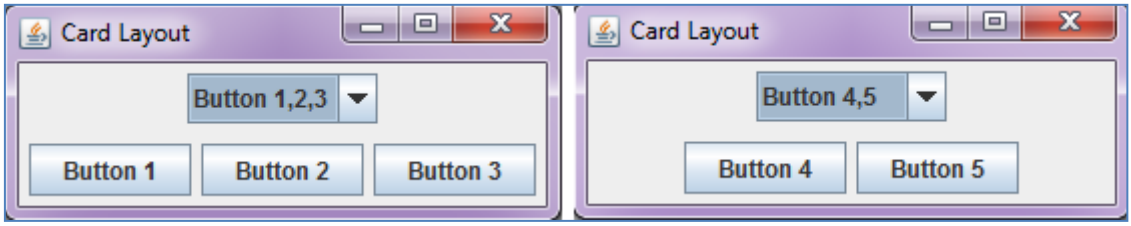


Gráfico 170. Resultado tras ejecutar el ejemplo CardLayout

Los botones Button1, Button2 y Button3 se colocan en Carta1 y los botones Button4 y Button5 se colocan en Carta2. Se ha utilizado un cuadro combinado para elegir la carta de debe aparecer en la pantalla.

A continuación vemos un ejemplo de este administrador de contenido:

En primer lugar vemos la declaración de la clase y las propiedades que utilizaremos

```
public class EjemploCardLayout implements ActionListener{

    private JFrame f;
    private JButton b1, b2, b3, b4, b5;
    public CardLayout cardLayout;
    public JPanel panelSuperior, panelInferior, panelInf1, panelInf2;
    public JComboBox combo;
    public String[] vectorCadena = {"Button 1,2,3", "Button 4,5"};
```

Gráfico 171. Declaración clase EjemploCardLayout

En la siguiente imagen podemos ver los métodos que utilizamos para construir los diferentes paneles. El panel superior donde a través de un combo se elije que carta ver, un panel inferior que será el encargado de mostrar la carta elegida y otros dos paneles inferiores que contendrán cada una de las cartas.

```
public void construyePanelSuperior() {
    panelSuperior = new JPanel();
    combo = new JComboBox(vectorCadena);
    panelSuperior.setLayout(new FlowLayout());
    panelSuperior.add(combo);
    panelSuperior.getPreferredSize();
}

public void construyePanelInferior() {
    panelInferior = new JPanel();
    cardLayout = new CardLayout();
    panelInferior.setLayout(cardLayout);
    /*Al agregar necesitamos 2 argumentos,
     el objeto a agregar y un nombre referencial */
    panelInferior.add(panelInf1, "Button 1,2,3");
    panelInferior.add(panelInf2, "Button 4,5");
}

public void construyePanelInf1() {
    panelInf1 = new JPanel(new FlowLayout());
    panelInf1.add(b1);
    panelInf1.add(b2);
    panelInf1.add(b3);
}

public void construyePanelInf2() {
    panelInf2 = new JPanel(new FlowLayout());
    panelInf2.add(b4);
    panelInf2.add(b5);
}
```

Gráfico 172. Fragmento de la clase EjemploCardLayout donde construimos los paneles

En el constructor construimos el JFrame y los botones:

```
public EjemploCardLayout() {
    f = new JFrame("Card Layout");
    b1 = new JButton("Button 1");
    b2 = new JButton("Button 2");
    b3 = new JButton("Button 3");
    b4 = new JButton("Button 4");
    b5 = new JButton("Button 5");
}
```

Gráfico 173. Constructor de la clase EjemploCardLayout

Como hemos visto en la declaración de la clase, esta implementa la interface ActionListener para poder manejar eventos.

El método que vemos a continuación pertenece a dicha interface y lo utilizamos para detectar el evento provocado por el usuario al elegir una carta u otra.

En función de lo que se ha seleccionado en el combo se mostrará un panel inferior u otro.

```
public void actionPerformed(ActionEvent evento) {
    if (evento.getSource() == combo) {
        if (combo.getSelectedIndex() == 0) {
            cardLayout.show(panelInferior, "Button 1,2,3");
        }
        if (combo.getSelectedIndex() == 1) {
            cardLayout.show(panelInferior, "Button 4,5");
        }
    }
}
```

Gráfico 174. Manejador de evento para la clase EjemploCardLayout

Para terminar con nuestro ejemplo vemos el método launchFrame que será el encargado de construir toda la interface gráfica y el método main a través del cual se ejecutará.

```
public void launchFrame() {
    construyePanelSuperior();
    construyePanelInf1();
    construyePanelInf2();
    construyePanelInferior();

    f.setLayout(new BoxLayout(f.getContentPane(), BoxLayout.Y_AXIS));
    f.add(panelSuperior);
    f.add(panelInferior);
    f.getPreferredSize();
    f.pack();
    f.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    f.setVisible(true);

    //creamos el objeto controlador de eventos
    combo.addActionListener(this);
}

public static void main(String args[]) {
    EjemploCardLayout guiWindow2 = new EjemploCardLayout();
    guiWindow2.launchFrame();
}
```

Gráfico 175. Lanzamos GUI para EjemploCardLayout

GRIDLAYOUT

GridLayout coloca los componentes en filas y columnas. Cada componente ocupa la misma cantidad de espacio en el contenedor.

Al crear el diseño de la cuadrícula, es preciso especificar el número de filas y columnas. Si no se especifica, el administrador crea una fila y una columna. También se puede indicar la distancia horizontal y vertical entre los componentes.

```
public class EjemploGridLayout {

    private JFrame f;
    private JButton b1, b2, b3, b4, b5;

    public EjemploGridLayout() {
        f = new JFrame("Grid Example");
        b1 = new JButton("Button 1");
        b2 = new JButton("Button 2");
        b3 = new JButton("Button 3");
        b4 = new JButton("Button 4");
        b5 = new JButton("Button 5");
    }

    public void launchFrame() {
        f.setLayout(new GridLayout(3, 2));
        f.add(b1);
        f.add(b2);
        f.add(b3);
        f.add(b4);
        f.add(b5);
        f.pack();
        f.setVisible(true);
    }

    public static void main(String args[]) {
        EjemploGridLayout grid = new EjemploGridLayout();
        grid.launchFrame();
    }
}
```

Gráfico 176. Ejemplo GridLayout

El resultado tras la ejecución será el siguiente:

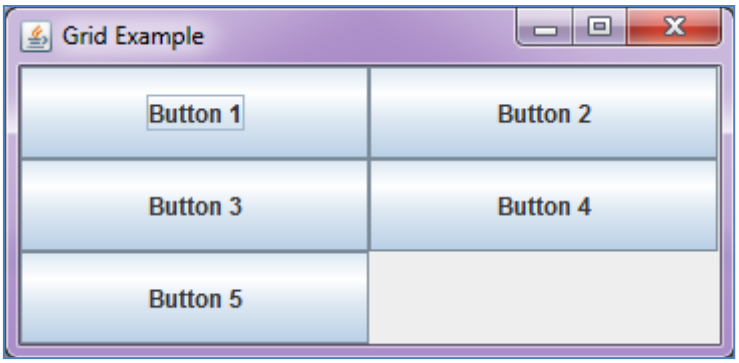


Gráfico 177. Resultado tras ejecutar el ejemplo CardLayout

GRIDBAGLAYOUT

GridBagLayout organiza los componentes en filas y columnas, de forma similar a la del diseñador de cuadrículas, pero, para mayor flexibilidad, incluye opciones para cambiar el tamaño y especificar la posición de los componentes. Es un diseño utilizado para crear interfaces complejas.

Las restricciones aplicadas a los componentes se especifican mediante la clase GridBagConstraints. Algunas restricciones de esta clase son gridwidth, gridheight, gridx, gridy, weightx y weighty.

En la siguiente figura vemos un ejemplo de este administrador de diseño.

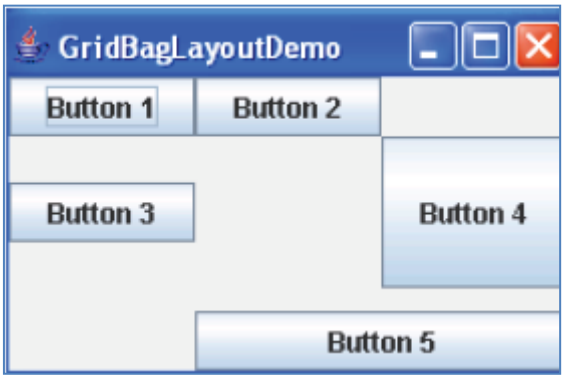


Gráfico 178. Ejemplo GridBagLayout

GROUPLAYOUT

A los administradores de diseño ya citados se ha agregado GroupLayout en la versión 6 de Java. Se ha añadido para mayor comodidad de los desarrolladores de herramientas. Es el diseñador utilizado en la herramienta de generación de interfaces gráficas del IDE NetBeans.

Este administrador presenta las siguientes características:

- Trabaja con las dimensiones horizontales y verticales de forma independiente.

- Coloca los componentes secuencialmente, uno a continuación de otro tomando como referencia el anterior.
- Define grupos de capas.
- Cada grupo puede definir los componentes secuencialmente o en paralelo.
- Cada grupo puede contener a su vez otros grupos.

Todo el código de este ejemplo lo encontrareis en **Ejemplo29_Layouts.zip**

COMPONENTES SWING

Las interfaces basadas en Swing utilizan dos tipos de clases: clases de interfaz gráfica y clases de soporte. Las clases de interfaz gráfica son de carácter visual, descienden de JComponent y se denominan clases J. Las clases de soporte proporcionan servicios y realizan funciones esenciales para las clases de interfaz, pero no generan ninguna salida gráfica.

Los componentes Swing proporcionan, sobre todo, elementos para manejar texto, botones, etiquetas, listas, paneles, recuadros, barras de desplazamiento, paneles de desplazamiento, menús, tablas y árboles.

En general, estos componentes pueden clasificarse en los grupos siguientes:

- Botones
- Componentes de texto
- Componentes de visualización de información no editable
- Menús
- Componentes de pantalla con formato
- Otros controles básicos

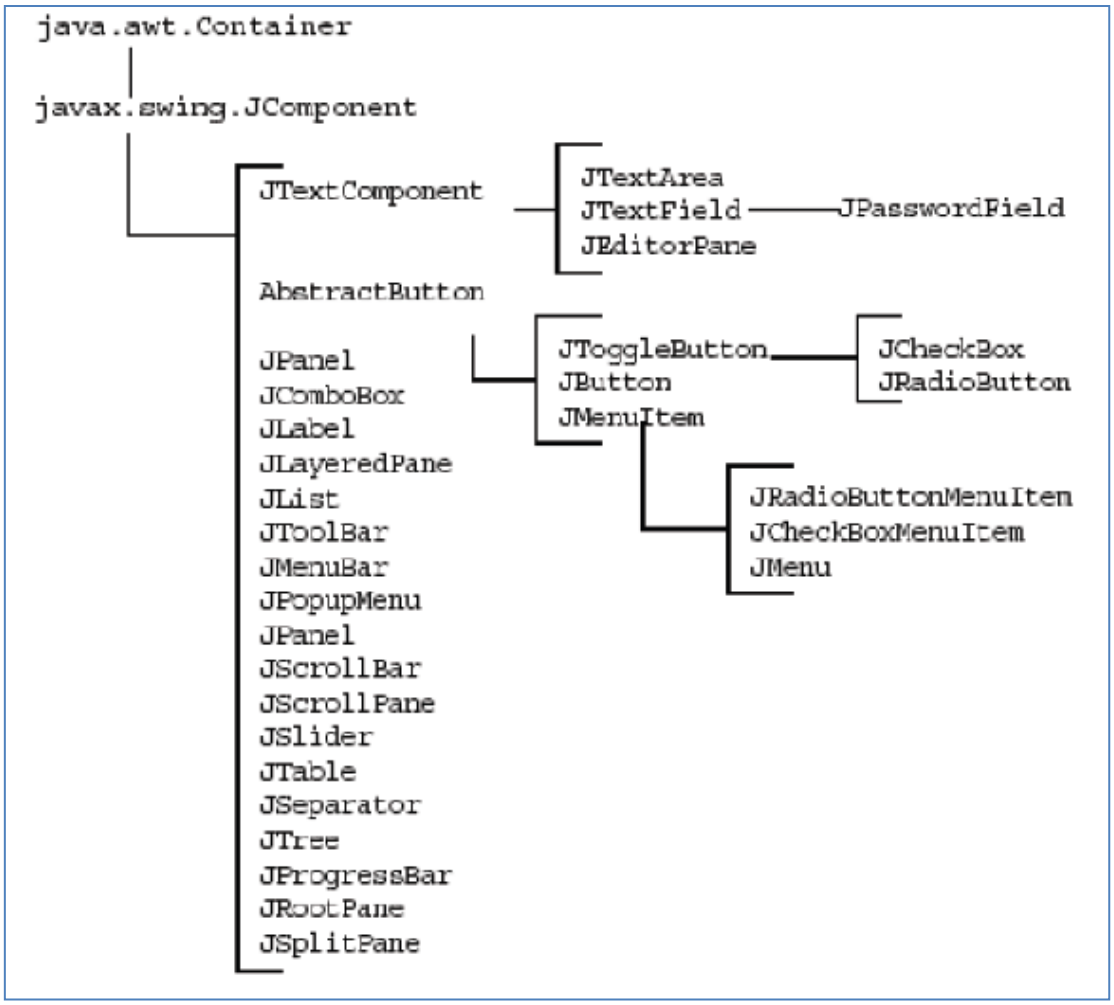


Gráfico 179. Jerarquía de componentes swing

Todos los componentes Swing poseen algunas propiedades en común porque todas ellas derivan de la clase JComponent.

Algunas propiedades, como las de tamaño preferido, sirven de información al administrador de diseño.

Aunque un componente puede proporcionar datos de utilidad al administrador de diseño, ésta puede hacer caso omiso de tal información y utilizar otra para representar el componente.

Propiedad	Métodos
Borde	`Border getBorder()` `void setBorder(Border b)`
Color de fondo y de primer plano	`void setBackground(Color bg)` `void setForeground(Color bg)`
Tipo de letra	`void setFont(Font f)`
Opacidad	`void setOpaque(boolean isOpaque)`
Tamaño máximo y mínimo	`void setMaximumSize(Dimension d)` `void setMinimumSize(Dimension d)`
Alineación	`void setAlignmentX(float ax)` `void setAlignmentY(float ay)`
Tamaño preferido	`void setPreferredSize(Dimension ps)`

Gráfico 180. Propiedades comunes de los componentes

Además de las propiedades comunes cada componente tendrá las suyas propias, en la siguiente imagen vemos las propiedades específicas del componente JComboBox.

Propiedades	Métodos
Cantidad máxima de filas	`void setMaximumRowCount(int cantidad)`
Modelo	`void setModel(ComboBoxModel cbm)`
Índice seleccionado	`int getSelectedIndex()`
Elemento seleccionado	`Object getSelectedItem()`
Cantidad de elementos	`int getItemCount()`
Mecanismo de representación (renderer)	`void setRenderer(ListCellRenderer ar)`
Editable	`void setEditable(boolean indicador)`

Gráfico 181. Propiedades específicas del componente JComboBox

CREAR INTERFACES GRAFICAS DE USUARIO

Las interfaces gráficas basadas en Java pueden crearse utilizando una de las siguientes técnicas:

- **Construcción programática**; Esta técnica consiste en utilizar código de programación para crear la interfaz. Resulta útil para aprender los diferentes

aspectos de la creación de interfaces gráficas. Por otra parte, es muy laboriosa si debe usarse en entornos de producción.

- **Construcción con una herramienta de generación de interfaces gráficas**; Esta técnica se basa en el uso de una herramienta de generación de interfaces. Los desarrolladores de interfaces gráficas utilizan un enfoque visual para desplazar y colocar los contenedores y componentes en el área de trabajo. Con cada paso, la herramienta genera automáticamente las clases Java necesarias para reproducir la interfaz.

CONSTRUCCIÓN PROGRAMÁTICA

En este ejemplo se describe la creación de una interfaz gráfica sencilla que muestra en la pantalla el saludo Hello World.

Se crea un contenedor JFrame con título HelloWorldSwing. Posteriormente se agrega un componente JLabel con el texto Hello World.

Se establece como dimensiones 300 x 200 pixeles y se hace visible.

```
public class Main {

    private static void createAndShowGUI() {
        JFrame frame = new JFrame("HelloWorldSwing");
        // Define la ventana.
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JLabel label = new JLabel("Hello World");
        // Agrega la etiqueta
        frame.add(label);
        frame.setSize(300, 200);
        // Muestra la ventana en la pantalla
        frame.setVisible(true);
    }

    public static void main(String[] args) {
        javax.swing.SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                createAndShowGUI();
            }
        });
    }
}
```

Gráfico 182. Ejemplo HelloWorld con construcción programática.

Todo el código de este ejemplo lo encontrareis en **Ejemplo30_ HelloWorldSwing.zip**

CONSTRUCCIÓN CON UNA HERRAMIENTA DE GENERACIÓN DE INTERFACES GRÁFICAS

Todos los entornos de desarrollo incluidos Eclipse y Netbeans tienen herramientas para la generación de interfaces gráficas.

A través de dichas herramientas simplemente arrastrando componentes desde la paleta y configurándolos se generará automáticamente todo el código necesario.

En el ejemplo siguiente creamos un proyecto Java y creamos un nuevo JFrame con el nombre Ejemplo.

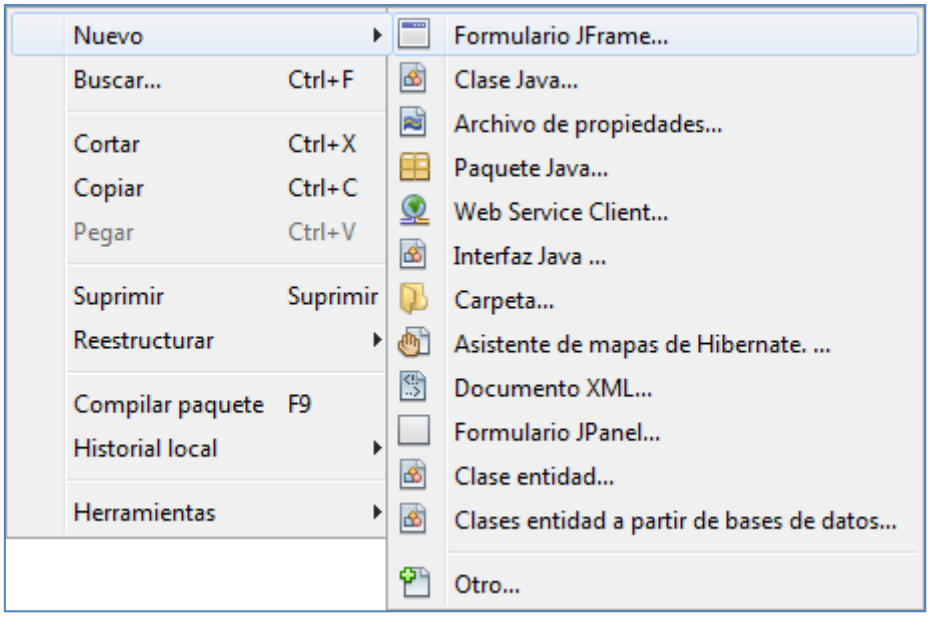


Gráfico 183. Crear un nuevo JFrame

La apariencia de nuestro IDE cambia para ofrecer dos vistas:

- **Diseño**; desde aquí podemos arrastrar los componentes swing al lienzo y configurarlos a través del panel de propiedades.
- **Fuente**; En esta vista podremos observar todo el código generado.

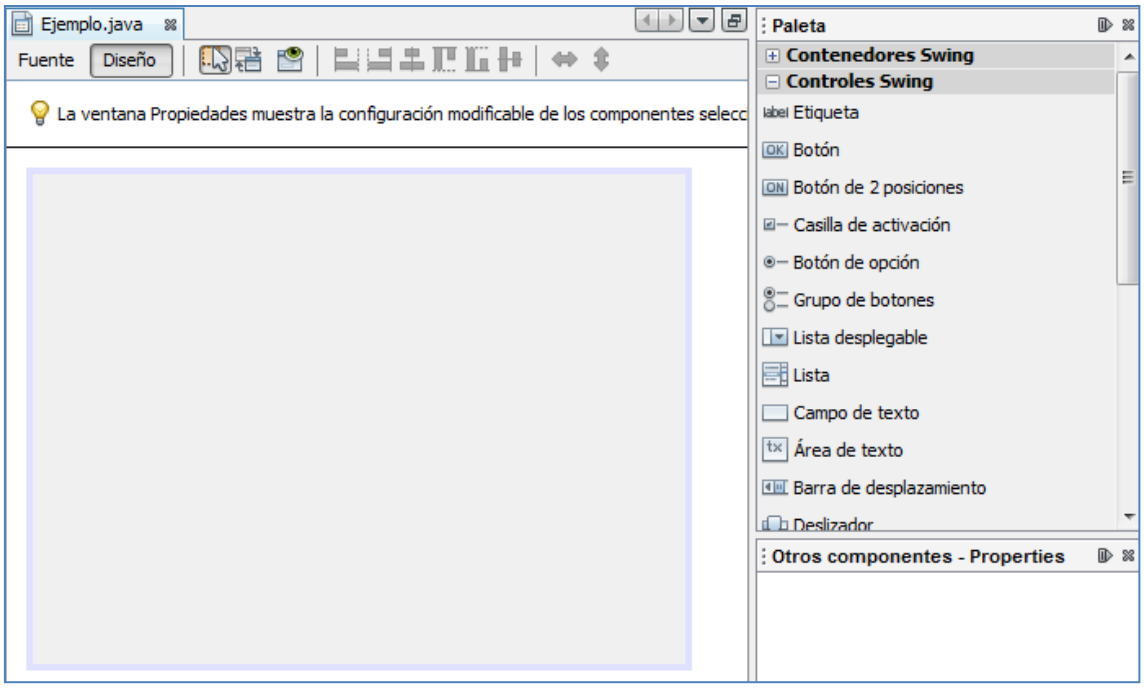


Gráfico 184. Diseño de un JFrame

EVENTOS

Cuando el usuario realiza una acción en la interfaz gráfica (pulsar el botón del ratón o presionar una tecla), se genera un evento. Los eventos son objetos que describen lo que ha ocurrido. Existen numerosas clases de eventos diferentes para describir las distintas categorías de acciones de los usuarios.

En la siguiente imagen se muestra una vista abstracta del modelo de delegación de eventos. Cuando un usuario hace clic en un botón de la interfaz gráfica, JVM crea un objeto evento que se envía al controlador de eventos (también conocido como manejador de eventos) correspondiente a ese botón mediante una llamada al método actionPerformed.

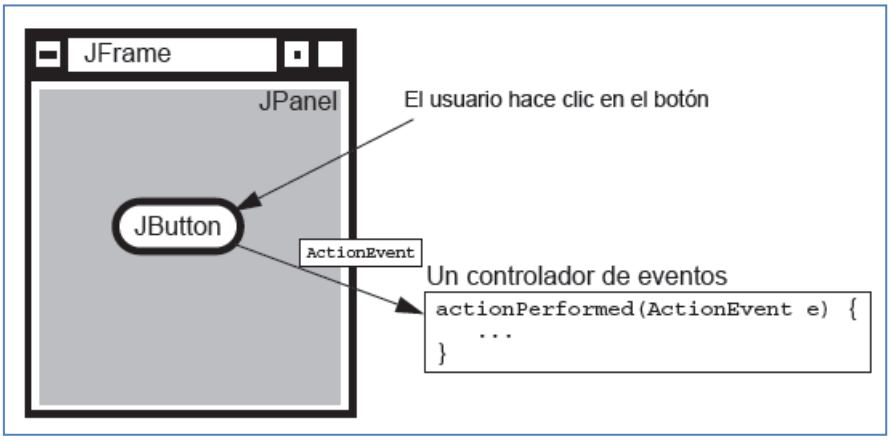


Gráfico 185. El usuario genera un evento al pulsar el botón

La fuente de un evento es aquella que genera el evento. Por ejemplo, pulsar un botón del ratón sobre un componente JButton genera una instancia de ActionEvent que tiene ese botón como fuente. La instancia de ActionEvent es un objeto que contiene información sobre el evento que acaba de producirse.

MODELO DE DELEGACIÓN

El modelo de delegación de eventos se implementó con la versión 1.1 del kit de desarrollo de Java (JDK). Con este modelo, los eventos se envían al componente desde el que se ha originado el evento, pero depende de cada componente su propagación a una o más clases registradas denominadas receptores de eventos (listeners). Estos receptores contienen controladores que reciben el evento y lo procesan. De esta forma, el controlador de eventos puede estar situado en un objeto independiente del componente. Los receptores de eventos son clases que implementan la interfaz EventListener.

En el siguiente ejemplo se muestra la forma de crear un marco (JFrame) sencillo que sólo contiene un botón (JButton).

```
public class TestBoton {
    private JFrame f;
    private JButton b;

    public TestBoton() {
        f = new JFrame("Test");
        b = new JButton("Pulsar");
        b.setActionCommand("ButtonPressed");
    }

    public void launchFrame() {
        b.addActionListener(new ButtonHandler());
        f.add(b, BorderLayout.CENTER);
        f.pack();
        f.setVisible(true);
    }

    public static void main(String args[]) {
        TestBoton guiApp = new TestBoton();
        guiApp.launchFrame();
    }
}
```

Gráfico 186. Clase donde se genera el evento

La clase ButtonHandler mostrada a continuación es la clase del controlador en el que se delega el evento.

```
public class ButtonHandler implements ActionListener {

    public void actionPerformed(ActionEvent e) {
        System.out.println("Se ha realizado una acción");
        System.out.println("El comando del botón es: "
                           + e.getActionCommand());
    }
}
```

Gráfico 187. Clase donde se maneja el evento

El resultado de la ejecución es el siguiente:

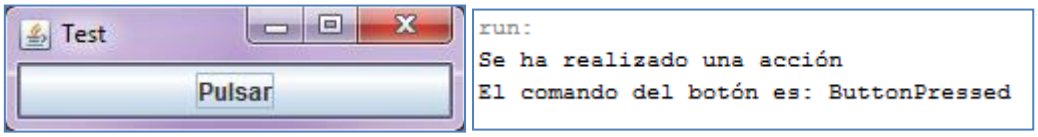


Gráfico 188. Resultado de la ejecución

Todo el código de este ejemplo lo encontrareis en **Ejemplo31_ DelegacionEventos.zip**

CATEGORÍAS DE EVENTOS

El mecanismo descrito para recibir eventos de los componentes se ha ilustrado en el contexto de un único tipo de evento. Muchas de las clases de eventos residen en el paquete java.awt.event, pero existen otras en otros lugares del API.

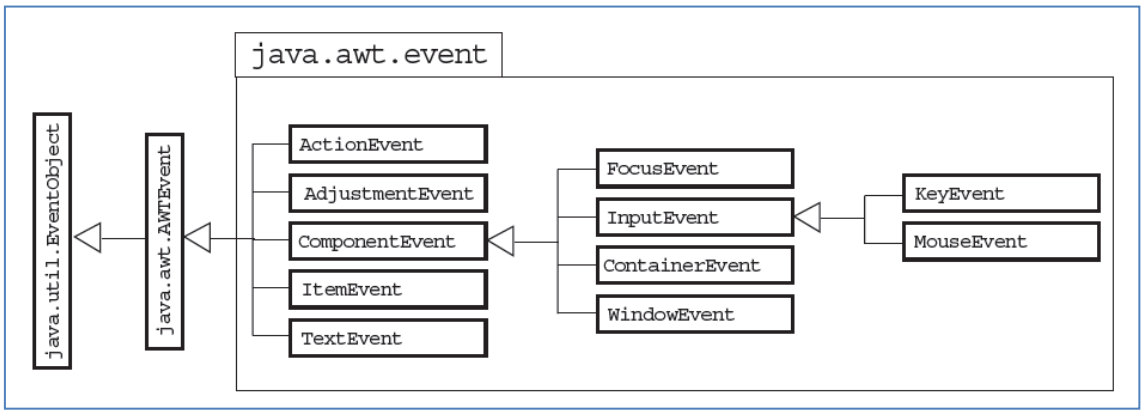


Gráfico 189. Jerarquía de clases de eventos de la interfaz gráfica

Por cada categoría de eventos, la clase de los objetos que quiera recibir los eventos debe implementar una interfaz. A su vez, esa interfaz exige la definición de uno o varios métodos. La llamada a estos métodos se realiza cuando se producen determinados eventos. A continuación vemos estas categorías e interfaces.

Categoría	Nombre de la interfaz	Métodos
Acción	`ActionListener`	`actionPerformed(ActionEvent)`
Elemento	`ItemListener`	`itemStateChanged(ItemEvent)`

Ratón	`MouseListener`	`mousePressed(MouseEvent)`
		`mouseReleased(MouseEvent)`
		`mouseEntered(MouseEvent)`
		`mouseExited(MouseEvent)`
		`mouseClicked(MouseEvent)`

Movimiento del ratón	`MouseMotionListener`	`mouseDragged(MouseEvent)` `mouseMoved(MouseEvent)`

Tecla	`KeyListener`	`keyPressed(KeyEvent)`
		`keyReleased(KeyEvent)`
		`keyTyped(KeyEvent)`

Activación	`FocusListener`	`focusGained(FocusEvent)` `focusLost(FocusEvent)`
Ajuste	`AdjustmentListener`	`adjustmentValueChanged (AdjustmentEvent)`

Componente	`ComponentListener`	`componentMoved(ComponentEvent)`
		`componentHidden(ComponentEvent)`
		`componentResized(ComponentEvent)`
		`componentShown(ComponentEvent)`

Ventana	`WindowListener`	`windowClosing(WindowEvent)`
		`windowOpened(WindowEvent)`
		`windowIconified(WindowEvent)`
		`windowDeiconified(WindowEvent)`
		`windowClosed(WindowEvent)`
		`windowActivated(WindowEvent)`
		`windowDeactivated(WindowEvent)`

Contenedor	`ContainerListener`	`componentAdded(ContainerEvent)` `componentRemoved(ContainerEvent)`
Estado de ventana	`WindowStateListener`	`windowStateChanged(WindowEvent e)`

Gráfico 190. Categorías de eventos, métodos e interfaces

Categoría	Nombre de la interfaz	Métodos
Activación de ventana	`WindowFocusListener`	`windowGainedFocus(WindowEvent e)` `windowLostFocus(WindowEvent e)`
Rueda del ratón	`MouseWheelListener`	`mouseWheelMoved(MouseWheelEvent e)`
Métodos de entrada	`InputMethodListener`	`caretPositionChanged (InputMethodEvent e)` `inputMethodTextChnaged (InputMethodEvent e)`
Jerarquía	`HierarchyListener`	`hierarchyChanged(HierarchyEvent e)`
Límites de la jerarquía	`HierarchyBoundsListener`	`ancestorMoved(HierarchyEvent e)` `ancestorResized(HierarchyEvent e)`
AWT	`AWTEventListener`	`eventDispatched(AWTEvent e)`
Texto	`TextListener`	`textValueChanged(TextEvent)`

Gráfico 191. Categorías de eventos, métodos e interfaces (Continuación)

CONTROL DE EVENTOS MEDIANTE CLASES INTERNAS

En el siguiente ejemplo se muestra la forma de crear controladores de eventos como clases internas. El uso de clases internas para manejar eventos permite acceder a los datos privados de la clase externa.

```
public class MouseClickHandler extends MouseAdapter {

    // Sólo necesitamos el método mouseClicked, así que usamos
    // un adaptador para no tener que escribir
    // todos los métodos de control de eventos

    @Override
    public void mouseClicked(MouseEvent e) {
        // Realiza acciones al pulsar el botón ....
    }
}
```

Gráfico 192. Clase que captura el evento

```
public class TestInner {

    private JFrame f;
    private JTextField tf;

    public TestInner() {
        f = new JFrame("Ejemplo de clases internas");
        tf = new JTextField(30);
    }

    class MyMouseMotionListener extends MouseMotionAdapter {
        public void mouseDragged(MouseEvent e) {
            String s = "Arrastre del ratón: X = " + e.getX()
                    + " Y = " + e.getY();
            tf.setText(s);
        }
    }

    public void launchFrame() {
        JLabel label = new JLabel("Hacer clic y arrastrar el ratón");
        f.add(label, BorderLayout.NORTH);
        f.add(tf, BorderLayout.SOUTH);
        // Agrega un Listener que utiliza una clase interna
        f.addMouseMotionListener(new MyMouseMotionListener());
        f.addMouseListener(new MouseClickHandler());
        // Define el tamaño del marco y lo hace visible
        f.setSize(300, 200);
        f.setVisible(true);
    }

    public static void main(String args[]) {
        TestInner obj = new TestInner();
        obj.launchFrame();
    }
}
```

Gráfico 193. Ejemplo de control de eventos con clase interna

Al pulsar y arrastrar el ratón se muestran las coordenadas de este.

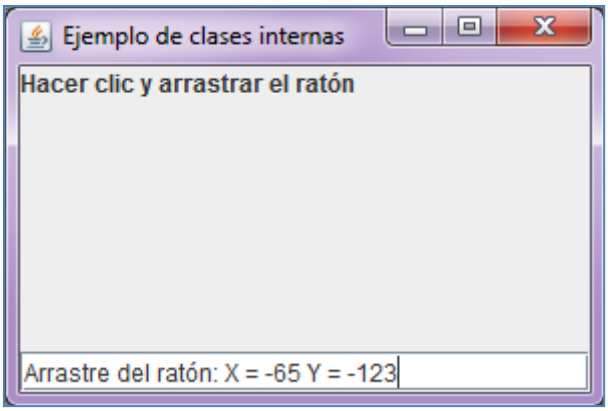


Gráfico 194. Resultado de la ejecución

Todo el código de este ejemplo lo encontrareis en **Ejemplo32_ Eventos_Internas.zip**

CONTROL DE EVENTOS MEDIANTE CLASES ANÓNIMAS

Es posible incluir una definición de clase completa dentro del ámbito de una expresión. Esta fórmula permite definir lo que se denomina una clase interna anónima y crear la instancia al mismo tiempo.

La compilación de una clase anónima genera un archivo del tipo TestAnonymous$1.class.

Veamos el mismo ejemplo realizado anteriormente con clase interna, ahora con clase anónima.

```
public class TestAnonymous {
    private Frame f;
    private TextField tf;

    public TestAnonymous() {
        f = new Frame("Anonymous classes example");
        tf = new TextField(30);
    }

    public void launchFrame() {
        Label label = new Label("Click and drag the mouse");
        f.add(label, BorderLayout.NORTH);
        f.add(tf, BorderLayout.SOUTH);
        // Añadimos el listener que utiliza la clase anonima
        f.addMouseMotionListener(new MouseMotionAdapter() {
            public void mouseDragged(MouseEvent e) {
                String s = "Mouse dragging:  X = " + e.getX()
                        + " Y = " + e.getY();
                tf.setText(s);
            }
        }); // Observar que cerramos los parentesis
        f.addMouseListener(new MouseClickHandler());
        f.setSize(300, 200);
        f.setVisible(true);
    }

    public static void main(String args[]) {
        TestAnonymous obj = new TestAnonymous();
        obj.launchFrame();
    }
}
```

Gráfico 195. Ejemplo de control de eventos con clase anónima

CREACIÓN DE MENÚS

Para crear un menú necesitamos seguir los siguientes pasos:

1. Hay que crear un objeto JMenuBar y adjuntarlo dentro de un contenedor de menús, como JFrame.
2. Crear uno o varios objetos JMenu y agregarlos al objeto barra de menús.
3. Crear uno o varios objetos JMenuItem y agregarlos al objeto menú.

1. CREACIÓN DE UNA BARRA DE MENÚS CON JMENUBAR

Un componente JMenuBar es una barra de menús, es decir, un menú horizontal. Sólo se puede agregar a un objeto JFrame y constituye la raíz de todos los árboles de menús. El objeto JFrame muestra un solo componente JMenuBar a la vez. Sin embargo, el componente JMenuBar puede modificarse en función del estado del programa para que aparezcan menús diferentes según la situación.

```
f = new JFrame("JMenuBar");
mb = new JMenuBar();
f.setJMenuBar(mb);
```

Gráfico 196. Creación del componente JMenuBar

2. CREACIÓN DE UN MENÚ CON JMENU

El componente JMenu crea un menú desplegable básico que puede añadirse a un componente JMenuBar o a otro objeto JMenu.

```
f = new JFrame("Menu");
mb = new JMenuBar();
m1 = new JMenu("File");
m2 = new JMenu("Edit");
m3 = new JMenu("Help");
mb.add(m1);
mb.add(m2);
mb.add(m3);
f.setJMenuBar(mb);
```

Gráfico 197. Creación de menús con JMenu

La salida genera será la siguiente:

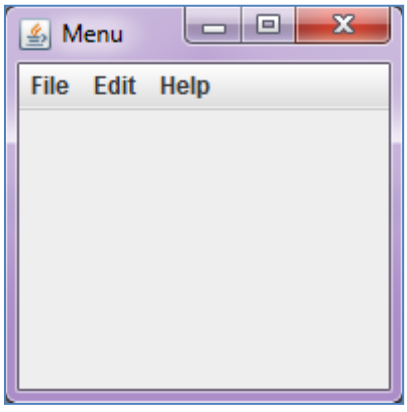


Gráfico 198. Menús

3. CREACIÓN DE UNA OPCIÓN DE MENÚ CON JMENUITEM

Los componentes JMenuItem son los nodos de texto de las hojas de un árbol de menús.

En general, para definir el comportamiento de un menú se agrega un elemento ActionListener al objeto JMenuItem.

```
mi1 = new JMenuItem("New");
mi2 = new JMenuItem("Save");
mi3 = new JMenuItem("Load");
mi4 = new JMenuItem("Quit");
mi1.addActionListener(this);
mi2.addActionListener(this);
mi3.addActionListener(this);
mi4.addActionListener(this);
m1.add(mi1);
m1.add(mi2);
m1.add(mi3);
m1.addSeparator();
m1.add(mi4);
```

Gráfico 199. Opciones del menú

El código anterior genera la siguiente salida:



Gráfico 200. Componente JMenuItem

CREACIÓN DE UN COMPONENTE JCHECKBOXMENUITEM

El componente JCheckboxMenuItem es una opción de menú que se puede activar o desactivar. Debe controlarse con la interfaz ItemListener.

```
f = new JFrame("CheckboxMenuItem");
mb = new JMenuBar();
m1 = new JMenu("File");
m2 = new JMenu("Edit");
m3 = new JMenu("Help");
mb.add(m1);
mb.add(m2);
mb.add(m3);
f.setJMenuBar(mb);
........
mi5 = new JCheckBoxMenuItem("Persistent");
mi5.addItemListener(this);
m1.add(mi5);
```

Gráfico 201. Opciones del menú con JCheckboxMenuItem

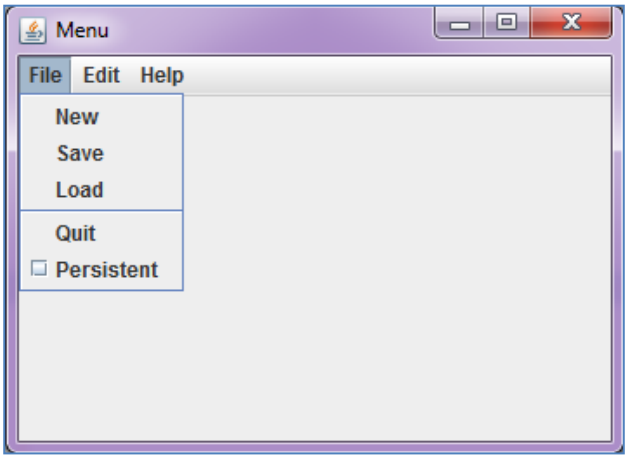


Gráfico 202. Salida generada con el componente JCheckboxMenuItem

Todo el código de este ejemplo lo encontrareis en **Ejemplo33_ Menus.zip**



RECUERDA QUE...

- Con Swing podemos crear interfaces gráficas. Para ello necesitamos de un administrador de diseño, un contenedor y los componentes necesarios.
- Los eventos son las acciones del usuario, mientras que los manejadores de eventos es el código que se ejecutará a modo de respuesta ante un evento dado.

11.- HILOS

Un hilo es una unidad de trabajo que permite ejecutar varias tareas de forma concurrente.

Una de las características que tiene Java es la programación multihilo como vimos en el primer capítulo, esto hace que sea un lenguaje más rápido y seguro.

Un hilo está formado por tres partes:

- Una CPU virtual
- El código que ejecuta la CPU
- Los datos con los que trabaja el código

Un proceso es un programa en ejecución. Uno o varios hilos constituyen un proceso.

FORMAS DE CREAR UN HILO

Para crear un hilo podemos hacerlo de dos formas:

- Heredar de la clase Thread y sobrescribir el método run()
- Implementar la interface Runnable con su método run()

En ambos casos el resultado es el mismo, pero que opción escogemos?

- Implementar la interface Runnable:
  - Es un mejor diseño orientado a objetos
  - Herencia simple
  - Consistente
- Heredar de la clase Thread:
  - Simplifica el código

Veamos un ejemplo con la clase Thread

```
public class MiHilo extends Thread{

    @Override
    public void run() {
        // todo el codigo que ejecuta el hilo
        int i = 0;
        while(true){
            System.out.println("Hola " + i++);
            if (i == 50){
                break;
            }
        }
    }
}
```

Gráfico 203. Creación de hilos con la clase Thread

Un ejemplo con la Interface Runnable

```
public class HelloRunner implements Runnable{

    public void run() {
        // todo el codigo que ejecuta el hilo
        int i = 0;
        while(true){
            System.out.println("Hola " + i++);
            if (i == 50){
                break;
            }
        }
    }
}
```

Gráfico 204. Creación de hilos con la interface Runnable

ARRANCAR UN HILO

Para arrancar un hilo utilizamos el método start() de la clase Thread. Arrancar un hilo no significa que se ejecute en el instante, sino que se pone en estado Runnable a la espera de que el planificador lo elija para comenzar su ejecución.

```
Thread miHilo = new MiHilo();
miHilo.start();
```

Gráfico 205. Arrancar un hilo

```
// crear el hilo
HelloRunner r = new HelloRunner();
Thread t1 = new Thread(r);
Thread t2 = new Thread(r);

// Arrancar el hilo en un hilo aparte.
t1.start();
t2.start();
```

Gráfico 206. Arrancar dos hilos

CICLO DE VIDA DE UN HILO

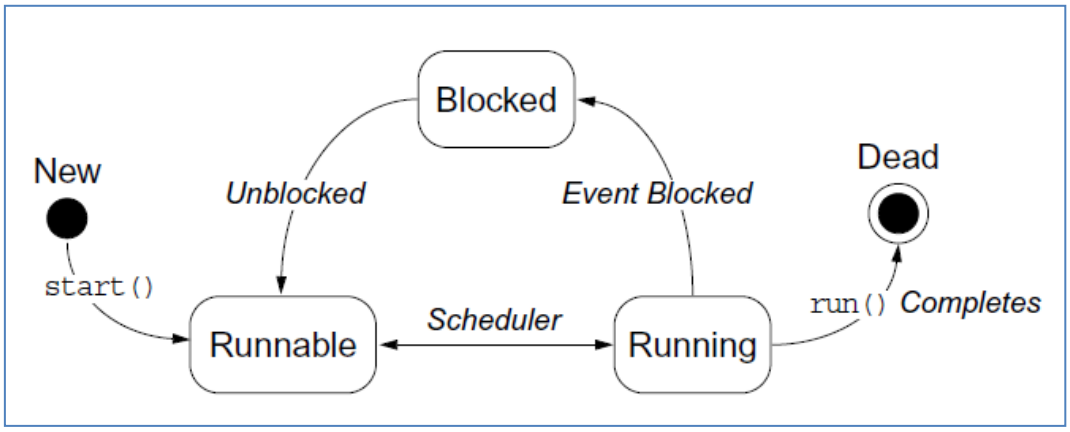


Gráfico 207. Ciclo de vida de un hilo

- **New**; Cuando un hilo se crea y todavía no se ha arrancado decimos que se encuentra en estado New.
- **Runnable**; Este es el estado de un hilo que se ha arrancado y todavía no ha sido elegido por el planificador para su ejecución.
- **Running**; Cuando el planificador elige un hilo de entre los que se encuentran en estado Runnable y este pasa a ejecución diremos que este hilo está en estado Running.
- **Blocked**; Si un hilo está en estado Running y no hay nada que lo bloque, el hilo se ejecutará hasta el final generando una programación secuencial y no concurrente. Lo ideal es ir bloqueando los hilos para que el planificador pueda elegir otro.
- **Dead**; Consideramos que un hilo está muerto cuando se ha completado el método run().

PLANIFICAR HILOS

Como hemos explicado anteriormente, para planificar hilos estos deben pasar a un estado bloqueado de vez en cuando permitiendo de esta manera la ejecución de otros hilos. Gracias a este mecanismo obtenemos una programación concurrente.

Existen varios métodos para poner el hilo en estado bloqueado.

Saber si un hilo está vivo:

- **isAlive**(); devuelve un valor booleano que nos indica si no ha terminado su ejecución (hilo vivo) o si (hilo muerto)

Acceder a la prioridad de un hilo:

- **getPriority**(); nos devuelve la prioridad de un hilo
- **setPriority**(); permite establecer la prioridad de un hilo (valores entre 1 y 10). Por defecto todos los hilos tienen una prioridad media (aprox. 5)

Bloquear un hilo:

- **Thread.sleep**(); es un método estático de la clase Thread que nos permite dormir un hilo por un periodo de tiempo expresado en milisegundos. Transcurrido este tiempo el hilo vuelve a estar en estado Runnable, listo para que el planificador lo pueda volver a elegir ara su proceso.
- **join**(); Este método permite unir hilos. El hilo al cual se le aplica este método entra directamente a ejecución uniéndose al que estaba ejecutándose en ese momento.
- **Thread.yield**() ; también es una método estático de la clase Thread y lo que hace es sacar el hilo de estado Running para ponerlo en estado Runnable.

Veamos un ejemplo del método sleep:

```
 */
public class DormirHilo implements Runnable{

    public void run() {
        // todo el codigo que ejecuta el hilo
        int i = 0;
        while(true){
            try{
                // Bloquear el hilo durante 10 milisegundos
                // Transcurrido este tiempo el hijo estara de nuevo
                // en estado Runnable
                Thread.sleep(10);
            }catch(InterruptedException ex){
                System.out.println("Error");
            }
            System.out.println("Hola " + i++);
            if (i == 50){
                break;
            }
        }
    }
}
```

Gráfico 208. Dormir hilos

Ejemplo del método join:

```
DormirHilo d = new DormirHilo();
Thread hilo1 = new Thread(d);
Thread hilo2 = new Thread(d);
hilo1.start();
hilo2.start();

try {
    hilo2.join();
} catch (InterruptedException e) {
    System.out.println(e.getMessage());
}
```

Gráfico 209. Unir hilos

Todo el código de este ejemplo lo encontrareis en **Ejemplo34_ Hilos.zip**

SINCRONIZACIÓN

Cuando varios hilos están accediendo al mismo recurso puede ocurrir lo que se denomina lecturas erróneas.

Lo explicamos con un ejemplo. Imaginamos que un hilo ha realizado un proceso en el cual debe actualizar el valor de una variable, pues bien ese cambio todavía no se ha producido y el otro hilo ha leído dicho valor. El resultado es que el segundo hilo está trabajando con un dato erróneo, lo correcto hubiese sido esperar a que el primer hilo actualizase el valor para que el segundo lo leyera.

Para evitar esto podemos utilizar la palabra **synchronized** para sincronizar un método o un fragmento de código.

El funcionamiento sería el siguiente. El primer hilo que accede al método o código sincronizado bloquea a los demás hilos, por lo cual no podrán acceder. Cuando el hilo finaliza la ejecución desbloquea a los demás para que el siguiente pueda acceder.

Veamos un ejemplo: Si tenemos dos hilos que acceden a la misma pila y no utilizásemos la sincronización podría darse el siguiente caso.

El primer hilo accede al método poner() y añade un carácter a la pila. A continuación debería incrementar el índice para que el siguiente elemento se añada a continuación. Supongamos que antes de que esto ocurra el otro hilo accede al método sacar y previamente decrementa el índice para recuperar el último carácter añadido.

Vemos que realmente no está accediendo al último carácter, sino al penúltimo ya que el índice no estaba actualizado cuando se ha producido la lectura de la variable índice.

La sincronización permite que una vez que un hilo accede a un recurso sincronizado el otro debe permanecer en estado bloqueado (no ejecuta ningún otro código) hasta que el hilo termine su ejecución.

```
public class MiPila {

    int indice = 0;
    char[] datos = new char[6];

    public synchronized void poner(char c) {
        datos[indice] = c;
        indice++;
    }

    public char sacar() {
        synchronized (this) {
            indice--;
            return datos[indice];
        }
    }
}
```

Gráfico 210. Pila con métodos sincronizados

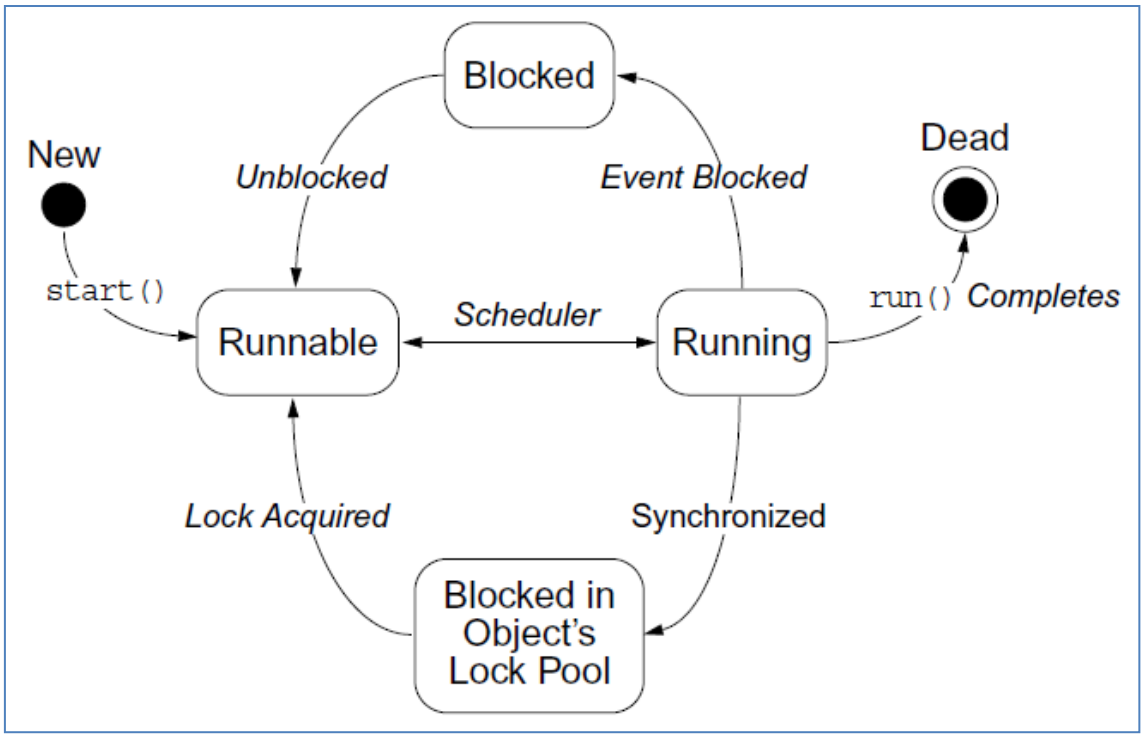


Gráfico 211. Sincronización de hilos

ITERACIÓN ENTRE HILOS

Para iterar con hilos utilizamos los métodos siguientes:

- **wait**(); Pone a los hilos en espera.
- **notify**(); Despierta un hilo elegido al azar.
- **notifyAll**(); Despierta a todos hilos que estaban en estado wait.

Para ver cómo utilizar estos métodos vamos a explicarlo con un ejemplo muy común productor/consumidor.

PRODUCTOR / CONSUMIDOR

El productor se encarga de generar caracteres mediante un método aleatorio y añadirlos a la pila.

En este ejemplo vamos a crear 2 productores por lo cual a cada uno de ellos le asignaremos un numero a partir de la variable contador.

Para dar oportunidad de que se ejecuten varios hilos a la vez (productores y consumidores) utilizamos el método sleep por un periodo de tiempo también aleatorio.

```
public class Productor implements Runnable {

    private MiPila pila;
    private int num;
    private static int contador = 1;

    public Productor(MiPila pila) {
        this.pila = pila;
        num = contador++;
    }

    public void run() {
        char c;
        for(int i=0; i < 10; i++){
            c = (char)(Math.random() * 26 + 'A');
            pila.poner(c);
            try{
                Thread.sleep((int)Math.random() * 300);
            }catch(InterruptedException ex){
                System.out.println("Error");
            }
        }
    }
}
```

Gráfico 212. Ejemplo Productor

El consumidor es quien va sacando caracteres de la pila.

También vamos a tener 2 consumidores, cada uno con un numero que asignamos a través de la variable contador.

Igualmente que en el productor, utilizamos el método sleep para dar oportunidad de ejecución a los otros hilos.

```
public class Consumidor implements Runnable {

    private MiPila pila;
    private int num;
    private static int contador = 1;

    public Consumidor(MiPila pila) {
        this.pila = pila;
        num = contador++;
    }

    public void run() {
        char c;
        for (int i = 0; i < 10; i++) {
            c = pila.sacar();
            try {
                Thread.sleep((int) Math.random() * 300);
            } catch (InterruptedException ex) {
                System.out.println("Error");
            }
        }
    }
}
```

Gráfico 213. Ejemplo Consumidor

La pila expone dos métodos: poner y sacar. Ambos métodos son sincronizados para evitar lecturas sucias como vimos en el apartado anterior.

Es en este código donde vamos a utilizar los métodos wait() y notify().

Cuando el consumidor intenta sacar un carácter de la pila pero esta está vacía entonces se pone en estado de espera.

Cuando el productor produce un carácter y lo pone en la pila, emite una notificación y despierta a un hilo de los que están en estado de espera (Consumidores).

```
public class MiPila {

    private List<Character> buffer = new ArrayList<Character>(20);

    public synchronized void poner(char c) {
        //this.notifyAll();
        this.notify(); // avisa al consumidor de que puede consumir
        System.out.println("Productor : " + c);
        buffer.add(c);
    }

    public synchronized char sacar() {
        char c;
        while(buffer.isEmpty()){
            try{
                this.wait(); // detener el hilo
            }catch (InterruptedException ex){
                System.out.println("Error");
            }
        }
        c= buffer.remove(buffer.size() -1);
        System.out.println("Consumidor : " + c);
        return c;
    }
}
```

Gráfico 214. Pila sincronizada

Por último, la clase principal donde creamos los hilos productor y consumidor y los arrancamos.

```
public class Main {

    public static void main(String[] args) {
        MiPila pila = new MiPila();

        Productor p1 = new Productor(pila);
        Thread hiloProductor1 = new Thread(p1);
        hiloProductor1.start();

        Productor p2 = new Productor(pila);
        Thread hiloProductor2 = new Thread(p2);
        hiloProductor2.start();

        Consumidor c1 = new Consumidor(pila);
        Thread hiloConsumidor1 = new Thread(c1);
        hiloConsumidor1.start();

        Consumidor c2 = new Consumidor(pila);
        Thread hiloConsumidor2 = new Thread(c2);
        hiloConsumidor2.start();
    }

}
```

Gráfico 215. Clase principal

Todo el código de este ejemplo lo encontrareis en **Ejemplo35_ Sincronizacion.zip**



RECUERDA QUE...

- Los hilos es el mecanismo utilizado en Java para una programación concurrente.
- Cada hilo está formado por: Código, datos y CPU.
- Los hilos se pueden sincronizar mediante wait, notify y notifyAll

12.- CONEXIONES EN RED CON SOCKETS

Un socket es una conexión que se estable a través de la red entre dos máquinas, ninguna de ellas es un servidor.

CREAR SOCKETS

Para crear socket cada aplicación instalada en cada máquina ha de tener:

- **Un stream de entrada**; se utiliza para escuchar los mensajes de la otra máquina.
- **Un stream de salida**; se utiliza para enviar mensajes a la otra máquina.

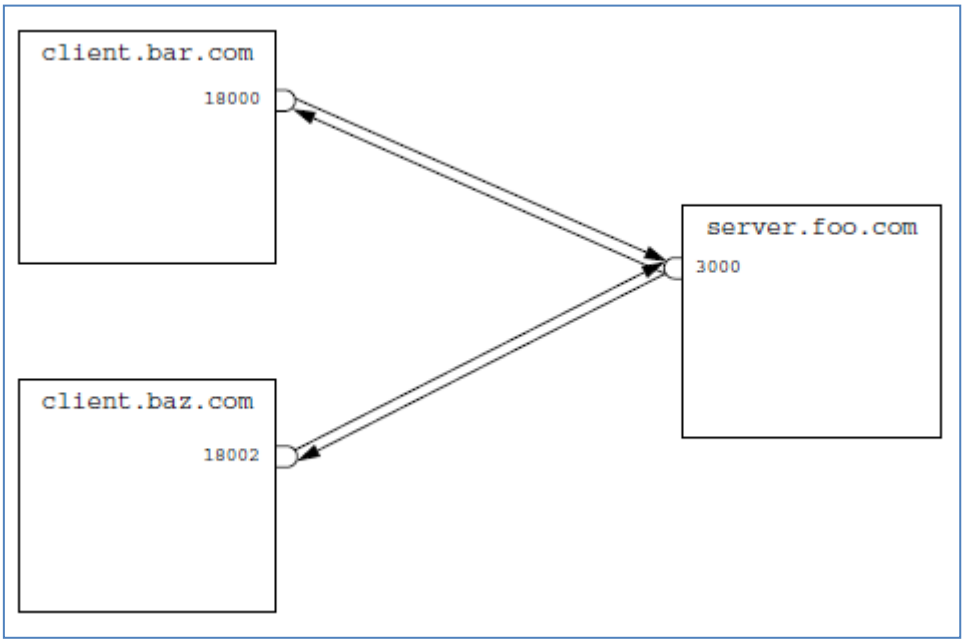


Gráfico 216. Ejemplo conexión con sockets

Para establecer la conexión es necesario conocer los siguientes datos:

- La dirección o el nombre de la maquina remota
- El numero de puerto de la maquina que actúa como servidor

SERVIDOR Y CLIENTE

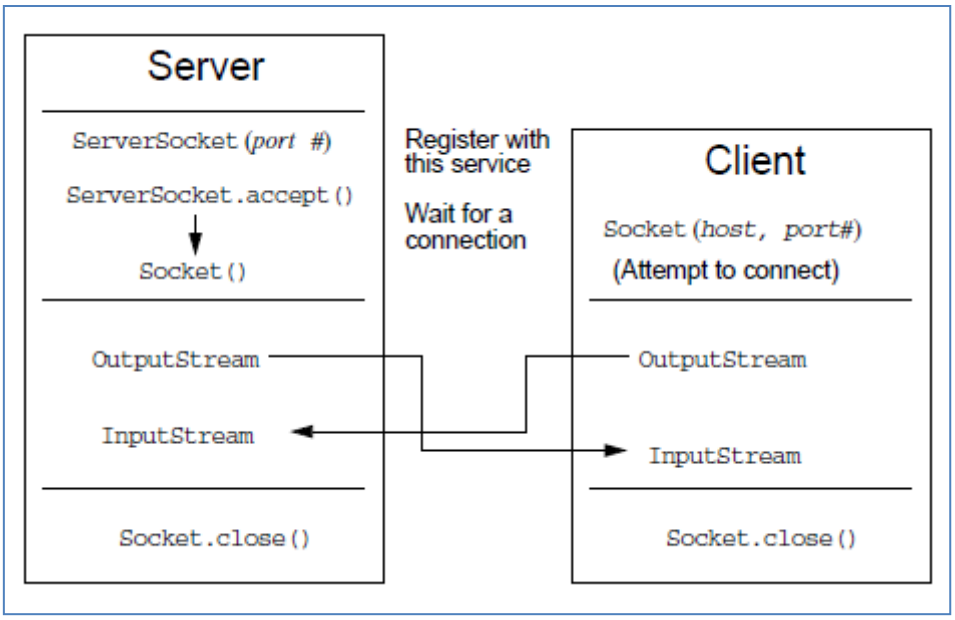


Gráfico 217. Modelo cliente - servidor

SERVIDOR .-

Desde el código del servidor se habilita un puerto para escuchar las conexiones a través del Socket.

A partir de ahí generamos un bucle infinito para que constantemente pueda escuchar las conexiones de los supuestos clientes y por supuesto aceptarlas.

En este ejemplo simplemente enviamos un mensaje al cliente.

```
ServerSocket s = null;

// Creamos el socket en el puerto 5432
try {
    s = new ServerSocket(5432);
} catch (IOException e) {
    e.printStackTrace();
}

// Ejecutamos un bucle para estar constantemente escuchando y aceptando
while (true) {
    try {
        //Espera aqui a escuchar una conexión
        Socket s1 = s.accept();

        // Generamos un output stream asociado con el socket
        OutputStream s1out = s1.getOutputStream();
        BufferedWriter bw = new BufferedWriter(
                new OutputStreamWriter(s1out));

        // Enviamos un mensaje
        bw.write("Hello Net World! \n");

        // Cerramos la conexion pero no el socket
        bw.close();
        s1.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Gráfico 218. Fragmento ejemplo Servidor

CLIENTE .-

Desde el cliente se establece una conexión al servidor a través del puerto abierto por este último.

Se genera un input stream para poder escuchar los mensajes recibidos y mostrarlos en consola.

```
try {
    // Abrimos una conexión al servidor en el puerto 5432
    Socket s1 = new Socket("127.0.0.1", 5432);

    // Generamos un input stream desde el socket
    InputStream is = s1.getInputStream();
    BufferedReader dis = new BufferedReader(new InputStreamReader(is));

    // Leemos el mensaje enviado por el servidor y lo mostramos
    System.out.println(dis.readLine());

    // Cerramos los recursos abiertos
    dis.close();
    s1.close();

} catch (ConnectException connExc) {
    System.err.println("No se pudo establecer la conexion.");

} catch (IOException e) {
    System.out.println("Error");
}
```

Gráfico 219. Fragmento ejemplo cliente

Todo el código de este ejemplo lo encontrareis en **Ejemplo36_ Sockets.zip**



RECUERDA QUE...

- Los sockets permiten la comunicación entre dos aplicaciones desplegadas en distintas máquinas sin utilizar un entorno cliente-servidor.

13.- CONEXIÓN CON BASES DE DATOS

API JDBC

JDBC (Java Database Connectivity) está compuesto por un número determinado de clases e interfaces que permiten a cualquier programa escrito en Java acceder a una base de datos.

Este conjunto de clases reside en los paquetes: java.sql y javax.sql

javax.sql añade funciones de servidor como los RowSet, los pools de conexiones o las transacciones.

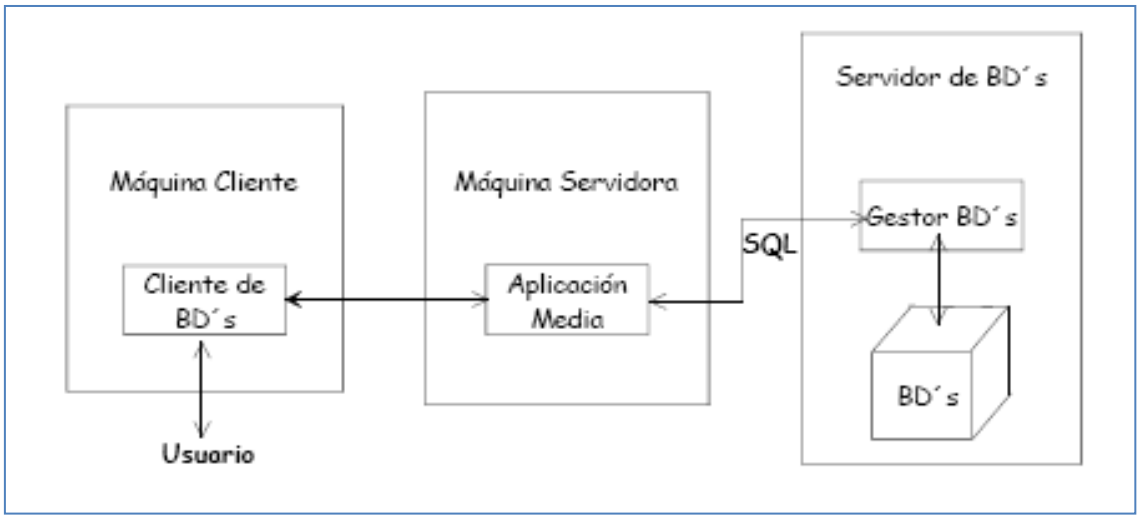


Gráfico 220. Esquema de conexión

Para podernos conectar a la base de datos necesitamos un controlador o también conocido como Driver. Este driver no es más que la implementación del API JDBC para un gestor de Base de datos concreto Oracle, MySQL, Derby, ...etc.

TIPOS DE CONTROLADORES

Los diferentes tipos de controladores que tenemos son:

- Puente JDBC-ODBC
- 100 % Java nativo
- 100 % Java / Protocolo independiente
- 100 % Java / Protocolo nativo

PUENTE JDBC-ODBC

Muchos servidores de BD's utilizan protocolos específicos para la comunicación. Esto implica que el cliente tiene que aprender un lenguaje nuevo para comunicarse con el servidor. Microsoft ha establecido una norma común para comunicarse con las bases de datos llamada Conectividad Abierta de Bases de Datos (ODBC). Hasta que esta norma no apareció, los clientes eran específicos del servidor. Utilizando la API ODBC lo que conseguimos es desarrollar software independiente del servidor.

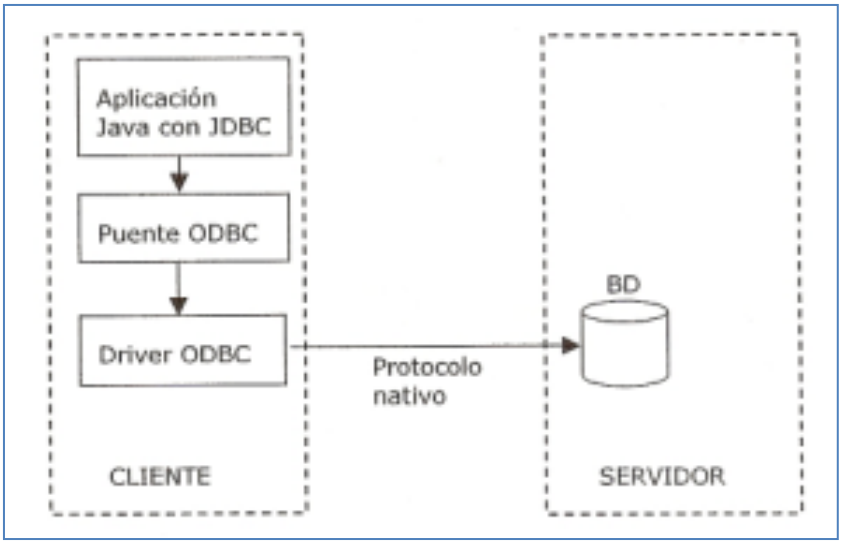


Gráfico 221. Conexión a través de ODBC

100 % JAVA NATIVO

Son controladores que usan el API de Java JNI (Java native interface) para presentar una interfaz Java a un controlador binario nativo del SGBD.

Su uso, igual que los anteriores, implica instalar el controlador nativo en la máquina cliente. Suelen tener un rendimiento mejor que los controladores escritos en Java completamente, aunque un error de funcionamiento de la parte nativa del controlador puede causar problemas en la máquina virtual de Java.

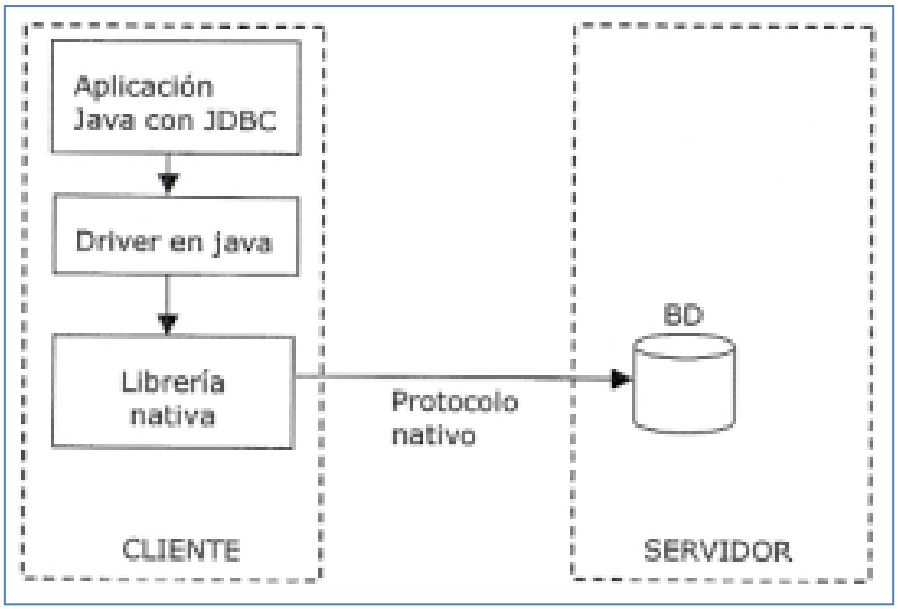


Gráfico 222. Conexión a través de Librería nativa

100 % JAVA / PROTOCOLO INDEPENDIENTE

Controladores escritos en Java que definen un protocolo de comunicaciones que interactúa con un programa de middleware que, a su vez, interacciona con un SGBD. El protocolo de comunicaciones con el middleware es un protocolo de red independiente del SGBD y el programa de middleware debe ser capaz de comunicar los clientes con diversas bases de datos.

El inconveniente de esta opción estriba en que debemos tener un nivel más de comunicación y un programa más (el middleware).

100 % JAVA / PROTOCOLO NATIVO

Son los controladores más usados en accesos de tipo intranet (los usados generalmente en aplicaciones web).

Son controladores escritos totalmente en Java, que traducen las llamadas JDBC al protocolo de comunicaciones propio del SGBD. No requieren ninguna instalación adicional ni ningún programa extra.

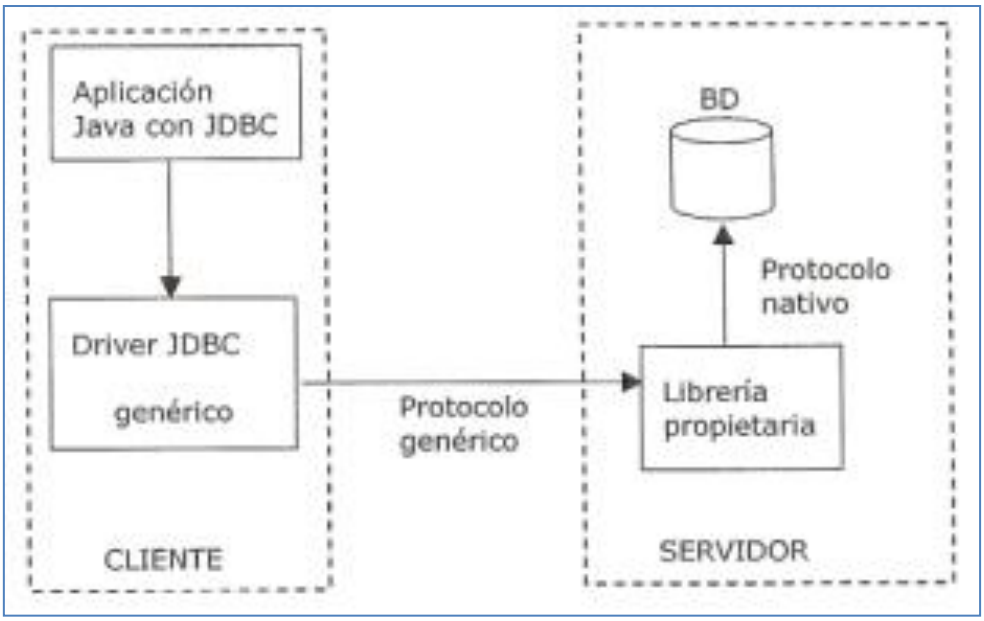


Gráfico 223. Conexión a través de protocolo nativo

ABRIR Y CERRAR CONEXIONES

Para poder acceder a una base de datos es necesario establecer una conexión.

Antes de conectarnos con la base de datos hay que registrar el controlador apropiado. Java utiliza la clase **DriverManager** para cargar inicialmente todos los controladores JDBC disponibles y que deseemos utilizar.

Para cargar un controlador se emplea del método **forName()** de la clase Class que devuelve un objeto Class asociado con la clase que se le pasa como parámetro.

La interfaz **Connection** es la que se encarga de la conexión con la base de datos. Mediante el método **getConnection()**, obtenemos un objeto de la clase Connection. Esta clase contiene todos los métodos que nos permiten manipular la base de datos.

Para cerrar la conexión utilizamos el método **close()** de la interfaz Connection.

```
private Connection conexion;

public void abrirConexion() {
    try {
        // Cargar el driver de Derby
        Class.forName("org.apache.derby.jdbc.ClientDriver");

        // Abrir la conexion a la BBDD
        conexion = DriverManager.getConnection("jdbc:derby://localhost:1527/Tienda",
                "curso", "curso");

    } catch (SQLException ex) {
        System.out.println("Error al abrir la conexion");
        ex.printStackTrace();
    } catch (Exception ex) {
        System.out.println("error al cargar el driver");
    }
}
```

Gráfico 224. Fragmento para abrir una conexión a Derby

```
public void cerrarConexion() {
    try {
        // cerrar la conexion
        conexion.close();
    } catch (SQLException ex) {
        System.out.println("Error al cerrar la conexion");
    }
}
```

Gráfico 225. Fragmento para cerrar la conexión

CREAR CONSULTAS CON SQL

Para crear una consulta a la base de datos podemos utilizar 3 interfaces diferentes, cada uno tiene un uso especifico:

- **Statement**; Para ejecutar querys completas, es decir, tenemos todos los datos de la query.
- **PreparedStatement**; Para ejecutar querys parametrizadas. No tenemos todos los datos en este momento y utilizamos parámetros.
- **CallableStatement**; Para ejecutar querys a través de procedimientos almacenados definidos en la BBDD.

Statement

Para incluir una sentencia SQL utilizaremos la interfaz **Statement**, con el método **createStatement()** de la interfaz Connection.

Al crear una instancia del objeto Statement, podremos realizar sentencias SQL sobre la base de datos. Existen dos tipos de sentencias a realizar:

- Sentencias de modificación (update); Engloban a todos los comandos SQL que no devuelven ningún tipo de resultado como pueden ser los comandos INSERT, UPDATE, DELETE o CREATE.
- Sentencias de consulta (query); Son sentencias del tipo SELECT (sentencias de consulta) que retornan algún tipo de resultado.

Para las sentencias “update” la clase Statement nos proporciona el método siguiente que devolverá el número de filas afectadas por la sentencia SQL:

public abstract int executeUpdate(String sentenciaSQL)

```
// 1.- Abrir la conexion
abrirConexion();

// 2.- Preparar la query
Statement stm = conexion.createStatement();

// 3.- Ejecutar la query
int resultados = stm.executeUpdate("delete * from APP.Productos");
```

Gráfico 226. Ejemplo executeUpdate

Para las sentencias “query” la clase Statement utiliza el método siguiente:

public abstract ResultSet executeQuery(String sentenciaSQL)

```
// 1.- Abrir la conexion
abrirConexion();

// 2.- Preparar la query
Statement stm = conexion.createStatement();

// 3.- Ejecutar la query
ResultSet resultados = stm.executeQuery("select * from APP.Productos");
```

Gráfico 227. Ejemplo executeQuery

PreparedStatement

Define métodos para trabajar con instrucciones SQL pre compiladas, que son más eficientes.

También se usan para poder utilizar instrucciones SQL parametrizadas. Para establecer parámetros en una instrucción SQL basta con sustituir el dato por un signo de interrogación. (Select * from Tabla where Nombre=?).

El parámetro se sustituye por el valor mediante el método setXXX(int lugar, XXX Valor).

PreparedStatement no utiliza los métodos de Statement para ejecutar las querys, en su lugar, se pasa la query en el propio constructor.

Igual que antes podemos ejecutar dos tipos de consultas query y update.

```
// 1.- Abrir la conexion
abrirConexion();

// 2.- Preparar la query
String query = "insert into APP.Productos values (?,?,?) ";
PreparedStatement stm = conexion.prepareStatement(query);

// sustituimos los parámetros
stm.setInt(1, p.getId());
stm.setString(2, p.getDescripcion());
stm.setDouble(3, p.getPrecio());

// 3.- Ejecutar la query
int resultados = stm.executeUpdate();
```

Gráfico 228. Ejemplo PreparedStatement con executeUpdate

```
// 1.- Abrir la conexion
abrirConexion();

// 2.- Preparar la query
String query = "select * from APP.Productos where ID = ? ";
PreparedStatement stm = conexion.prepareStatement(query);

// asignar parámetros
stm.setInt(1, id);

// 3.- Ejecutar la query
ResultSet resultados = stm.executeQuery();
```

Gráfico 229. Ejemplo PreparedStatement con executeQuery

CallableStatement

Los objetos de este tipo se crean a través del método de Connection, prepareCall(). Se pasa como argumento de este método la llamada al procedimiento almacenado. Los CallableStatement también pueden ser parametrizados.

Un procedimiento almacenado es una macro registrada en la base de datos y que realiza operaciones de cualquier tipo.

```
// 1.- Abrir la conexion
abrirConexion();

// 2.- Preparar la query
String query = "{call procedimiento(?)}";
CallableStatement cstm = conexion.prepareCall(query);
cstm.setDouble(1, precio);

// 3.- Ejecutar la query
ResultSet resultados = cstm.executeQuery();
```

Gráfico 230. Ejemplo CallableStatement con executeQuery

PROCESAR DATOS

Para procesar los datos recibidos tras ejecutar la query depende del tipo de consulta que hemos lanzado.

- Recordamos que si era una query de consulta devolvía un objeto de tipo ResultSet.
- Si era una query de modificación entonces recogemos una valor entero que será el número de registros modificados

La interfaz ResultSet contendrá las filas o registros obtenidas mediante la ejecución de la sentencia de tipo “query”. Cada una de esas filas obtenidas se divide en columnas.

La interfaz ResultSet contiene un puntero que está apuntando a la fila actual. Inicialmente está apuntando por delante del primer registro. Para avanzar el puntero utilizamos el método **next().**

Una vez posicionados en una fila concreta, podemos obtener los datos de una columna específica utilizando los métodos getxxx() que proporciona la interfaz ResultSet, la “xxx” especifica el tipo de dato presente en la columna.

Para cada tipo de dato existen dos métodos getxxx():

- getxxx(String nombreColumna); Donde especificamos el nombre de la columna donde se encuentra el dato.
- getxxx(int numeroColumna); Donde se especifica la posición de la columna dentro de la consulta. Siempre empezando desde 1.

```
// 3.- Ejecutar la query
ResultSet resultados = cstm.executeQuery();

// 4.- Recorrer la coleccion de registros
while (resultados.next()) {
    lista.add(new Producto(resultados.getInt("ID"),
                           resultados.getString("DESCRIPCION"),
                           resultados.getDouble("PRECIO")));
}
```

Gráfico 231. Procesar resultados

MANEJO DE TRANSACCIONES

Para poder manejar transacciones debemos utilizar el método setAutocommit para especificar si queremos un commit implícito o no.

Connection.setAutocommit(boolean);

- Si lo ponemos a true; estamos diciendo que se hace un commit implícito.
- Si se pone a false; no se realizará ninguna modificación en la base de datos hasta que explícitamente se haga un commit.

Si la conexión no hace commit implícito:

- connection.commit(): valida la transacción
- connection.rollback(): anula la transacción

METADATOS

Los metadatos se pueden considerar como información adicional a los datos que almacena la tabla.

Se utiliza un objeto ResultSetMetaData para recoger los metadatos de una consulta. Dicho objeto expone los siguientes métodos:

- getColumnName(); devuelve el nombre de la columna.
- getColumnCount(); nos indica el numero de columnas de la consulta.
- getTableName(); devuelve el nombre de la tabla.
- getColumnType(); nos dice el tipo de datos de la columna.
- isReadOnly(); especifica si los datos son de solo lectura.

```
// 5.- Obtener los metadatos
ResultSetMetaData metadataRS = resultados.getMetaData();
int columnCount = metadataRS.getColumnCount();
for (int i = 0; i < columnCount; i++) {
    System.out.println("nombre columna " +
                            metadataRS.getColumnName(i));
}
```

Gráfico 232. Obtención de los metadatos de la consulta

Todo el código de este ejemplo lo encontrareis en **Ejemplo37_ JDBC.zip**



RECUERDA QUE...

- Con el API JDBC podemos acceder una base del datos sin importarnos el fabricante.
- JDBC permite abrir y cerrar conexiones, así como ejecutar consultas a la BBDD.

14.- NOVEDADES JAVA 8

EXPRESIONES LAMBDA

Una expresión lambda es un bloque de código que puede ser manipulado como una función sin necesidad de atarla a un identificador y que es posible manipularla en el lenguaje como se manipula cualquier otro valor u objeto.

Se usan varios términos para referirse a lo mismo, entre ellos, función anónima, literal función y constante función son de los que más abundan.

De nada serviría dar soporte en el lenguaje a esa construcción si esos bloques no pudieran ser tratados como valores: asignarlos a variables, pasarlos como argumentos a funciones, ser retornados como funciones, etc. Y entre todas esas cosas, la que sobre sale es la de que el lenguaje permita diseñar funciones de orden superior (forma funcional, funcional, functor); funciones que al menos permiten recibir como entrada funciones y devolver funciones como salida.

Una vez que se tiene ésto, es posible entonces enriquecer el diseño del programa con los famosos closures o clausuras.

Esa necesidad de pasar como argumento un bloque de código a un método de algún objeto siempre ha estado presente en las aplicaciones que se escriben con Java y cualquier otro lenguaje. El ejemplo que casi todo el mundo utiliza para hacer evidente esa necesidad es en el procesamiento de eventos y, en ese contexto casi siempre se toma a Swing como el escenario típico. Desde sus inicios, Java nos ha "obligado" a hacer eso con clases anónimas que implementan alguna interfaz funcional de manera directa o indirecta (indirecta cuando heredamos de alguna clase que ya brinda implementación por default para algunos métodos).

```
boton.addActionListener(new  ActionListener() {
    public void actionPerformed(ActionEvent event) {
        System.out.println ("Presionaste el botón y puedes ver este mensaje en consola");
    }
});
```

Gráfico 233. Eventos de boton

Como es conocido, ActionListener es una interfaz funcional (la descripción de una acción específica que debe ser implementada por quien quiera usarla). Esa acción o código como queramos llamarla es lo que requiere el objeto boton para realizar su trabajo. Las expresiones lambda llegan en su auxilio y la porción de código anterior, podemos escribirla ahora así:

```
boton.addActionListener(event -> System.out.println("Presionaste el botón y puedes ver este mensaje en consola");
```

Gráfico 234. Eventos con Lambda

STREAMS

Los streams no son un nuevo tipo de colección son una nueva forma de recorrer las colecciones distinta a los Iterator. La ventaja de los streams es que pueden procesarse de forma serializada o paralela y proporcionan un estilo de operaciones más funcionales. Un flujo consiste un una fuente (una colección), varias operaciones intermedias (de filtrado o transformación) y una operación final que produce un resultado (suma, cuenta...). Los streams son lazy de modo que las operaciones solo se realizan cuando se llama a la operación final, también son eficientes no necesitando en algunos casos procesar todos los elementos del stream para devolver el resultado final.

Tradicionalmente en la API de colecciones la iteración sobre los elementos debíamos proporcionarla de forma externa. Con Java 8 podemos expresarla de la siguiente forma.

```
int alturaMedia = personas.stream()
    .mapToInt((Persona p) -> { return p.getAltura(); })
    .average();
```

Gráfico 235. Recorrido de colecciones con stream()

REFERENCIAS DE MÉTODOS

Todo el código desarrollado previamente a Java 8 no hace uso de las lambdas, pero con las referencias a métodos podemos usar esos métodos ya implementados como si se tratasen de funciones lambdas. Hay diferentes formas de hacer referencias a métodos:

- A métodos estáticos
- A un método de una instancia concreta
- A un método de instancia de una instancia arbitraria de un tipo
- A un constructor

```
// Método estático
Persona::compareAltura

// Método de una instancia concreta
persona::getAltura

// Método de instancia de una instancia arbitraria de un tipo
Persona::getAltura

// Constructor
Persona::new
```

Gráfico 236. Métodos de referencia

INTERFACES FUNCIONALES

Una interfaz funcional es aquella que solo tiene un método abstracto (sin implementación). Algunos ejemplos de interfaces funcionales son Runnable, ActionListener, Comparator y Callable. Para definir una interfaz funcional se puede usar la anotación @FunctionalInterface y pueden representarse con una expresión lambda. En el siguiente ejemplo puede apreciarse que con las interfaces funcionales y las lambdas podemos hacer lo mismo de forma más clara, menos verbosa y con código más legible.

```
Collections.sort(personas, new Comparator<User>() {
    public int compare(Persona p1, Persona p2) {
        return p1.getAltura().compareTo(p2.getAltura());
    }
});
```

Gráfico 237. Clase anónima

En Java 8 podemos hacer:

```
Collections.sort(personas, (Persona u1, Persona u2) -> {
    p2.getAltura().compareTo(p2.getAltura())
});
```

Gráfico 238. Interfaces funcionales

MÉTODOS POR DEFECTO

Hasta ahora las interfaces en Java solo podían definir métodos pero no sus implementaciones. El problema con las interfaces es que cuando se modifican se rompen todas las clases que las usan. Esto se ha resuelto de tal forma que se puedan añadir nuevos métodos con implementación a las interfaces y ha sido necesario para incorporar las lambdas a interfaces existentes como List. En Java 8 las interfaces podrán incorporar implementaciones para algunos de sus métodos, teniendo así algo parecido a herencia múltiple.

```
public interface Math {
    int add(int a, int b);

    default int multiply(int a, int b) {
        return a * b;
    }
}
```

Gráfico 239. Método por defecto

MÉTODOS ESTÁTICOS EN INTERFACES

Además de definir métodos por defecto en las interfaces a partir de ahora podemos definir métodos estáticos. Definiendo métodos estáticos en las interfaces evitaremos tener que crear clases de utilidad. Podremos incluir en un mismo tipo (la interfaz) todos los métodos relacionados.

```
public interface Persona {

    String getNombre();
    int getAltura();

    static String toStringDatos() {
        return getNombre() + " " + getAltura();
    }
}
```

Gráfico 240. Método estático en interfaces

Todo el código de este ejemplo lo encontrareis en **Ejemplo38_ NovedadesJava8.zip**



RECUERDA QUE...

- Java 8 es la última versión liberada por Oracle.
- Siempre es interesante consultar la nueva API para ver sus cambios.

ÍNDICE DE GRÁFICOS

Gráfico 1. El código fuente se compila para cada plataforma 7
Gráfico 2. Se efectúa el linkado para cada plataforma 8
Gráfico 3. El ejecutable se distribuye a cada plataforma 8
Gráfico 4. El código fuente se compila independiente de la plataforma 9
Gráfico 5. El archivo Bytecode puede ser interpretado por distintas plataformas 9
Gráfico 6. Ediciones Java 10
Gráfico 7. Ejemplo declaración de clase 13
Gráfico 8. Clase no encapsulada 19
Gráfico 9. Crear un objeto con datos erróneos 19
Gráfico 10. Fragmento de una clase encapsulada 20
Gráfico 11. Fragmento de acceso a la clase encapsulada 20
Gráfico 12. Palabras clave 22
Gráfico 13. Ejemplos de declaración de arrays 34
Gráfico 14 . Ejemplo de posición de los corchetes 35
Gráfico 15. Ejemplo de creación de arrays 35
Gráfico 16. Almacenar elementos en un array 36
Gráfico 17. Representación en memoria de los arrays números y alumnos 36
Gráfico 18. Forma rápida de manejar arrays 37
Gráfico 19. Acceso a los elementos del array 37
Gráfico 20. Recorremos el array de números con el bucle for 38
Gráfico 21. Recorremos el array de alumnos con el bucle for-each 38
Gráfico 22. Sintaxis del método arraycopy 38
Gráfico 23. Ejemplo del método arraycopy 39
Gráfico 24. Resultado del array numeros 39
Gráfico 25. Declaración de matrices 39
Gráfico 26. Creación de matrices cuadradas 40
Gráfico 27. Creación de matriz no cuadrada 40
Gráfico 28. Almacenar elementos en una matriz 41
Gráfico 29. Representación en memoria de las matrices números y nums 41
Gráfico 30. Forma rápida de manejar matrices 41
Gráfico 31. Acceder al elemento situado en la segunda fila y primera columna 42
Gráfico 32. Recorrer una matriz con el bucle for 42

Gráfico 33. Resultado obtenido 42
Gráfico 34. Recorrer una matriz con el bucle for-each 43
Gráfico 35. Resultado obtenido 43
Gráfico 36. Diagramas de clase Empleado y Gerente 44
Gráfico 37. Implementación de la clase Empleado 44
Gráfico 38. Implementación de la clase Gerente 44
Gráfico 39. Diagramas de clase Empleado y Gerente a través de Herencia 45
Gráfico 40. Implementación de la clase Gerente a través de herencia 45
Gráfico 41. Ejemplo árbol de herencia 46
Gráfico 42. Método getDetails en la clase Empleado y Gerente 47
Gráfico 43. Sobre escritura incorrecta de un método 47
Gráfico 44. Invocar a un método sobreescrito 48
Gráfico 45. Método println sobrecargado 49
Gráfico 46. Ejemplo sobrecarga de constructores 51
Gráfico 47. Invocar constructores de la superclase 52
Gráfico 48. Declaración de clase heredando de Object 53
Gráfico 49. Ejemplo sobreescritura del método equals y hashCode 54
Gráfico 50. Ejemplo comparación de fechas 55
Gráfico 51. Resultados de la comparación de fechas 55
Gráfico 52. Impresión de objetos 56
Gráfico 53. Clase sin sobreescritura método toString 56
Gráfico 54. Clase Principal donde creamos e imprimimos los objetos 57
Gráfico 55. Resultados tras la impresión de los objetos 57
Gráfico 56. Clase con sobreescritura del método toString 57
Gráfico 57. Ejemplo de clase abstracta 59
Gráfico 58. Implementación del método abstracto en la clase Rectángulo 60
Gráfico 59. Implementación del método abstracto en la clase Circulo 60
Gráfico 60. Clase principal donde generamos las instancias 61
Gráfico 61. Diagrama UML clases abstractas 62
Gráfico 62. Diagrama UML con interface 63
Gráfico 63. Código interface ObjetoVolador 63
Gráfico 64. Implementación de la interface ObjetoVolador en la clase Avion 64
Gráfico 65. Varias clases implementan la misma interface 64
Gráfico 66. Diagrama UML con herencia múltiple 65

Gráfico 67. Implementación de la clase Animal 66
Gráfico 68. Implementación de la clase Ave simulando herencia múltiple 66
Gráfico 69. Diagrama UML con varias interfaces 67
Gráfico 70. Diagrama de clases para aplicar polimorfismo 68
Gráfico 71. Representación de objetos por herencia 69
Gráfico 72. Ejemplo polimorfismo 69
Gráfico 73. Error en el uso de polimorfismo 69
Gráfico 74. Ejemplo de colecciones homogéneas 70
Gráfico 75. Ejemplo de colecciones heterogéneas 71
Gráfico 76. Ejemplo de argumentos polimórficos 71
Gráfico 77. Ejemplo operador instanceof 72
Gráfico 78. Ejemplo conversión de objetos 72
Gráfico 79. Clases envolventes 73
Gráfico 80. Ejemplos de conversión de primitivos en clases envoltorio 74
Gráfico 81. Ejemplo de uso de la clase envolvente Integer 74
Gráfico 82. Ejemplo de autoboxing 74
Gráfico 83. Diagrama UML de la clase Count y dos instancias únicas 75
Gráfico 84. Ejemplo de variable estática 76
Gráfico 85. Acceso a una variable estática fuera de la clase 76
Gráfico 86. Ejemplo método estático 77
Gráfico 87. Ejemplo invocación a un método estático 77
Gráfico 88. Error al intentar acceder a un recurso no estático 78
Gráfico 89. Ejemplo inicializador estático 78
Gráfico 90. Clase principal que muestra la variable estática 79
Gráfico 91. Ejemplo de ejecución 79
Gráfico 92. Ejemplo propiedad final 80
Gráfico 93. Ejemplo método final 80
Gráfico 94. Ejemplo fragmento de clase final 81
Gráfico 95. Ejemplo de tipo enumerado 81
Gráfico 96. Fragmento de clase que utiliza el tipo enumerado 82
Gráfico 97. Clase principal donde se crea una carta 82
Gráfico 98. Tipo enumerado avanzado 83
Gráfico 99. Clase principal donde se usa el tipo enumerado 84
Gráfico 100. Consulta API 84

Gráfico 101. Ejemplo que suma números introducidos como argumentos iniciales 87
Gráfico 102. Resultado con datos correctos 87
Gráfico 103. Resultado con datos erróneos 87
Gráfico 104. Ejemplo try-catch 88
Gráfico 105. Resultado tras capturar excepciones 88
Gráfico 106. Ejemplo mejorado de try-catch 89
Gráfico 107. Resultado del ejemplo mejorado 89
Gráfico 108. Categorías de Excepciones 91
Gráfico 109. Métodos que lanzan excepciones 91
Gráfico 110. Excepción personalizada 92
Gráfico 111. Creación de excepción personalizada 93
Gráfico 112. Ejemplo sin utilizar aserciones 93
Gráfico 113. Ejemplo aserciones 94
Gráfico 114. API Collections 97
Gráfico 115. Ejemplo Set 98
Gráfico 116. Ejemplo List 98
Gráfico 117. API interface Map 99
Gráfico 118. Ejemplo Map 100
Gráfico 119. Clase que implementa la interface Comparable 102
Gráfico 120. Clase principal que añade elementos al árbol para ordenarlos 103
Gráfico 121. Resultado de ejecución 103
Gráfico 122. Sintaxis del método compare 103
Gráfico 123. Clase ClienteSimple 104
Gráfico 124. Clase ComparadorNombre 104
Gráfico 125. Clase ComparadorNif 104
Gráfico 126. Clase principal que añade elementos al árbol para luego ordenarlos 105
Gráfico 127. Recuperar elementos de una colección 105
Gráfico 128. Recuperar elementos de una colección usando genéricos 106
Gráfico 129. Recuperar elementos de una colección usando el método for-each 106
Gráfico 130. Genéricos con mapas 106
Gráfico 131. Tabla que recoge los usos de los tipos genéricos 107
Gráfico 131. API Collections con tipos genéricos 107
Gráfico 132. Creación de un iterador con tipo genérico 108
Gráfico 133. Diagrama UML de Iterator 109

Gráfico 134 . Código que procesa argumentos de entrada 110
Gráfico 135. Ejemplo de ejecución 110
Gráfico 136. Ejemplo que muestra las propiedades 111
Gráfico 137. Creamos una nueva propiedad al ejecutar la clase 111
Gráfico 138. Ejemplo de ejecución 111
Gráfico 139. Ejemplo que lee un fichero para copiar su contenido en otro 115
Gráfico 140. Ejemplo anterior mejorado utilizando buffers 116
Gráfico 141. Lectura de datos a través de flujos de bytes 117
Gráfico 142. Escritura de datos a través de flujos de bytes 117
Gráfico 143. Herencia InputStream 118
Gráfico 144. Herencia OutputStream 118
Gráfico 145. Herencia Reader 119
Gráfico 146. Herencia Writer 119
Gráfico 147. Ejemplo de Serialización 120
Gráfico 148. Ejemplo de deserialización 121
Gráfico 149. Solicitar datos por consola 123
Gráfico 150. Lectura de datos por consola con Scanner 124
Gráfico 151. Lectura de fichero 126
Gráfico 152. Escritura en fichero 127
Gráfico 153. Elementos de una interface gráfica 131
Gráfico 154. Jerarquía de los contenedores de nivel superior 132
Gráfico 155. Representación de un contenedor JApplet 132
Gráfico 156. Representación de un contenedor JFrame 133
Gráfico 157. Representación de un contenedor JDialog 133
Gráfico 158. Representación de un contenedor JWindow 133
Gráfico 159. Representación de un contenedor JPanel 134
Gráfico 160. Representación de un contenedor JScrollPane 134
Gráfico 161. Representación de un contenedor JSplitPane 135
Gráfico 162. Representación de un contenedor JTabbedPane 135
Gráfico 163. Representación de un contenedor JToolBar 136
Gráfico 164. Ejemplo BorderLayout 137
Gráfico 165. Resultado tras ejecutar el ejemplo BorderLayout 137
Gráfico 166. Resultado tras ejecutar el ejemplo FlowLayout 138
Gráfico 167. Ejemplo FlowLayout 139

Gráfico 168. Resultado tras ejecutar el ejemplo BoxLayout 140
Gráfico 169. Ejemplo BoxLayout 140
Gráfico 170. Resultado tras ejecutar el ejemplo CardLayout 141
Gráfico 171. Declaración clase EjemploCardLayout 141
Gráfico 172. Fragmento de la clase EjemploCardLayout donde construimos los paneles 142
Gráfico 173. Constructor de la clase EjemploCardLayout 142
Gráfico 174. Manejador de evento para la clase EjemploCardLayout 143
Gráfico 175. Lanzamos GUI para EjemploCardLayout 143
Gráfico 176. Ejemplo GridLayout 144
Gráfico 177. Resultado tras ejecutar el ejemplo CardLayout 145
Gráfico 178. Ejemplo GridBagLayout 145
Gráfico 179. Jerarquía de componentes swing 147
Gráfico 180. Propiedades comunes de los componentes 148
Gráfico 181. Propiedades específicas del componente JComboBox 148
Gráfico 182. Ejemplo HelloWorld con construcción programática. 149
Gráfico 183. Crear un nuevo JFrame 150
Gráfico 184. Diseño de un JFrame 151
Gráfico 185. El usuario genera un evento al pulsar el botón 151
Gráfico 186. Clase donde se genera el evento 152
Gráfico 187. Clase donde se maneja el evento 153
Gráfico 188. Resultado de la ejecución 153
Gráfico 189. Jerarquía de clases de eventos de la interfaz gráfica 153
Gráfico 190. Categorías de eventos, métodos e interfaces 154
Gráfico 191. Categorías de eventos, métodos e interfaces (Continuación) 155
Gráfico 192. Clase que captura el evento 155
Gráfico 193. Ejemplo de control de eventos con clase interna 156
Gráfico 194. Resultado de la ejecución 157
Gráfico 195. Ejemplo de control de eventos con clase anónima 158
Gráfico 196. Creación del componente JMenuBar 159
Gráfico 197. Creación de menús con JMenu 159
Gráfico 198. Menús 159
Gráfico 199. Opciones del menú 160
Gráfico 200. Componente JMenuItem 160
Gráfico 201. Opciones del menú con JCheckboxMenuItem 161

Gráfico 202. Salida generada con el componente JCheckboxMenuItem 161
Gráfico 203. Creación de hilos con la clase Thread 163
Gráfico 204. Creación de hilos con la interface Runnable 163
Gráfico 205. Arrancar un hilo 163
Gráfico 206. Arrancar dos hilos 164
Gráfico 207. Ciclo de vida de un hilo 164
Gráfico 208. Dormir hilos 166
Gráfico 209. Unir hilos 166
Gráfico 210. Pila con métodos sincronizados 167
Gráfico 211. Sincronización de hilos 168
Gráfico 212. Ejemplo Productor 169
Gráfico 213. Ejemplo Consumidor 170
Gráfico 214. Pila sincronizada 171
Gráfico 215. Clase principal 172
Gráfico 216. Ejemplo conexión con sockets 173
Gráfico 217. Modelo cliente - servidor 174
Gráfico 218. Fragmento ejemplo Servidor 175
Gráfico 219. Fragmento ejemplo cliente 176
Gráfico 220. Esquema de conexión 177
Gráfico 221. Conexión a través de ODBC 178
Gráfico 222. Conexión a través de Librería nativa 179
Gráfico 223. Conexión a través de protocolo nativo 180
Gráfico 224. Fragmento para abrir una conexión a Derby 181
Gráfico 225. Fragmento para cerrar la conexión 181
Gráfico 226. Ejemplo executeUpdate 182
Gráfico 227. Ejemplo executeQuery 182
Gráfico 228. Ejemplo PreparedStatement con executeUpdate 183
Gráfico 229. Ejemplo PreparedStatement con executeQuery 183
Gráfico 230. Ejemplo CallableStatement con executeQuery 184
Gráfico 231. Procesar resultados 185
Gráfico 232. Obtención de los metadatos de la consulta 186
Gráfico 233. Eventos de boton 187
Gráfico 234. Eventos con Lambda 188
Gráfico 235. Recorrido de colecciones con stream() 188

Gráfico 236. Métodos de referencia 189
Gráfico 237. Clase anónima 189
Gráfico 238. Interfaces funcionales 189
Gráfico 239. Método por defecto 190
Gráfico 240. Método estático en interfaces 190